```python
"""Rule-based failure tagging for failure-case clustering (RQ4).

Taxonomy (mutually exclusive, applied in order):
  success          — answered correctly
  no_answer        — empty/refusal output
  retrieval_miss   — NO gold evidence step made it into the read context
  partial_retrieval— some but not all gold evidence retrieved, answer wrong
  stale_fact       — (Mode B) answered with an OLDER planted value for the fact
  reasoning_failure— all evidence was in context, answer still wrong
"""

from __future__ import annotations


def tag_failure(
    success: bool,
    prediction: str,
    evidence_recall: float | None,
    stale_hit: bool = False,
) -> str:
    if success:
        return "success"
    if not prediction or not prediction.strip() or "i don't know" in prediction.lower():
        return "no_answer"
    if stale_hit:
        return "stale_fact"
    if evidence_recall is None:
        return "reasoning_failure"
    if evidence_recall == 0.0:
        return "retrieval_miss"
    if evidence_recall < 1.0:
        return "partial_retrieval"
    return "reasoning_failure"
```

```python
"""Answer grading.

Mode B: programmatic (grade via tasks.custom_ds.grade_answer) — no judge.
Mode A (LongMemEval): normalised-containment match by default; optional
LLM-as-judge (fixed judge model, frozen rubric) for free-form answers. Judge
calls are logged with their own tracker and NEVER charged to the condition.
"""

from __future__ import annotations

import re

from agentmem.llm.client import LLMClient

JUDGE_RUBRIC = (
    "You are grading a model answer against a gold answer.\n"
    "QUESTION: {question}\nGOLD ANSWER: {gold}\nMODEL ANSWER: {prediction}\n\n"
    "Is the model answer factually correct and consistent with the gold "
    "answer? Reply with exactly one word: CORRECT or INCORRECT."
)


def _norm(s: str) -> str:
    s = s.lower().strip()
    s = re.sub(r"[^\w\s]", " ", s)
    return " ".join(s.split())


def exact_match(prediction: str, gold: str) -> bool:
    """Normalised containment either way (LongMemEval golds are short spans)."""
    p, g = _norm(prediction), _norm(gold)
    if not g:
        return False
    return g in p or (bool(p) and p in g)


class Judge:
    def __init__(self, llm: LLMClient | None = None):
        """llm=None -> pure string matching (pilot / mock mode)."""
        self.llm = llm
        self.calls = 0

    def grade(self, question: str, prediction: str, gold: str) -> bool:
        if exact_match(prediction, gold):
            return True
        if self.llm is None:
            return False
        resp = self.llm.generate(
            [{"role": "user", "content": JUDGE_RUBRIC.format(
                question=question, gold=gold, prediction=prediction)}],
            max_tokens=5,
        )
        self.calls += 1
        return resp.text.strip().upper().startswith("CORRECT")
```

```python
"""Memory-recall accuracy from read() provenance — no extra LLM calls needed.

evidence_recall    = |gold evidence steps retrieved| / |gold evidence steps|
evidence_precision = |retrieved steps that are gold| / |retrieved steps|

Provenance entries may reference a single step ("step": int) or several
("steps": [ints]) for summaries/facts — both count as retrieving those steps.
"""

from __future__ import annotations


def provenance_steps(provenance: list[dict]) -> set[int]:
    steps: set[int] = set()
    for p in provenance:
        if isinstance(p.get("step"), int) and p["step"] >= 0:
            steps.add(p["step"])
        for s in p.get("steps", []) or []:
            if isinstance(s, int) and s >= 0:
                steps.add(s)
    return steps


def evidence_metrics(provenance: list[dict], evidence_steps: list[int]) -> dict:
    retrieved = provenance_steps(provenance)
    gold = set(evidence_steps)
    if not gold:
        return {"evidence_recall": None, "evidence_precision": None,
                "n_retrieved": len(retrieved)}
    hit = retrieved & gold
    return {
        "evidence_recall": len(hit) / len(gold),
        "evidence_precision": (len(hit) / len(retrieved)) if retrieved else 0.0,
        "n_retrieved": len(retrieved),
    }
```

```
"""The single prompt template. IDENTICAL across all conditions — only the
{memory} slot content differs (that content is what each architecture chose
to spend the shared budget on). Changing this template invalidates
cross-condition comparability; version it if it must change.
"""

# CERES serves vLLM under Python 3.8, where PEP 585 generics (list[dict]) and
# PEP 604 unions (dict | None) raise TypeError at import time. This makes all
# annotations lazy strings so the module imports on 3.8. Do not remove.
from __future__ import annotations

PROMPT_VERSION = "v1.0"

SYSTEM_PROMPT = (
    "You are a helpful assistant. Answer the question using the MEMORY "
    "section below. If the memory contains multiple values for the same "
    "fact, the most recent one is correct. Answer concisely."
)

USER_TEMPLATE = """MEMORY:
{memory}

QUESTION:
{question}

Answer concisely:"""


def build_messages(memory_text: str, question: str) -> list[dict]:
    return [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": USER_TEMPLATE.format(
            memory=memory_text or "(empty)", question=question)},
    ]
```

```python
"""Run loops for both evaluation modes.

Mode A (memory probing, LongMemEval): ingest the full history into the memory,
then ask the probe question with a budget-limited read.

Mode B (memory use during execution, custom task): stream observations into
memory chronologically; at each query step, read + answer BEFORE continuing.

Both emit one JSONL record per question (RESEARCH_SCHEMA, trajectory.py).
DP1: every condition reads with the same budget_tokens from configs/base.yaml.
"""

from __future__ import annotations

import time

from agentmem.eval.failures import tag_failure
from agentmem.eval.judge import Judge
from agentmem.eval.recall import evidence_metrics
from agentmem.harness.prompts import PROMPT_VERSION, build_messages
from agentmem.llm.client import LLMClient
from agentmem.logging.trajectory import TrajectoryWriter
from agentmem.memory import make_memory
from agentmem.tasks.custom_ds import ModeBEpisode, grade_answer
from agentmem.tasks.longmemeval import ModeAItem
from agentmem.utils.seeds import derive_seed, set_seed


def _base_record(mode: str, task: str, condition: str, item_id: str, seed: int) -> dict:
    return {
        "run_id": f"{task}-{condition}-{item_id}-s{seed}",
        "mode": mode, "task": task, "condition": condition,
        "item_id": item_id, "seed": seed, "prompt_version": PROMPT_VERSION,
    }


def _meter(llm: LLMClient) -> tuple[int, int, int]:
    """Cumulative (input, output, calls) on the shared client, including the
    write-path/overhead calls that _ask() cannot see. Deltas of this are the
    ONLY figures that satisfy Ch3's definition of token cost as 'total prompt
    and completion tokens per item'."""
    return (llm.cost.input_tokens, llm.cost.output_tokens, llm.cost.calls)


def _billed(before: tuple[int, int, int], after: tuple[int, int, int]) -> dict:
    return {
        "total_prompt_tokens": after[0] - before[0],
        "total_completion_tokens": after[1] - before[1],
        "total_llm_calls": after[2] - before[2],
    }


def _ask(memory, llm: LLMClient, question: str, budget: int) -> tuple[str, dict, dict]:
    """One budget-limited read + one generation. Returns (answer, ctx_info, cost)."""
    t0 = time.perf_counter()
    ctx = memory.read(question, budget_tokens=budget)
    assert ctx.tokens <= budget, f"DP1 violated: read {ctx.tokens} > budget {budget}"
    resp = llm.generate(build_messages(ctx.text, question))
    latency_ms = (time.perf_counter() - t0) * 1000
    cost = {
        "prompt_tokens": resp.input_tokens,
        "completion_tokens": resp.output_tokens,
        "memory_read_tokens": ctx.tokens,
        "latency_ms": round(latency_ms, 1),
    }
    return resp.text.strip(), {"provenance": ctx.provenance}, cost


def run_mode_a(
    items: list[ModeAItem],
    condition: str,
    llm: LLMClient,
    config: dict,
    writer: TrajectoryWriter,
    seed: int = 1,
    judge: Judge | None = None,
    condition_params: dict | None = None,
) -> int:
```

```
     """Returns number of new records written (skips already-done, resumable)."""
     budget = int(config["budgets"]["memory_read_tokens"])
     judge = judge or Judge()
     written = 0
     for item in items:
         if writer.is_done(condition, item.item_id, seed, "A"):
             continue
         set_seed(derive_seed(seed, condition, item.item_id))
         memory = make_memory(condition, condition_params, llm=llm)
         memory.reset()
         meter0 = _meter(llm)          # opens BEFORE ingest: catches write-path calls
         for turn in item.turns:
             memory.write(turn)
         answer, ctx_info, cost = _ask(memory, llm, item.question, budget)
         success = judge.grade(item.question, answer, item.answer)
         rec_m = evidence_metrics(ctx_info["provenance"], item.evidence_steps)
         stats = memory.stats()
         rec = {
             **_base_record("A", "longmemeval", condition, item.item_id, seed),
             "question_type": item.question_type,
             "question": item.question, "gold": item.answer, "prediction": answer,
             "success": bool(success),
             "failure_tag": tag_failure(success, answer, rec_m["evidence_recall"]),
             **rec_m,
             **cost,
             **_billed(meter0, _meter(llm)),
             "memory_written_tokens": stats.tokens_written,
             "overhead_calls": stats.overhead_calls,
             "memory_items": stats.items,
             "gbp_spent": round(llm.cost.gbp_spent, 6),
             "provenance": ctx_info["provenance"],  # FULL (was [:20] — truncation broke ordering
 audits)
         }
         writer.write(rec)
         written += 1
     return written


 def _stale_hit(episode: ModeBEpisode, query, prediction: str) -> bool:
     """Did the model answer with an OLDER planted value for this fact?"""
     older_values = [
         t.metadata["value"] for t in episode.turns
         if t.metadata.get("kind") == "fact"
         and t.metadata.get("project") == query.project
         and t.metadata.get("attr") == query.attr
         and t.step < query.step
         and t.metadata["value"] != query.answer
     ]
     return any(grade_answer(prediction, v) for v in older_values)


 def run_mode_b(
     episodes: list[ModeBEpisode],
     condition: str,
     llm: LLMClient,
     config: dict,
     writer: TrajectoryWriter,
     seed: int = 1,
     condition_params: dict | None = None,
 ) -> int:
     budget = int(config["budgets"]["memory_read_tokens"])
     written = 0
     for ep in episodes:
         memory = make_memory(condition, condition_params, llm=llm)
         memory.reset()
         set_seed(derive_seed(seed, condition, ep.episode_id))
         queries_by_step = {q.step: q for q in ep.queries}
         pending = [q for q in ep.queries
                    if not writer.is_done(condition, f"{ep.episode_id}:q{q.step}", seed, "B")]
         if not pending:
             continue
         # Overhead accrued since the previous query is amortised onto this query,
         # so the per-episode sum of total_prompt_tokens is the true episode spend.
         meter0 = _meter(llm)
         for turn in ep.turns:
             q = queries_by_step.get(turn.step)
             if q is not None:
                 answer, ctx_info, cost = _ask(memory, llm, q.question, budget)
```

```
                success = grade_answer(answer, q.answer)
                rec_m = evidence_metrics(ctx_info["provenance"], q.evidence_steps)
                stale = (not success) and _stale_hit(ep, q, answer)
                stats = memory.stats()
                rec = {
                    **_base_record("B", "custom_ds", condition,
                                   f"{ep.episode_id}:q{q.step}", seed),
                    "question": q.question, "gold": q.answer, "prediction": answer,
                    "success": bool(success),
                    "failure_tag": tag_failure(success, answer,
                                               rec_m["evidence_recall"], stale_hit=stale),
                    "attr": q.attr, "evidence_distance": q.distance,
                    **rec_m,
                    **cost,
                    **_billed(meter0, _meter(llm)),
                    "memory_written_tokens": stats.tokens_written,
                    "overhead_calls": stats.overhead_calls,
                    "memory_items": stats.items,
                    "gbp_spent": round(llm.cost.gbp_spent, 6),
                    "provenance": ctx_info["provenance"],  # FULL (was [:20] — truncation broke
    ordering audits)
                }
                writer.write(rec)
                written += 1
                meter0 = _meter(llm)   # reset the window for the next query
            memory.write(turn)  # observation always enters memory (after any query)
    return written
```

```
"""Fixed-model LLM wrapper: one client, swappable backend, tokens/GBP/retry/kill-switch.

Backends
--------
- MockLLM              deterministic, zero-cost; develops/tests the whole pipeline on CPU.
- OpenAICompatibleLLM  any /chat/completions endpoint: vLLM on university HPC
                       (primary), Groq, OpenRouter (fallbacks). Base URL + key
                       come from env (.env), never from code.

The model is FIXED across all conditions (design invariant): the factory reads
it once from config; conditions cannot override it.
"""

from __future__ import annotations

import os
import re
import time
from abc import ABC, abstractmethod
from dataclasses import dataclass

from agentmem.utils.costs import CostTracker
from agentmem.utils.tokens import count_tokens


@dataclass
class LLMResponse:
    text: str
    input_tokens: int
    output_tokens: int
    latency_ms: float


class LLMClient(ABC):
    """generate() is the ONLY way any component talks to the model."""

    def __init__(self, model: str, cost_tracker: CostTracker | None = None):
        self.model = model
        self.cost = cost_tracker or CostTracker(price_key="mock")

    @abstractmethod
    def _complete(self, messages: list[dict], temperature: float, max_tokens: int) -> str: ...

    def generate(
        self, messages: list[dict], temperature: float = 0.0, max_tokens: int = 512
    ) -> LLMResponse:
        prompt_text = "\n".join(m["content"] for m in messages)
        t0 = time.perf_counter()
        text = self._complete(messages, temperature, max_tokens)
        latency_ms = (time.perf_counter() - t0) * 1000
        resp = LLMResponse(
            text=text,
            input_tokens=count_tokens(prompt_text),
            output_tokens=count_tokens(text),
            latency_ms=latency_ms,
        )
        self.cost.add(resp.input_tokens, resp.output_tokens)  # kill-switch lives here
        return resp


class MockLLM(LLMClient):
    """Deterministic stand-in with just enough behaviour to exercise the pipeline.

    - summarise prompts  -> keeps lines containing digits/proper nouns (facts survive)
    - extract prompts    -> emits '- fact' lines for fact-bearing sentences
    - answer prompts     -> returns the context line with max word overlap with the
                            question (so answer quality really depends on what the
                            memory architecture put in the context — conditions
                            differentiate even in mock mode)
    """

    def __init__(self, model: str = "mock"):
        super().__init__(model, CostTracker(price_key="mock", gbp_cap=float("inf")))

    def _complete(self, messages: list[dict], temperature: float, max_tokens: int) -> str:
        user = messages[-1]["content"]
        low = user.lower()
        if "summarize" in low or "summarise" in low:
```

```
            return self._keep_facts(user)
        if "extract" in low and "fact" in low:
            return self._extract_facts(user)
        return self._answer(user)

    @staticmethod
    def _turn_lines(text: str) -> list[str]:
        """Content lines look like '[role step N] ...' — strip the bracket prefix."""
        out = []
        for ln in text.splitlines():
            ln = ln.strip()
            if ln.startswith("[") and "]" in ln:
                body = ln.split("]", 1)[1].strip()
                if body and re.search(r"\d|[A-Z][a-z]+ [A-Z][a-z]+|:", body):
                    out.append(body)
        return out

    def _keep_facts(self, text: str) -> str:
        return " ".join(self._turn_lines(text)[:40])[:2000] or "No salient facts."

    def _extract_facts(self, text: str) -> str:
        lines = self._turn_lines(text)
        return "\n".join(f"- {ln}" for ln in lines[:30]) or "- none"

    @staticmethod
    def _words(s: str) -> set[str]:
        return set(re.findall(r"[a-z0-9]+", s.lower()))

    def _answer(self, prompt: str) -> str:
        # Question = last line ending in '?'; context = everything else.
        lines = [ln.strip() for ln in prompt.splitlines() if ln.strip()]
        q_lines = [ln for ln in lines if ln.endswith("?")]
        question = q_lines[-1] if q_lines else (lines[-1] if lines else "")
        qw = self._words(question) - {"what", "which", "when", "where", "who", "how",
                                      "the", "a", "an", "is", "was", "did", "does", "of"}
        best, best_score = "I don't know.", 0
        for ln in lines:
            if ln == question:
                continue
            score = len(qw & self._words(ln))
            if score > best_score:
                best, best_score = ln, score
        return best


class OpenAICompatibleLLM(LLMClient):
    """Any OpenAI-compatible /chat/completions server (vLLM on HPC, Groq, OpenRouter)."""

    def __init__(
        self,
        model: str,
        base_url: str,
        api_key: str = "",
        cost_tracker: CostTracker | None = None,
        max_retries: int = 4,
        timeout_s: int = 120,
    ):
        super().__init__(model, cost_tracker)
        self.base_url = base_url.rstrip("/")
        self.api_key = api_key
        self.max_retries = max_retries
        self.timeout_s = timeout_s

    def _complete(self, messages: list[dict], temperature: float, max_tokens: int) -> str:
        import requests  # local import: mock mode needs no network stack

        headers = {"Content-Type": "application/json"}
        if self.api_key:
            headers["Authorization"] = f"Bearer {self.api_key}"
        payload = {
            "model": self.model,
            "messages": messages,
            "temperature": temperature,
            "max_tokens": max_tokens,
        }
        last_err: Exception | None = None
        for attempt in range(self.max_retries):
            try:
```

```
                r = requests.post(
                    f"{self.base_url}/chat/completions",
                    json=payload, headers=headers, timeout=self.timeout_s,
                )
                if r.status_code == 429 or r.status_code >= 500:
                    raise RuntimeError(f"HTTP {r.status_code}: {r.text[:200]}")
                r.raise_for_status()
                return r.json()["choices"][0]["message"]["content"]
            except Exception as e:  # noqa: BLE001 — retry then surface
                last_err = e
                time.sleep(2**attempt)
        raise RuntimeError(f"LLM call failed after {self.max_retries} retries: {last_err}")


def make_llm(config: dict) -> LLMClient:
    """Factory. provider: mock | university_hpc | groq | openrouter."""
    provider = config.get("provider", "mock")
    model = config.get("model", "") or "mock"
    if provider == "mock":
        return MockLLM()

    cap = float(config.get("budgets", {}).get("gbp_cap", 30))
    tracker = CostTracker(price_key=f"{provider}/{model}"
                          if provider != "university_hpc" else "university_hpc",
                          gbp_cap=cap)
    if provider == "university_hpc":
        base_url = os.environ.get("HPC_BASE_URL", "http://localhost:8000/v1")
        api_key = os.environ.get("HPC_API_KEY", "")
    elif provider == "groq":
        base_url = "https://api.groq.com/openai/v1"
        api_key = os.environ["GROQ_API_KEY"]
    elif provider == "openrouter":
        base_url = "https://openrouter.ai/api/v1"
        api_key = os.environ["OPENROUTER_API_KEY"]
    else:
        raise ValueError(f"Unknown provider: {provider}")
    return OpenAICompatibleLLM(model, base_url, api_key, tracker)
```

```python
"""JSONL run records: one line per (item|query) x condition x seed.

RESEARCH_SCHEMA — every record carries:
  identity   : run_id, mode, task, condition, item_id, seed
  outcome    : question, gold, prediction, success, failure_tag
  memory     : evidence_recall, evidence_precision, provenance (truncated)
  cost       : prompt_tokens, completion_tokens, memory_read_tokens,
               memory_written_tokens, overhead_calls, latency_ms, gbp_spent
Resumable: existing (condition, item_id, seed, mode) keys are skipped on rerun.
"""

from __future__ import annotations

import json
from pathlib import Path


def record_key(rec: dict) -> tuple:
    return (rec["condition"], rec["item_id"], rec["seed"], rec["mode"])


class TrajectoryWriter:
    def __init__(self, path: str | Path):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self._done: set[tuple] = set()
        if self.path.exists():
            for line in self.path.read_text(encoding="utf-8").splitlines():
                if line.strip():
                    try:
                        self._done.add(record_key(json.loads(line)))
                    except (json.JSONDecodeError, KeyError):
                        continue

    def is_done(self, condition: str, item_id: str, seed: int, mode: str) -> bool:
        return (condition, item_id, seed, mode) in self._done

    def write(self, rec: dict) -> None:
        with self.path.open("a", encoding="utf-8") as f:
            f.write(json.dumps(rec, ensure_ascii=False, default=str) + "\n")
        self._done.add(record_key(rec))


def load_records(path: str | Path) -> list[dict]:
    p = Path(path)
    if not p.exists():
        return []
    return [json.loads(ln) for ln in p.read_text(encoding="utf-8").splitlines()
            if ln.strip()]
```

```python
"""Memory package: locked ABC + the five conditions + factory."""

# CERES serves vLLM under Python 3.8, where PEP 585 generics (list[dict]) and
# PEP 604 unions (dict | None) raise TypeError at import time. This makes all
# annotations lazy strings so the module imports on 3.8. Do not remove.
from __future__ import annotations

from agentmem.memory.base import Memory, MemoryContext, MemoryStats, Turn

CONDITIONS = ("b", "m1", "m2", "m3", "m4")

# Control arms: run on demand (--condition b_reversed), NOT part of the main
# grid, so --all-conditions and the registered analysis stay as pre-specified.
CONTROL_ARMS = ("b_reversed", "m1_chrono", "m3_chrono", "m4_chrono")
ALL_ARMS = CONDITIONS + CONTROL_ARMS


def make_memory(condition: str, config: dict | None = None, llm=None) -> Memory:
    """Factory: condition id -> Memory instance.

    b  = NoMemory (recency window)      m1 = EpisodicMemory (retrieval)
    m2 = SummaryMemory (hierarchical)   m3 = SemanticMemory (fact store)
    m4 = DecayMemory (retrieval+decay)

    b_reversed = NoMemory with the chronological re-sort disabled. Ordering-only
    control: identical turn selection to b, reverse-chronological prompt layout.
    """
    from agentmem.memory.decay import DecayMemory
    from agentmem.memory.episodic import EpisodicMemory
    from agentmem.memory.none import NoMemory
    from agentmem.memory.semantic import SemanticMemory
    from agentmem.memory.summary import SummaryMemory

    cfg = dict(config or {})
    condition = condition.lower()
    if condition == "b":
        return NoMemory(**cfg)
    if condition == "b_reversed":
        cfg.setdefault("chronological_order", False)
        return NoMemory(**cfg)
    if condition == "m1":
        return EpisodicMemory(**cfg)
    if condition == "m1_chrono":
        cfg.setdefault("chronological_order", True)
        return EpisodicMemory(**cfg)
    if condition == "m2":
        return SummaryMemory(llm=llm, **cfg)
    if condition == "m3":
        return SemanticMemory(llm=llm, **cfg)
    if condition == "m3_chrono":
        cfg.setdefault("chronological_order", True)
        return SemanticMemory(llm=llm, **cfg)
    if condition == "m4":
        return DecayMemory(**cfg)
    if condition == "m4_chrono":
        cfg.setdefault("chronological_order", True)
        return DecayMemory(**cfg)
    raise ValueError(f"Unknown condition: {condition!r} (expected one of {ALL_ARMS})")


__all__ = ["Memory", "MemoryContext", "MemoryStats", "Turn", "make_memory",
           "CONDITIONS", "CONTROL_ARMS", "ALL_ARMS"]
```

```python
"""The memory abstraction — the ONLY thing that varies between conditions.

Locked interface from blueprint §6.3. `read()`'s `budget_tokens` parameter IS
the fairness mechanism (DP1 Equal-Budget Principle); provenance powers the
recall-accuracy metric for free.

Do not implement subclasses until the design is signed off (18 Jun meeting).
Implementations live in: none.py, episodic.py, summary.py, semantic.py, decay.py
"""

from abc import ABC, abstractmethod
from dataclasses import dataclass, field


@dataclass
class Turn:
    """One step/observation in an episode (schema finalised in W2)."""
    role: str
    content: str
    step: int
    metadata: dict = field(default_factory=dict)


@dataclass
class MemoryContext:
    """What read() returns: text fitted to budget + provenance."""
    text: str
    tokens: int
    provenance: list  # which items were injected, with scores


@dataclass
class MemoryStats:
    tokens_written: int = 0
    tokens_read: int = 0
    items: int = 0
    overhead_calls: int = 0
    latency_ms: float = 0.0


class Memory(ABC):
    @abstractmethod
    def write(self, turn: Turn) -> None:
        """Store a step/observation."""

    @abstractmethod
    def read(self, query: str, budget_tokens: int) -> MemoryContext:
        """Return text fitted to budget + provenance (which items, scores)."""

    @abstractmethod
    def reset(self) -> None:
        """Fresh state per item/episode."""

    @abstractmethod
    def stats(self) -> MemoryStats:
        """tokens_written/read, items, overhead_calls, latency_ms."""
```

```python
"""M4 (Decay): M1 retrieval with the Generative-Agents scoring formula
    score = relevance * exp(-lambda * age)
where age is measured in steps since the turn was written. Isolates the value
of recency-weighting ON TOP of pure relevance (M1 is the ablation partner).
"""

from __future__ import annotations

import math
import time

import numpy as np

from agentmem.memory.base import Memory, MemoryContext, MemoryStats, Turn
from agentmem.memory.retrieval import (
    fit_to_budget,
    make_scorer,
    reorder_chronologically,
)
from agentmem.utils.tokens import count_tokens


class DecayMemory(Memory):
    def __init__(self, retriever: str = "tfidf", decay_lambda: float = 0.01,
                 chronological_order: bool = False, **_kwargs):
        self._scorer = make_scorer(retriever)
        self._lambda = decay_lambda
        # OFF by default (preserves the reported M4 results). NOTE: with this
        # off, equally-relevant turns are ranked recency-first, so M4's prompt
        # tends REVERSE-chronological — the same layout that cost M2 11.9pp.
        # The M4 deficit is therefore ordering-confounded; see §4.6.
        self._chronological = bool(chronological_order)
        self._turns: list[Turn] = []
        self._clock = 0  # current step count (advances on write)
        self._stats = MemoryStats()

    def write(self, turn: Turn) -> None:
        self._turns.append(turn)
        self._clock = max(self._clock, turn.step)
        self._stats.items += 1
        self._stats.tokens_written += count_tokens(turn.content)

    def read(self, query: str, budget_tokens: int) -> MemoryContext:
        t0 = time.perf_counter()
        rel = self._scorer.score(query, [t.content for t in self._turns])
        combined = np.array([
            float(r) * math.exp(-self._lambda * max(0, self._clock - t.step))
            for r, t in zip(rel, self._turns)
        ])
        order = np.argsort(-combined)
        items = [
            (self._turns[i].content,
             {"step": self._turns[i].step, "role": self._turns[i].role,
              "kind": "raw_turn", "relevance": round(float(rel[i]), 4)},
             float(combined[i]))
            for i in order
        ]
        ctx = fit_to_budget(items, budget_tokens)
        if self._chronological:
            ctx = reorder_chronologically(ctx)
        self._stats.tokens_read += ctx.tokens
        self._stats.latency_ms += (time.perf_counter() - t0) * 1000
        return ctx

    def reset(self) -> None:
        self._turns = []
        self._clock = 0
        self._stats = MemoryStats()

    def stats(self) -> MemoryStats:
        return self._stats
```

```python
"""M1 (Episodic/retrieval): store every turn verbatim; read = top-k similarity
fitted to budget.

Scorer is TF-IDF by default (CPU-safe); switch to sentence-transformer
embeddings with config `retriever: embedding` on HPC. The architecture claim
(selective retrieval beats recency) is scorer-agnostic; report which was used.
"""

from __future__ import annotations

import time

import numpy as np

from agentmem.memory.base import Memory, MemoryContext, MemoryStats, Turn
from agentmem.memory.retrieval import (
    fit_to_budget,
    make_scorer,
    reorder_chronologically,
)
from agentmem.utils.tokens import count_tokens


class EpisodicMemory(Memory):
    def __init__(self, retriever: str = "tfidf", chronological_order: bool = False,
                 **_kwargs):
        self._scorer = make_scorer(retriever)
        self._retriever_name = retriever
        # OFF by default: retrieval memories have always emitted score-ordered
        # prompts and the reported M1 results were produced that way. See
        # reorder_chronologically() for why you may want to turn this on.
        self._chronological = bool(chronological_order)
        self._turns: list[Turn] = []
        self._stats = MemoryStats()

    def write(self, turn: Turn) -> None:
        self._turns.append(turn)
        self._stats.items += 1
        self._stats.tokens_written += count_tokens(turn.content)

    def _rank(self, query: str) -> list[tuple[Turn, float]]:
        scores = self._scorer.score(query, [t.content for t in self._turns])
        order = np.argsort(-scores)
        return [(self._turns[i], float(scores[i])) for i in order]

    def read(self, query: str, budget_tokens: int) -> MemoryContext:
        t0 = time.perf_counter()
        ranked = self._rank(query)
        items = [
            (t.content, {"step": t.step, "role": t.role, "kind": "raw_turn"}, s)
            for t, s in ranked
        ]
        ctx = fit_to_budget(items, budget_tokens)
        if self._chronological:
            ctx = reorder_chronologically(ctx)
        self._stats.tokens_read += ctx.tokens
        self._stats.latency_ms += (time.perf_counter() - t0) * 1000
        return ctx

    def reset(self) -> None:
        self._turns = []
        self._stats = MemoryStats()

    def stats(self) -> MemoryStats:
        return self._stats
```

```python
"""B (NoMemory baseline): sliding window of the MOST RECENT turns fitting the budget.

Built first because it defines fairness: every other condition gets exactly the
same read budget — they just choose differently what to spend it on.
"""

from __future__ import annotations

import time

from agentmem.memory.base import Memory, MemoryContext, MemoryStats, Turn
from agentmem.memory.retrieval import fit_to_budget
from agentmem.utils.tokens import count_tokens


class NoMemory(Memory):
    """Recency-window baseline.

    `chronological_order` exists to support the ordering-only control arm
    (`b_reversed`). Selection is always recency-first so the same turns win the
    budget under both settings; the flag changes ONLY the order in which those
    turns are laid out in the prompt. That isolates prompt ordering from memory
    architecture — see configs/conditions/b_reversed.yaml.
    """

    def __init__(self, chronological_order: bool = True, **_kwargs):
        self._turns: list[Turn] = []
        self._stats = MemoryStats()
        self._chronological = bool(chronological_order)

    def write(self, turn: Turn) -> None:
        self._turns.append(turn)
        self._stats.items += 1
        self._stats.tokens_written += count_tokens(turn.content)

    def read(self, query: str, budget_tokens: int) -> MemoryContext:
        t0 = time.perf_counter()
        # Most recent first so recency wins the budget; restore chronology after.
        recent = list(reversed(self._turns))
        items = [
            (t.content, {"step": t.step, "role": t.role, "kind": "raw_turn"}, 1.0)
            for t in recent
        ]
        ctx = fit_to_budget(items, budget_tokens)
        # Re-order chosen text chronologically for the prompt (unless the
        # ordering-control arm has asked for the reverse-chronological layout).
        chosen_steps = {p["step"] for p in ctx.provenance}
        source = self._turns if self._chronological else recent
        ordered = [t.content for t in source if t.step in chosen_steps]
        provenance = sorted(ctx.provenance, key=lambda p: p["step"],
                            reverse=not self._chronological)
        ctx = MemoryContext(text="\n".join(ordered), tokens=ctx.tokens,
                            provenance=provenance)
        self._stats.tokens_read += ctx.tokens
        self._stats.latency_ms += (time.perf_counter() - t0) * 1000
        return ctx

    def reset(self) -> None:
        self._turns = []
        self._stats = MemoryStats()

    def stats(self) -> MemoryStats:
        return self._stats
```

```python
"""Shared retrieval machinery for M1/M3/M4.

Default scorer is TF-IDF cosine (numpy only — runs on the CPU laptop with zero
model downloads). An optional sentence-transformers backend can be enabled via
config `retriever: embedding` once on HPC; both expose .score(query, texts).

Also home of fit_to_budget(), the single DP1 enforcement point for
retrieval-style memories: items are admitted in score order until the token
budget is exhausted. Every memory read() funnels through budget fitting.
"""

from __future__ import annotations

import math
import re
from collections import Counter

import numpy as np

from agentmem.memory.base import MemoryContext
from agentmem.utils.tokens import count_tokens

_TOKEN_RE = re.compile(r"[a-z0-9]+")


def _terms(text: str) -> list[str]:
    return _TOKEN_RE.findall(text.lower())


class TfidfScorer:
    """Cosine similarity over TF-IDF vectors; rebuilt lazily on demand."""

    def score(self, query: str, texts: list[str]) -> np.ndarray:
        if not texts:
            return np.zeros(0)
        docs = [Counter(_terms(t)) for t in texts]
        q = Counter(_terms(query))
        vocab = set(q)
        for d in docs:
            vocab |= set(d)
        vocab_list = sorted(vocab)
        n = len(texts)
        df = Counter()
        for d in docs:
            df.update(set(d))
        idf = {t: math.log((1 + n) / (1 + df[t])) + 1 for t in vocab_list}

        def vec(c: Counter) -> np.ndarray:
            v = np.array([c.get(t, 0) * idf[t] for t in vocab_list], dtype=float)
            norm = np.linalg.norm(v)
            return v / norm if norm > 0 else v

        qv = vec(q)
        return np.array([float(vec(d) @ qv) for d in docs])


class EmbeddingScorer:
    """sentence-transformers backend (HPC / optional). Lazy model load."""

    def __init__(self, model_name: str = "sentence-transformers/all-MiniLM-L6-v2"):
        from sentence_transformers import SentenceTransformer  # heavy import

        self._model = SentenceTransformer(model_name)

    def score(self, query: str, texts: list[str]) -> np.ndarray:
        if not texts:
            return np.zeros(0)
        embs = self._model.encode([query] + texts, normalize_embeddings=True)
        return embs[1:] @ embs[0]


def make_scorer(name: str = "tfidf"):
    if name == "embedding":
        return EmbeddingScorer()
    return TfidfScorer()


def fit_to_budget(
```

```
    items: list[tuple[str, dict, float]], budget_tokens: int, sep: str = "\n"
) -> MemoryContext:
    """Admit (text, meta, score) items in given order until budget is spent.

    THE DP1 enforcement point: no memory may return more than budget_tokens.
    """
    chosen: list[str] = []
    provenance: list[dict] = []
    used = 0
    for text, meta, score in items:
        t = count_tokens(text) + (count_tokens(sep) if chosen else 0)
        if used + t > budget_tokens:
            continue  # try smaller later items rather than stopping dead
        chosen.append(text)
        used += t
        provenance.append({**meta, "score": round(float(score), 4)})
    return MemoryContext(text=sep.join(chosen), tokens=used, provenance=provenance)


def reorder_chronologically(ctx: MemoryContext, sep: str = "\n") -> MemoryContext:
    """Re-lay a fitted context oldest-first WITHOUT changing what was selected.

    Selection stays score-first (so the highest-scoring turns still win the
    budget) and only the prompt layout changes.

    WHY THIS EXISTS. Retrieval-style memories (M1, M4) emit turns in score
    order, so their prompts are neither chronological nor consistently ordered,
    while B and M2 emit chronologically. That is the same class of confound that
    made M2 look like a harmful architecture (see B1 in the expert review):
    a contrast between a score-ordered and a chronologically-ordered condition
    mixes architecture with presentation. M4 is the sharpest case — among
    equally relevant turns its decay term ranks by recency, producing a
    reverse-chronological prompt, exactly the layout that cost M2 11.9pp.

    Left OFF by default so existing results stay valid. Turn it on per condition
    via `chronological_order: true` and pair it with the b_reversed control arm.
    """
    steps = [p.get("step") for p in ctx.provenance]
    if any(s is None for s in steps) or steps == sorted(steps):
        return ctx
    order = sorted(range(len(ctx.provenance)), key=lambda i: ctx.provenance[i]["step"])
    parts = ctx.text.split(sep)
    if len(parts) != len(ctx.provenance):
        return ctx  # summary blocks or separators inside text: leave alone
    return MemoryContext(
        text=sep.join(parts[i] for i in order),
        tokens=ctx.tokens,
        provenance=[ctx.provenance[i] for i in order],
    )
```

```python
"""M3 (Structured semantic): the fixed LLM extracts atomic facts from turns into
a fact store (update-on-conflict, Mem0-style but custom — auditable and free).
read() retrieves the most query-relevant FACTS, not raw turns.

Extraction is batched every `extract_every` turns; each batch = 1 overhead call.
Conflict policy: a new fact whose key (first ~6 content words) matches an
existing fact REPLACES it (newest-wins), mirroring Mem0's update operation.
"""

from __future__ import annotations

import re
import time
from dataclasses import dataclass

import numpy as np

from agentmem.llm.client import LLMClient
from agentmem.memory.base import Memory, MemoryContext, MemoryStats, Turn
from agentmem.memory.retrieval import (
    fit_to_budget,
    make_scorer,
    reorder_chronologically,
)
from agentmem.utils.tokens import count_tokens, truncate_to_tokens

EXTRACT_PROMPT = (
    "Extract the atomic facts from this conversation excerpt. One fact per "
    "line, each starting with '- '. Keep names, numbers, dates, decisions and "
    "preferences exactly. Skip chit-chat.\n\nEXCERPT:\n{chunk}\n\nFACTS:"
)

_STOP = {"the", "a", "an", "is", "was", "are", "were", "to", "of", "and", "in",
         "on", "for", "that", "this", "user", "assistant"}


@dataclass
class Fact:
    text: str
    key: str
    source_steps: list[int]


def _fact_key(text: str) -> str:
    words = [w for w in re.findall(r"[a-z0-9]+", text.lower())
             if w not in _STOP and not any(ch.isdigit() for ch in w)]
    return " ".join(words[:6])


class SemanticMemory(Memory):
    def __init__(
        self,
        llm: LLMClient | None = None,
        retriever: str = "tfidf",
        extract_every: int = 5,
        chronological_order: bool = False,
        **_kwargs,
    ):
        if llm is None:
            raise ValueError("SemanticMemory needs the shared fixed LLM client")
        self._llm = llm
        self._scorer = make_scorer(retriever)
        self._extract_every = extract_every
        # OFF by default: the reported M3 results were produced score-ordered.
        # m3_chrono turns this on: facts are re-laid oldest-first (by earliest
        # source step) after selection, an ordering-only control for M3. Note
        # this repairs LAYOUT only — extraction already merged conflicts at
        # write time, which is why the m3_chrono arm did not recover (0.740
        # vs 0.753; see supervision/ADDENDUM_m3_chrono_2026-07-30.md).
        self._chronological = bool(chronological_order)
        self._facts: list[Fact] = []
        self._pending: list[Turn] = []
        self._stats = MemoryStats()

    def write(self, turn: Turn) -> None:
        self._pending.append(turn)
        self._stats.items += 1
        self._stats.tokens_written += count_tokens(turn.content)
```

```
        if len(self._pending) >= self._extract_every:
            self._extract()

    # Largest excerpt sent to the extractor in ONE call. Mode A (LongMemEval)
    # sessions can be thousands of tokens, so extraction is split into
    # serving-window-sized pieces; each piece is its own overhead call.
    _MAX_EXTRACT_CHUNK_TOKENS = 900

    def _extract(self) -> None:
        if not self._pending:
            return
        steps = [t.step for t in self._pending]
        lines_in = [f"[{t.role} step {t.step}] {t.content}" for t in self._pending]
        self._pending = []

        pieces: list[str] = []
        current: list[str] = []
        current_tokens = 0
        for line_in in lines_in:
            line_tokens = count_tokens(line_in)
            if line_tokens > self._MAX_EXTRACT_CHUNK_TOKENS:
                if current:
                    pieces.append("\n".join(current)); current, current_tokens = [], 0
                pieces.append(truncate_to_tokens(line_in, self._MAX_EXTRACT_CHUNK_TOKENS))
                continue
            if current_tokens + line_tokens > self._MAX_EXTRACT_CHUNK_TOKENS:
                pieces.append("\n".join(current)); current, current_tokens = [], 0
            current.append(line_in); current_tokens += line_tokens
        if current:
            pieces.append("\n".join(current))

        response_lines: list[str] = []
        for piece in pieces:
            resp = self._llm.generate(
                [{"role": "user", "content": EXTRACT_PROMPT.format(chunk=piece)}],
                max_tokens=400,
            )
            self._stats.overhead_calls += 1
            response_lines.extend(resp.text.splitlines())

        for line in response_lines:
            line = line.strip()
            if not line.startswith("- ") or len(line) < 4:
                continue
            text = line[2:].strip()
            key = _fact_key(text)
            if not key:
                continue
            # Update-on-conflict: newest fact with the same key wins.
            existing = next((f for f in self._facts if f.key == key), None)
            if existing:
                existing.text = text
                existing.source_steps = sorted(set(existing.source_steps + steps))
            else:
                self._facts.append(Fact(text=text, key=key, source_steps=steps))

    def read(self, query: str, budget_tokens: int) -> MemoryContext:
        t0 = time.perf_counter()
        self._extract()  # flush pending turns so nothing recent is invisible
        scores = self._scorer.score(query, [f.text for f in self._facts])
        order = np.argsort(-scores) if len(scores) else []
        items = [
            (self._facts[i].text,
             {"kind": "fact", "steps": self._facts[i].source_steps,
              "step": self._facts[i].source_steps[0] if self._facts[i].source_steps else -1},
             float(scores[i]))
            for i in order
        ]
        ctx = fit_to_budget(items, budget_tokens)
        if self._chronological:
            ctx = reorder_chronologically(ctx)
        self._stats.tokens_read += ctx.tokens
        self._stats.latency_ms += (time.perf_counter() - t0) * 1000
        return ctx

    def reset(self) -> None:
        self._facts = []
        self._pending = []
```

```
157          self._stats = MemoryStats()
158
159      def stats(self) -> MemoryStats:
160          return self._stats
```

```python
"""M2 (Summary/hierarchical, MemGPT-style): running summary maintained by the
SAME fixed LLM; oldest raw turns are folded into the summary when the raw
buffer exceeds a threshold. Summarisation calls are charged to the condition
as overhead (overhead_calls + the tracker inside the shared LLM client).

read() returns: [running summary] + [most recent raw turns], fitted to budget.
Query-independent by design — that is the architectural contrast with M1.
"""

from __future__ import annotations

import time

from agentmem.llm.client import LLMClient
from agentmem.memory.base import Memory, MemoryContext, MemoryStats, Turn
from agentmem.memory.retrieval import fit_to_budget
from agentmem.utils.tokens import count_tokens, truncate_to_tokens

SUMMARISE_PROMPT = (
    "Summarize the following conversation excerpt into concise notes, "
    "PRESERVING all concrete facts (names, numbers, dates, decisions, "
    "preferences). Merge with the existing notes.\n\n"
    "EXISTING NOTES:\n{summary}\n\nNEW EXCERPT:\n{chunk}\n\nUPDATED NOTES:"
)


class SummaryMemory(Memory):
    def __init__(
        self,
        llm: LLMClient | None = None,
        buffer_token_threshold: int = 1500,
        fold_fraction: float = 0.5,
        **_kwargs,
    ):
        if llm is None:
            raise ValueError("SummaryMemory needs the shared fixed LLM client")
        self._llm = llm
        self._threshold = buffer_token_threshold
        self._fold_fraction = fold_fraction
        self._summary = ""
        self._summary_sources: list[int] = []  # steps folded into the summary
        self._buffer: list[Turn] = []
        self._stats = MemoryStats()

    def write(self, turn: Turn) -> None:
        self._buffer.append(turn)
        self._stats.items += 1
        self._stats.tokens_written += count_tokens(turn.content)
        if sum(count_tokens(t.content) for t in self._buffer) > self._threshold:
            self._fold_oldest()

    # Largest excerpt sent to the summariser in ONE call. Mode A (LongMemEval)
    # sessions can be thousands of tokens each, so a single fold must be split
    # into serving-window-sized pieces (MemGPT folds recursively for the same
    # reason). Each piece is charged as its own overhead call, so the cost
    # accounting stays honest.
    _MAX_FOLD_CHUNK_TOKENS = 900

    def _fold_oldest(self) -> None:
        n = max(1, int(len(self._buffer) * self._fold_fraction))
        oldest, self._buffer = self._buffer[:n], self._buffer[n:]

        # Build fold pieces that fit the summariser's input window.
        pieces: list[str] = []
        current: list[str] = []
        current_tokens = 0
        for t in oldest:
            line = f"[{t.role} step {t.step}] {t.content}"
            line_tokens = count_tokens(line)
            if line_tokens > self._MAX_FOLD_CHUNK_TOKENS:
                # A single oversized turn becomes its own (truncated) piece.
                if current:
                    pieces.append("\n".join(current)); current, current_tokens = [], 0
                pieces.append(truncate_to_tokens(line, self._MAX_FOLD_CHUNK_TOKENS))
                continue
            if current_tokens + line_tokens > self._MAX_FOLD_CHUNK_TOKENS:
                pieces.append("\n".join(current)); current, current_tokens = [], 0
            current.append(line); current_tokens += line_tokens
```

```
        if current:
            pieces.append("\n".join(current))

        for piece in pieces:
            # Prior notes are kept compact so (notes + excerpt + 400 output)
            # fits comfortably inside the smallest serving window we use.
            prior = truncate_to_tokens(self._summary or "(none)", 500)
            resp = self._llm.generate(
                [{"role": "user",
                  "content": SUMMARISE_PROMPT.format(summary=prior, chunk=piece)}],
                max_tokens=400,
            )
            self._summary = resp.text.strip()
            self._stats.overhead_calls += 1

        self._summary_sources.extend(t.step for t in oldest)

    def read(self, query: str, budget_tokens: int) -> MemoryContext:
        t0 = time.perf_counter()
        items: list[tuple[str, dict, float]] = []
        if self._summary:
            summary_text = truncate_to_tokens(
                f"[MEMORY NOTES]\n{self._summary}", max(0, budget_tokens - 8))
            if summary_text:
                items.append(
                    (summary_text,
                     {"kind": "summary", "steps": list(self._summary_sources),
                      "step": -1},
                     2.0)
                )
        for t in reversed(self._buffer):  # newest raw turns first
            items.append(
                (t.content, {"step": t.step, "role": t.role, "kind": "raw_turn"}, 1.0)
            )
        ctx = fit_to_budget(items, budget_tokens)
        # Selection is recency-first (above), but the PROMPT must be chronological,
        # exactly as NoMemory.read() does. Without this the only difference between
        # M2 and B on short histories is turn ordering, which confounds the
        # architecture contrast with a presentation artefact.
        chosen = {p["step"] for p in ctx.provenance}
        ordered_parts: list[str] = []
        if -1 in chosen and self._summary:
            ordered_parts.append(
                truncate_to_tokens(f"[MEMORY NOTES]\n{self._summary}",
                                   max(0, budget_tokens - 8))
            )
        ordered_parts += [t.content for t in self._buffer if t.step in chosen]
        ctx = MemoryContext(
            text="\n".join(ordered_parts),
            tokens=ctx.tokens,
            provenance=sorted(ctx.provenance, key=lambda p: p["step"]),
        )
        self._stats.tokens_read += ctx.tokens
        self._stats.latency_ms += (time.perf_counter() - t0) * 1000
        return ctx

    def reset(self) -> None:
        self._summary = ""
        self._summary_sources = []
        self._buffer = []
        self._stats = MemoryStats()

    def stats(self) -> MemoryStats:
        return self._stats
```

```
"""Custom long-horizon task (Mode B: memory use DURING execution — carries the
agent claim).

"Project-Tracker Assistant": the agent observes a long stream of project
updates (facts planted: deadlines, budgets, owners, statuses) mixed with
distractor chatter. At interleaved ACTION steps it must answer operational
queries whose evidence lies many steps back. Facts get UPDATED over time
(newest value is correct), probing update-on-conflict (M3) and recency
weighting (M4) as well as plain retrieval (M1) and recency windows (B).

Fully deterministic from seed; grading is programmatic (normalised match on
the planted value) — no judge needed for Mode B.
"""

from __future__ import annotations

import random
from dataclasses import dataclass, field

from agentmem.memory.base import Turn

PROJECTS = ["Falcon", "Nimbus", "Quasar", "Zephyr", "Basilisk", "Aurora",
            "Meridian", "Sable", "Krypton", "Vortex"]
OWNERS = ["Priya Sharma", "Tom Becker", "Lena Fischer", "Marco Rossi",
          "Aisha Khan", "David Osei", "Ingrid Larsen", "Omar Haddad"]
STATUSES = ["on track", "at risk", "blocked", "ahead of schedule"]
DISTRACTORS = [
    "Reminder: the kitchen on floor 3 is closed for cleaning today.",
    "The all-hands meeting moved to the main auditorium.",
    "IT will run scheduled maintenance over the weekend.",
    "New starters should complete security training by Friday.",
    "The travel policy document was reformatted, no content changes.",
    "Facilities replaced the printers on floor 2.",
]

ATTRS = ("deadline", "budget", "owner", "status")


@dataclass
class ModeBQuery:
    step: int                 # step index at which the query is asked
    question: str
    answer: str               # gold = latest planted value at that step
    evidence_steps: list[int] # steps where the current gold value was stated
    attr: str
    project: str
    distance: int             # steps since the evidence was planted


@dataclass
class ModeBEpisode:
    episode_id: str
    turns: list[Turn]         # observation stream (chronological)
    queries: list[ModeBQuery]
    metadata: dict = field(default_factory=dict)


def _value(rng: random.Random, attr: str) -> str:
    if attr == "deadline":
        return f"{rng.randint(1, 28)} {rng.choice(['July', 'August', 'September', 'October'])} 2
026"
    if attr == "budget":
        return f"£{rng.randint(10, 500)}k"
    if attr == "owner":
        return rng.choice(OWNERS)
    return rng.choice(STATUSES)


def _statement(project: str, attr: str, value: str) -> str:
    return {
        "deadline": f"Update: project {project}'s deadline is now {value}.",
        "budget": f"Update: project {project}'s budget is set to {value}.",
        "owner": f"Update: {value} is now the owner of project {project}.",
        "status": f"Update: project {project} is currently {value}.",
    }[attr]


def _question(project: str, attr: str) -> str:
```

```
    return {
        "deadline": f"What is the current deadline of project {project}?",
        "budget": f"What is the current budget of project {project}?",
        "owner": f"Who currently owns project {project}?",
        "status": f"What is the current status of project {project}?",
    }[attr]


def generate_episode(
    episode_id: str,
    seed: int,
    n_steps: int = 120,
    n_projects: int = 6,
    n_queries: int = 10,
    update_prob: float = 0.25,
    distractor_prob: float = 0.35,
    min_evidence_gap: int = 20,
) -> ModeBEpisode:
    rng = random.Random(seed)
    projects = rng.sample(PROJECTS, n_projects)
    # state[(project, attr)] = (value, [steps where current value was stated])
    state: dict[tuple[str, str], tuple[str, list[int]]] = {}
    turns: list[Turn] = []
    step = 0

    def plant(project: str, attr: str) -> None:
        nonlocal step
        value = _value(rng, attr)
        state[(project, attr)] = (value, [step])
        turns.append(Turn(role="system", content=_statement(project, attr, value),
                          step=step, metadata={"kind": "fact", "project": project,
                                               "attr": attr, "value": value}))
        step += 1

    # Seed every project/attr once up front so all queries are answerable.
    for p in projects:
        for a in ATTRS:
            plant(p, a)

    # Observation stream: distractors + fact updates.
    while step < n_steps:
        if rng.random() < distractor_prob:
            turns.append(Turn(role="system", content=rng.choice(DISTRACTORS),
                              step=step, metadata={"kind": "distractor"}))
            step += 1
        elif rng.random() < update_prob:
            plant(rng.choice(projects), rng.choice(ATTRS))
        else:
            p = rng.choice(projects)
            turns.append(Turn(role="user",
                              content=f"Thanks for the update on {p}, keep me posted.",
                              step=step, metadata={"kind": "chatter"}))
            step += 1

    # Queries at deterministic late positions (last 40% of the episode).
    queries: list[ModeBQuery] = []
    keys = sorted(state.keys())
    rng.shuffle(keys)
    q_positions = sorted(rng.sample(range(int(n_steps * 0.6), n_steps),
                                    min(n_queries, n_steps - int(n_steps * 0.6))))
    for pos, (project, attr) in zip(q_positions, keys):
        value, ev_steps = state[(project, attr)]
        # Gold = latest value planted BEFORE the query position.
        valid = [s for s in ev_steps if s < pos]
        if not valid:
            continue
        # Validity control: the evidence for the current gold value must lie at
        # least `min_evidence_gap` steps back, so the baseline recency window is
        # genuinely memory-limited on every retained query (not trivially caught
        # by keeping the last few turns). Queries violating the floor are dropped.
        if pos - max(valid) < min_evidence_gap:
            continue
        queries.append(ModeBQuery(
            step=pos, question=_question(project, attr), answer=value,
            evidence_steps=valid, attr=attr, project=project,
            distance=pos - max(valid),
        ))

```

```
    return ModeBEpisode(
        episode_id=episode_id, turns=turns, queries=queries,
        metadata={"seed": seed, "n_steps": n_steps, "n_projects": n_projects},
    )


def generate_episodes(n_episodes: int = 10, seed: int = 1, **kwargs) -> list[ModeBEpisode]:
    return [
        generate_episode(episode_id=f"ep{seed}_{i:03d}", seed=seed * 10_000 + i, **kwargs)
        for i in range(n_episodes)
    ]


def grade_answer(predicted: str, gold: str) -> bool:
    """Normalised containment: gold value must appear in the prediction."""
    norm = lambda s: " ".join(s.lower().replace("£", "").split())  # noqa: E731
    return norm(gold) in norm(predicted)
```

```python
"""LongMemEval loader + stratified capper (Mode A: memory probing).

Data file (not committed; see data/README): LongMemEval_s.json from
https://github.com/xiaowu0162/LongMemEval — a list of items with fields:
  question_id, question_type, question, answer,
  haystack_session_ids, haystack_dates, haystack_sessions (list of sessions,
  each a list of {"role": ..., "content": ...} turns),
  answer_session_ids  (which sessions contain the evidence)

Capping: stratified by question_type (equal share per type, seeded), so the
capped subset preserves the ability mix — decided 14 Jun.
"""

from __future__ import annotations

import json
import random
from dataclasses import dataclass, field
from pathlib import Path

from agentmem.memory.base import Turn


@dataclass
class ModeAItem:
    item_id: str
    question_type: str
    question: str
    answer: str
    turns: list[Turn]                    # flattened history to ingest
    evidence_steps: list[int] = field(default_factory=list)  # gold steps
    metadata: dict = field(default_factory=dict)


def _flatten_sessions(item: dict) -> tuple[list[Turn], list[int]]:
    """Flatten haystack sessions to a single turn stream; mark evidence steps."""
    turns: list[Turn] = []
    evidence_steps: list[int] = []
    answer_ids = set(item.get("answer_session_ids", []))
    step = 0
    session_ids = item.get("haystack_session_ids", [])
    dates = item.get("haystack_dates", [])
    for si, session in enumerate(item.get("haystack_sessions", [])):
        sid = session_ids[si] if si < len(session_ids) else f"s{si}"
        date = dates[si] if si < len(dates) else ""
        for turn in session:
            content = f"[{date}] {turn['role']}: {turn['content']}" if date else \
                      f"{turn['role']}: {turn['content']}"
            turns.append(Turn(role=turn["role"], content=content, step=step,
                              metadata={"session_id": sid}))
            # Gold evidence: any turn in an answer session flagged has_answer,
            # else all turns of that session count as evidence.
            if sid in answer_ids and turn.get("has_answer", True):
                evidence_steps.append(step)
            step += 1
    return turns, evidence_steps


def load_longmemeval(
    path: str | Path,
    cap: int | None = None,
    seed: int = 1,
    exclude_types: tuple[str, ...] = (),
) -> list[ModeAItem]:
    raw = json.loads(Path(path).read_text(encoding="utf-8"))
    items: list[ModeAItem] = []
    for it in raw:
        qtype = it.get("question_type", "unknown")
        if qtype in exclude_types:
            continue
        turns, evidence = _flatten_sessions(it)
        items.append(ModeAItem(
            item_id=str(it["question_id"]),
            question_type=qtype,
            question=it["question"],
            answer=str(it.get("answer", "")),
            turns=turns,
            evidence_steps=evidence,
```

```
            metadata={"n_turns": len(turns)},
        ))
    if cap is None or cap >= len(items):
        return items
    return _stratified_cap(items, cap, seed)


def _stratified_cap(items: list[ModeAItem], cap: int, seed: int) -> list[ModeAItem]:
    by_type: dict[str, list[ModeAItem]] = {}
    for it in items:
        by_type.setdefault(it.question_type, []).append(it)
    rng = random.Random(seed)
    types = sorted(by_type)
    per_type = max(1, cap // len(types))
    chosen: list[ModeAItem] = []
    for t in types:
        pool = sorted(by_type[t], key=lambda x: x.item_id)
        rng.shuffle(pool)
        chosen.extend(pool[:per_type])
    # Top up to exactly `cap` from the leftovers, deterministically.
    if len(chosen) < cap:
        leftovers = [it for t in types for it in by_type[t] if it not in chosen]
        rng.shuffle(leftovers)
        chosen.extend(leftovers[: cap - len(chosen)])
    return chosen[:cap]
```

```python
"""Token -> GBP cost accounting with a hard kill-switch (configs/base.yaml gbp_cap).

Prices are PER MILLION tokens in GBP. University HPC and mock are free; the
hosted-API rows are fallback paths. Update from provider price sheets before
any paid run and record the date in the manifest.
"""

from __future__ import annotations

from dataclasses import dataclass, field

# (input_gbp_per_mtok, output_gbp_per_mtok) — verify before paid runs.
PRICE_SHEET: dict[str, tuple[float, float]] = {
    "mock": (0.0, 0.0),
    "university_hpc": (0.0, 0.0),
    "groq/llama-3.1-8b-instant": (0.04, 0.06),
    "groq/llama-3.3-70b-versatile": (0.47, 0.63),
    "openrouter/meta-llama/llama-3.1-8b-instruct": (0.03, 0.05),
}


class BudgetExceededError(RuntimeError):
    """Raised when cumulative spend would exceed the configured GBP cap."""


@dataclass
class CostTracker:
    price_key: str
    gbp_cap: float = 30.0
    input_tokens: int = 0
    output_tokens: int = 0
    calls: int = 0
    _prices: tuple[float, float] = field(init=False)

    def __post_init__(self) -> None:
        self._prices = PRICE_SHEET.get(self.price_key, (0.0, 0.0))

    @property
    def gbp_spent(self) -> float:
        pin, pout = self._prices
        return (self.input_tokens * pin + self.output_tokens * pout) / 1e6

    def add(self, input_tokens: int, output_tokens: int) -> None:
        self.calls += 1
        self.input_tokens += input_tokens
        self.output_tokens += output_tokens
        if self.gbp_spent > self.gbp_cap:
            raise BudgetExceededError(
                f"Spend £{self.gbp_spent:.2f} exceeds cap £{self.gbp_cap:.2f} "
                f"after {self.calls} calls — halting run."
            )
```

```python
"""Run manifest: every experiment logs exactly what config/seed/code produced it."""

from __future__ import annotations

import json
import platform
import subprocess
import sys
from datetime import datetime, timezone
from pathlib import Path


def git_commit() -> str:
    try:
        return (
            subprocess.check_output(
                ["git", "rev-parse", "--short", "HEAD"], stderr=subprocess.DEVNULL
            )
            .decode()
            .strip()
        )
    except Exception:
        return "unknown"


def write_manifest(out_dir: str | Path, config: dict, extra: dict | None = None) -> Path:
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    manifest = {
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "git_commit": git_commit(),
        "python": sys.version.split()[0],
        "platform": platform.platform(),
        "config": config,
        **(extra or {}),
    }
    path = out_dir / "manifest.json"
    path.write_text(json.dumps(manifest, indent=2, default=str), encoding="utf-8")
    return path
```

```python
"""Deterministic seeding for reproducibility (FASER: fixed seeds, logged configs)."""

from __future__ import annotations

import hashlib
import random

import numpy as np


def set_seed(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed % (2**32))
    try:
        import torch
        torch.manual_seed(seed)
    except ImportError:
        pass


def derive_seed(base_seed: int, *parts: str) -> int:
    """Stable per-item seed: same (base_seed, condition, item_id) -> same seed."""
    h = hashlib.sha256(("|".join([str(base_seed), *parts])).encode()).hexdigest()
    return int(h[:8], 16)
```

```
"""Token counting.

Model-agnostic approximation (~4 chars/token) so the harness runs anywhere.
If `tiktoken` is installed it is used instead (closer for most BPE vocabs).
The SAME counter is used for every condition, so DP1 budget comparisons are
fair even if the absolute count differs from the served model's tokenizer.
"""

from __future__ import annotations

try:
    import tiktoken

    _ENC = tiktoken.get_encoding("cl100k_base")

    def count_tokens(text: str) -> int:
        if not text:
            return 0
        # disallowed_special=() : count special-token strings (e.g. literal
        # "<|endoftext|>" inside LongMemEval haystacks) as plain text instead
        # of raising — discovered in the real CERES run, 20 Jul 2026.
        return len(_ENC.encode(text, disallowed_special=()))

except ImportError:  # pragma: no cover - environment dependent

    def count_tokens(text: str) -> int:
        if not text:
            return 0
        # ~4 chars/token is a standard approximation for English BPE.
        return max(1, round(len(text) / 4))


def truncate_to_tokens(text: str, budget: int) -> str:
    """Truncate text so count_tokens(result) <= budget (approximate, safe)."""
    if budget <= 0:
        return ""
    if count_tokens(text) <= budget:
        return text
    # Binary search on character length.
    lo, hi = 0, len(text)
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if count_tokens(text[:mid]) <= budget:
            lo = mid
        else:
            hi = mid - 1
    return text[:lo]
```

```
#!/usr/bin/env python
"""Audit the PROMPT ORDER actually used by each condition in a runs file.

    python scripts/audit_prompt_order.py --file results/rerun_2026_07_29/custom_ds_merged.jsonl

WHY THIS EXISTS
The expert review (B1) found that M2 presented its turns reverse-
chronologically while the baseline presented them chronologically, so the
measured "hierarchical summarisation harms performance" was really a measure of
prompt ordering. That patch is in. But the same question applies to every other
condition, and it had not been asked: if B is chronological and M1/M3/M4 are
score-ordered, then EVERY B-vs-Mx contrast mixes architecture with presentation,
not just the M2 one.

This script answers it from the logged provenance rather than from the code, so
it reports what the reported numbers were actually produced under.

Interpretation:
  chronological          same layout discipline as the baseline: clean contrast
  REVERSE-chronological  the M2 bug pattern: worth -11.9pp on Mode B
  score-order (mixed)    neither; ordering is a free variable in the contrast

A defensible write-up needs ordering declared as a design parameter in Ch3, the
b_reversed control arm quantifying what ordering alone is worth, and any arm
whose deficit is smaller than the ordering effect flagged as unresolved.
"""

from __future__ import annotations

import argparse
import json
from collections import Counter, defaultdict
from pathlib import Path


def classify(steps: list[int]) -> str:
    if len(steps) < 2:
        return "<2 turns"
    if steps == sorted(steps):
        return "chronological"
    if steps == sorted(steps, reverse=True):
        return "REVERSE-chronological"
    return "score-order (mixed)"


def inversion_rate(steps: list[int]) -> float:
    """Fraction of turn pairs presented out of chronological order (0 = perfect)."""
    n = len(steps)
    if n < 2:
        return 0.0
    inv = sum(1 for i in range(n) for j in range(i + 1, n) if steps[i] > steps[j])
    return inv / (n * (n - 1) / 2)


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--file", default="results/rerun_2026_07_29/custom_ds_merged.jsonl")
    args = ap.parse_args()

    path = Path(args.file)
    if not path.exists():
        raise SystemExit(f"no such file: {path}")

    kinds: dict[str, Counter] = defaultdict(Counter)
    inversions: dict[str, list[float]] = defaultdict(list)

    for line in path.read_text(encoding="utf-8").splitlines():
        if not line.strip():
            continue
        rec = json.loads(line)
        prov = rec.get("provenance") or []
        steps = [p["step"] for p in prov
                 if isinstance(p, dict) and p.get("step") is not None and p["step"] != -1]
        cond = rec.get("condition", "?")
        kinds[cond][classify(steps)] += 1
        if len(steps) >= 2:
            inversions[cond].append(inversion_rate(steps))

```

```
    cols = ["chronological", "REVERSE-chronological", "score-order (mixed)", "<2 turns"]
    print(f"\nprompt-order audit — {path.name}\n")
    print(f"{'cond':<8}" + "".join(f"{c:>24}" for c in cols) + f"{'inversion':>12}{'n':>7}")
    for cond in sorted(kinds):
        c = kinds[cond]
        inv = inversions[cond]
        mean_inv = sum(inv) / len(inv) if inv else 0.0
        print(f"{cond:<8}" + "".join(f"{c[k]:>24}" for k in cols)
              + f"{mean_inv:>12.3f}{sum(c.values()):>7}")

    print("\ninversion = mean fraction of turn pairs shown out of chronological order")
    print("  0.00 = identical layout discipline to the baseline")
    print("  1.00 = fully reversed (the pre-patch M2 pattern, worth -11.9pp)")
    print("\nAny condition with a non-zero inversion rate is contrasted against a")
    print("chronological baseline, so its effect is architecture + ordering.")


if __name__ == "__main__":
    main()
```

```
#!/usr/bin/env python
"""Chapter 4 confirmatory statistics — regenerated from the merged run records.

    python scripts/ch4_stats.py

Data (post-fix, canonical per results/audit_2026_08_02.md):
  Mode B: results/rerun_2026_07_29/custom_ds_merged.jsonl   (1,971 records, 9 arms x 219)
  Mode A: results/rerun_2026_07_29/longmemeval_s_merged.jsonl (1,800 records)

Tests:
  * exact McNemar on paired per-item success, Holm-corrected across the declared
    family of 15 contrasts (Ch3 SS stats protocol);
  * episode-clustered bootstrap 95% CIs (4000 resamples) for success rates and
    risk differences (Mode B clusters = (seed, episode); Mode A clusters = items);
  * Wilcoxon signed-rank STRICTLY for the continuous efficiency metrics
    (total tokens, latency), Holm-corrected within each metric;
  * logistic regression of Mode B success on arm + evidence distance with
    episode-cluster-robust SEs.
"""

from __future__ import annotations

import collections
import json
import zlib
from pathlib import Path

import numpy as np
import pandas as pd
from scipy import stats as sps
import statsmodels.formula.api as smf

ROOT = Path(__file__).resolve().parent.parent
MODE_B = ROOT / "results" / "rerun_2026_07_29" / "custom_ds_merged.jsonl"
MODE_A = ROOT / "results" / "rerun_2026_07_29" / "longmemeval_s_merged.jsonl"

BASE_SEED = 20260802
B_RESAMPLES = 4000

B_ARMS = ["b", "m1", "m2", "m3", "m4",
          "b_reversed", "m1_chrono", "m3_chrono", "m4_chrono"]
A_ARMS = ["b", "m1", "m2", "m3", "m4"]

# Declared confirmatory family (contrast arm, reference arm, mode).
CONTRASTS = [
    ("m1", "b", "A"), ("m4", "b", "A"), ("m3", "b", "A"), ("m2", "b", "A"),
    ("m4_chrono", "m4", "B"), ("m1_chrono", "b", "B"), ("m4_chrono", "b", "B"),
    ("m1_chrono", "m1", "B"), ("b_reversed", "b", "B"), ("m2", "b", "B"),
    ("m1", "b", "B"), ("m3_chrono", "m3", "B"), ("m3", "b", "B"),
    ("m3_chrono", "b", "B"), ("m4", "b", "B"),
]


def load(path):
    with open(path, encoding="utf-8") as fh:
        return [json.loads(ln) for ln in fh if ln.strip()]


def episode(rec):
    return rec["item_id"].split(":")[0]


def total_tokens(rec):
    tp, tc = rec.get("total_prompt_tokens"), rec.get("total_completion_tokens")
    if tp is None or tc is None:
        return None
    return tp + tc


# ------------------------------------------------------------------ bootstrap
def _label_seed(label: str) -> int:
    """Stable per-contrast seed: base seed mixed with a CRC of the label.

    Each interval is drawn from its own generator seeded only by BASE_SEED and
    the contrast label, so an interval does not depend on how many intervals
    were computed before it. Using one module-level generator for every call
    (as an earlier version did) makes each CI a function of call ORDER, which
    means adding or reordering a contrast silently shifts every later interval
```

```
    and the printed bounds cannot be reproduced by re-running the script.
    """
    return (BASE_SEED + zlib.crc32(label.encode("utf-8"))) % (2 ** 32)


def clustered_boot(values, clusters, label, B=B_RESAMPLES):
    """Percentile 95% CI of the mean, resampling whole clusters.

    `label` identifies the estimand (arm or contrast name) and seeds this
    interval's own generator — see _label_seed.
    """
    rng = np.random.default_rng(_label_seed(label))
    values = np.asarray(values, dtype=float)
    clusters = np.asarray(clusters)
    uniq = np.unique(clusters)
    by_cluster = {c: values[clusters == c] for c in uniq}
    n = len(uniq)
    stats = np.empty(B)
    for i in range(B):
        draw = rng.choice(uniq, size=n, replace=True)
        pooled = np.concatenate([by_cluster[c] for c in draw])
        stats[i] = pooled.mean()
    return np.percentile(stats, [2.5, 97.5])


def paired_boot_ci(diffs, clusters, label, B=B_RESAMPLES):
    return clustered_boot(np.asarray(diffs, dtype=float),
                          np.asarray(clusters), label, B)


# ---------------------------------------------------------------- pairing
def index_by_key(recs):
    return {(r["seed"], r["item_id"]): r for r in recs}


def pair_outcomes(ix, iy):
    keys = sorted(set(ix) & set(iy))
    xs = np.array([ix[k]["success"] for k in keys], dtype=float)
    ys = np.array([iy[k]["success"] for k in keys], dtype=float)
    return keys, xs, ys


def mcnemar(xs, ys):
    n10 = int(np.sum((xs == 1) & (ys == 0)))  # contrast-only correct
    n01 = int(np.sum((xs == 0) & (ys == 1)))  # reference-only correct
    n = n10 + n01
    if n == 0:
        return n10, n01, np.nan, 1.0
    p = sps.binomtest(min(n10, n01), n, 0.5).pvalue
    oratio = np.inf if n01 == 0 else n10 / n01
    return n10, n01, oratio, p


def holm(pvals):
    """Holm step-down adjusted p-values (same order as input)."""
    p = np.asarray(pvals, dtype=float)
    m = len(p)
    order = np.argsort(p)
    adj = np.empty(m)
    running = 0.0
    for rank, idx in enumerate(order):
        running = max(running, (m - rank) * p[idx])
        adj[idx] = min(running, 1.0)
    return adj


# ------------------------------------------------------------------- main
def main():
    b_recs = load(MODE_B)
    a_recs = load(MODE_A)
    print(f"Loaded Mode B: {len(b_recs)} records; Mode A: {len(a_recs)} records")

    b_by_arm = collections.defaultdict(list)
    for r in b_recs:
        b_by_arm[r["condition"]].append(r)
    a_by_arm = collections.defaultdict(list)
    for r in a_recs:
        a_by_arm[r["condition"]].append(r)
```

```
157
158     # ---- 1. headline tables ------------------------------------------------
159     print("\n=== 1. Mode B headline (n per arm, success [cluster-boot CI], recall, "
160           "answering tokens, latency p50/p95) ===")
161     for c in B_ARMS:
162         rs = b_by_arm[c]
163         succ = np.array([r["success"] for r in rs], dtype=float)
164         cl = np.array([(r["seed"], episode(r)) for r in rs], dtype=object)
165         cl = np.array([f"{s}|{e}" for s, e in cl])
166         lo, hi = clustered_boot(succ, cl, f"B|{c}")
167         rec = np.mean([r["evidence_recall"] for r in rs])
168         tok = np.mean([r["prompt_tokens"] for r in rs])
169         lat = np.percentile([r["latency_ms"] for r in rs], [50, 95])
170         print(f"  {c:12s} n={len(rs)} succ={succ.mean():.4f} "
171               f"[{lo:.3f},{hi:.3f}] recall={rec:.3f} tok={tok:.0f} "
172               f"lat50={lat[0]:.0f} lat95={lat[1]:.0f}")
173
174     print("\n=== 1b. Mode A headline (recall: undefined on abstention items, "
175           "conservatively counted 0; n_recall = defined records) ===")
176     for c in A_ARMS:
177         rs = a_by_arm[c]
178         succ = np.array([r["success"] for r in rs], dtype=float)
179         # cluster on item_id alone: the same question may appear under several
180         # seeds (seeded cap-120 draws overlap), and repeated draws are not
181         # independent trials of the item.
182         cl = np.array([r["item_id"] for r in rs])
183         lo, hi = clustered_boot(succ, cl, f"A|{c}")
184         rec_defined = [r["evidence_recall"] for r in rs
185                        if r.get("evidence_recall") is not None]
186         rec_zero = np.sum(rec_defined) / len(rs)
187         tok = np.mean([r["prompt_tokens"] for r in rs])
188         lat = np.percentile([r["latency_ms"] for r in rs], 50)
189         print(f"  {c:4s} n={len(rs)} succ={succ.mean():.4f} [{lo:.3f},{hi:.3f}] "
190               f"recall0={rec_zero:.3f} (defined-only mean="
191               f"{np.mean(rec_defined):.3f} on {len(rec_defined)}) "
192               f"tok={tok:.0f} lat50={lat:.0f}")
193
194     # ---- 2. McNemar battery + Holm -----------------------------------------
195     print("\n=== 2. Registered confirmatory family: exact McNemar, Holm over 15 ===")
196     b_idx = {c: index_by_key(b_by_arm[c]) for c in B_ARMS}
197     a_idx = {c: index_by_key(a_by_arm[c]) for c in A_ARMS}
198     rows = []
199     for x, y, mode in CONTRASTS:
200         idx = b_idx if mode == "B" else a_idx
201         keys, xs, ys = pair_outcomes(idx[x], idx[y])
202         n10, n01, oratio, p = mcnemar(xs, ys)
203         rd = float(np.mean(xs - ys))
204         if mode == "B":
205             cl = np.array([f"{k[0]}|{k[1].split(':')[0]}" for k in keys])
206         else:
207             cl = np.array([k[1] for k in keys])  # cluster on item identity
208         lo, hi = paired_boot_ci(xs - ys, cl, f"{mode}|{x}|{y}")
209         rows.append(dict(mode=mode, contrast=f"{x} vs {y}", n=len(keys),
210                          n10=n10, n01=n01, rd=rd, lo=lo, hi=hi,
211                          oratio=oratio, p=p))
212     adj = holm([r["p"] for r in rows])
213     for r, pa in zip(rows, adj):
214         r["p_holm"] = pa
215         print(f"  {r['mode']} {r['contrast']:22s} n={r['n']} disc={r['n10']}/{r['n01']} "
216               f"RD={r['rd']:+.3f} [{r['lo']:+.3f},{r['hi']:+.3f}] "
217               f"OR={r['oratio']:.2f} p={r['p']:.3e} p_Holm={pa:.3e}")
218     print(f"  significant after Holm: {sum(r['p_holm'] < 0.05 for r in rows)}/15")
219
220     # ---- 3. Wilcoxon on continuous efficiency metrics ----------------------
221     print("\n=== 3. Wilcoxon signed-rank (continuous metrics only), Holm within metric ===")
222     for mode, idx, arms in (("B", b_idx, B_ARMS), ("A", a_idx, A_ARMS)):
223         for metric, getter in (("total_tokens", total_tokens),
224                                ("latency_ms", lambda r: r.get("latency_ms"))):
225             fam = []
226             for c in arms:
227                 if c == "b":
228                     continue
229                 keys = sorted(set(idx[c]) & set(idx["b"]))
230                 pairs = [(getter(idx[c][k]), getter(idx["b"][k])) for k in keys]
231                 pairs = [(u, v) for u, v in pairs if u is not None and v is not None]
232                 diffs = np.array([u - v for u, v in pairs], dtype=float)
233                 nz = diffs[diffs != 0]
234                 if len(nz) < 10:
```

```
                    fam.append((f"{c} vs b", len(pairs), np.nan))
                    continue
                w = sps.wilcoxon(diffs, zero_method="wilcox",
                                 alternative="two-sided")
                fam.append((f"{c} vs b", len(pairs), float(np.median(diffs)),
                            float(np.mean(diffs)), w.pvalue))
            adj = holm([f[4] for f in fam])
            for (name, n, med, mean, p), pa in zip(fam, adj):
                print(f"  Mode {mode} {metric:13s} {name:18s} n={n} "
                      f"meddiff={med:+.1f} meandiff={mean:+.1f} "
                      f"p={p:.3e} p_Holm={pa:.3e}")

    # ---- 4. depth sensitivity (Mode B) ----------------------------------
    print("\n=== 4. Mode B depth sensitivity (min evidence_distance in panel = "
          f"{min(r['evidence_distance'] for r in b_recs)}) ===")
    for thr in (20, 40, 60, 80):
        line = [f"d>={thr:3d}"]
        for c in ("b", "m1", "m2", "m4", "m1_chrono", "m4_chrono"):
            rs = [r for r in b_by_arm[c] if r["evidence_distance"] >= thr]
            line.append(f"{c}={np.mean([r['success'] for r in rs]):.3f}(n={len(rs)})")
        print("  " + "  ".join(line))
        # key paired contrasts within the subset
        for x, y in (("b_reversed", "b"), ("m1_chrono", "b"), ("m1_chrono", "m1"),
                     ("m1", "b"), ("m4_chrono", "m4")):
            ix = {k: r for k, r in b_idx[x].items() if r["evidence_distance"] >= thr}
            iy = {k: r for k, r in b_idx[y].items() if r["evidence_distance"] >= thr}
            keys, xs, ys = pair_outcomes(ix, iy)
            n10, n01, _, p = mcnemar(xs, ys)
            print(f"      {x} vs {y}: RD={np.mean(xs-ys):+.3f} "
                  f"disc={n10}/{n01} p={p:.2e}")

    # ---- 5. logistic regression (Mode B) --------------------------------
    print("\n=== 5. Logistic regression, Mode B (cluster-robust by seed x episode) ===")
    df = pd.DataFrame(dict(
        success=[int(r["success"]) for r in b_recs],
        condition=pd.Categorical([r["condition"] for r in b_recs],
                                 categories=B_ARMS),
        dist=[r["evidence_distance"] for r in b_recs],
        cluster=[f"{r['seed']}|{episode(r)}" for r in b_recs],
    ))
    model = smf.logit("success ~ C(condition, Treatment(reference='b')) + dist",
                      data=df).fit(disp=0, cov_type="cluster",
                                   cov_kwds={"groups": df["cluster"]})
    ci = model.conf_int()
    for name in model.params.index:
        print(f"  {name:45s} {model.params[name]:+.3f} "
              f"p={model.pvalues[name]:.4f} "
              f"[{ci.loc[name, 0]:+.3f},{ci.loc[name, 1]:+.3f}]")
    print(f"  McFadden pseudo-R2 = {model.prsquared:.3f}; n = {int(model.nobs)}")

    # ---- 6. descriptive blocks ------------------------------------------
    print("\n=== 6a. Mode B failure tags ===")
    for c in B_ARMS:
        tags = collections.Counter(r["failure_tag"] for r in b_by_arm[c])
        n = len(b_by_arm[c])
        print(f"  {c:12s} " + "  ".join(f"{k}={v / n:.3f}" for k, v in sorted(tags.items())))

    print("\n=== 6b. Mode A failure tags ===")
    for c in A_ARMS:
        tags = collections.Counter(r["failure_tag"] for r in a_by_arm[c])
        n = len(a_by_arm[c])
        print(f"  {c:4s} " + "  ".join(f"{k}={v / n:.3f}" for k, v in sorted(tags.items())))

    print("\n=== 6c. Mode A per-ability success ===")
    abilities = ["single-session-user", "single-session-assistant",
                 "knowledge-update", "multi-session", "temporal-reasoning",
                 "single-session-preference"]
    for a in abilities:
        row = [f"{a:28s}"]
        for c in A_ARMS:
            rs = [r for r in a_by_arm[c] if r["question_type"] == a]
            row.append(f"{c}={np.mean([r['success'] for r in rs]):.3f}(n={len(rs)})")
        print("  " + "  ".join(row))

    print("\n=== 6d. Cost, Mode B (metered totals) ===")
    base = np.mean([total_tokens(r) for r in b_by_arm["b"]])
    for c in B_ARMS:
        rs = b_by_arm[c]
```

```
        answ = np.mean([r["prompt_tokens"] for r in rs])
        tot = np.mean([total_tokens(r) for r in rs])
        calls = np.mean([r["total_llm_calls"] for r in rs])
        print(f"  {c:12s} answ={answ:.1f} total={tot:.1f} calls={calls:.2f} "
              f"answ_vs_b={100 * (answ - 1123.95) / 1123.95:+.1f}% "
              f"total_vs_b={100 * (tot - base) / base:+.1f}%")
    print("  per-seed m3 total vs b:")
    for s in (1, 2, 3):
        t = np.mean([total_tokens(r) for r in b_by_arm["m3"] if r["seed"] == s])
        tb = np.mean([total_tokens(r) for r in b_by_arm["b"] if r["seed"] == s])
        print(f"    seed{s}: {100 * (t - tb) / tb:+.1f}%")

    print("\n=== 6e. Cost, Mode A (instrumented records only) ===")
    base_a = None
    for c in A_ARMS:
        rs = [r for r in a_by_arm[c] if total_tokens(r) is not None]
        tot = np.mean([total_tokens(r) for r in rs])
        if c == "b":
            base_a = tot
        answ = np.mean([r["prompt_tokens"] for r in rs])
        calls = np.mean([r["total_llm_calls"] for r in rs])
        succ = np.mean([r["success"] for r in rs])
        print(f"  {c:4s} n={len(rs)} succ={succ:.3f} answ={answ:.0f} "
              f"total={tot:,.1f} calls={calls:.1f} x_base={tot / base_a:,.2f}")

    print("\n=== 6f. Seed spreads ===")
    for mode, by_arm, arms in (("B", b_by_arm, B_ARMS), ("A", a_by_arm, A_ARMS)):
        for c in arms:
            ss = [np.mean([r["success"] for r in by_arm[c] if r["seed"] == s])
                  for s in (1, 2, 3)]
            print(f"  Mode {mode} {c:12s} seeds={[round(x, 3) for x in ss]} "
                  f"spread={max(ss) - min(ss):.3f}")

    print("\n=== 6g. Baseline read coverage (Mode B) ===")
    rb = b_by_arm["b"]
    print(f"  n_retrieved mean={np.mean([r['n_retrieved'] for r in rb]):.1f}; "
          f"memory_items mean={np.mean([r['memory_items'] for r in rb]):.1f} "
          f"(minus current query turn = "
          f"{np.mean([r['memory_items'] for r in rb]) - 1:.1f})")


if __name__ == "__main__":
    main()
```

```
"""Remove all records for one condition from a runs JSONL so it can be re-run.

TrajectoryWriter resumes by skipping (condition, item_id, seed, mode) keys that
already exist in the file. Re-running a condition without clearing its rows is a
silent no-op: the job starts, writes nothing, and exits successfully.

Usage:
    python scripts/drop_condition.py --file results/runs/custom_ds.jsonl --condition m2
    python scripts/drop_condition.py --file results/runs/custom_ds.jsonl --condition m2 --seed 1

Writes a timestamped .bak alongside the original before touching anything.
"""

from __future__ import annotations

import argparse
import json
import shutil
from datetime import datetime
from pathlib import Path


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--file", required=True, help="results/runs/*.jsonl")
    ap.add_argument("--condition", required=True, help="e.g. m2")
    ap.add_argument("--seed", type=int, default=None,
                    help="drop only this seed (default: all seeds)")
    ap.add_argument("--dry-run", action="store_true")
    args = ap.parse_args()

    path = Path(args.file)
    if not path.exists():
        raise SystemExit(f"no such file: {path}")

    kept, dropped = [], 0
    for line in path.read_text(encoding="utf-8").splitlines():
        if not line.strip():
            continue
        rec = json.loads(line)
        match = rec.get("condition") == args.condition and (
            args.seed is None or rec.get("seed") == args.seed
        )
        if match:
            dropped += 1
        else:
            kept.append(line)

    scope = f"{args.condition}" + (f" seed {args.seed}" if args.seed is not None else " (all see
ds)")
    print(f"{path.name}: {dropped} records match {scope}, {len(kept)} kept")

    if args.dry_run:
        print("dry run — nothing written")
        return
    if dropped == 0:
        print("nothing to do")
        return

    stamp = datetime.now().strftime("%Y%m%d-%H%M%S")
    backup = path.with_suffix(f".jsonl.{stamp}.bak")
    shutil.copy2(path, backup)
    path.write_text("\n".join(kept) + "\n", encoding="utf-8")
    print(f"backup: {backup.name}\nrewrote: {path.name}")


if __name__ == "__main__":
    main()
```

```
#!/usr/bin/env python
"""Export all project code to ONE machine-readable PDF (FASER requirement).

Uses matplotlib's PDF backend (already a project dependency) so the text is
real selectable text, not images. Scripted early, not submission-eve.

  python scripts/export_code_pdf.py       # -> docs/agentmem_code.pdf
"""

from __future__ import annotations

from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
from matplotlib.backends.backend_pdf import PdfPages  # noqa: E402

INCLUDE = ["src/**/*.py", "scripts/*.py", "tests/*.py", "analysis/*.py",
           "configs/**/*.yaml", "requirements.txt", "README.md",
           "data/README.md", ".env.example"]
LINES_PER_PAGE = 78
CHARS_PER_LINE = 96


def gather() -> list[Path]:
    files: list[Path] = []
    for pat in INCLUDE:
        files.extend(sorted(ROOT.glob(pat)))
    return [f for f in files if f.is_file() and "__pycache__" not in str(f)]


def wrap(line: str) -> list[str]:
    line = line.rstrip("\n").replace("\t", "    ")
    return [line[i:i + CHARS_PER_LINE] for i in range(0, max(len(line), 1), CHARS_PER_LINE)]


def main() -> None:
    out = ROOT / "docs" / "agentmem_code.pdf"
    out.parent.mkdir(parents=True, exist_ok=True)
    files = gather()
    with PdfPages(out) as pdf:
        for f in files:
            rel = f.relative_to(ROOT)
            body: list[str] = []
            for n, raw in enumerate(f.read_text(encoding="utf-8",
                                                errors="replace").splitlines(), 1):
                body.extend(f"{n:4d}  {w}" if i == 0 else f"      {w}"
                            for i, w in enumerate(wrap(raw)))
            pages = [body[i:i + LINES_PER_PAGE]
                     for i in range(0, max(len(body), 1), LINES_PER_PAGE)]
            for pno, page in enumerate(pages, 1):
                fig = plt.figure(figsize=(8.27, 11.69))  # A4
                # parse_math=False: source code containing '$...$' (e.g. a
                # docstring mentioning $\iota$) must be typeset literally,
                # not handed to matplotlib's mathtext parser.
                fig.text(0.06, 0.97, f"{rel}  (page {pno}/{len(pages)})",
                         fontsize=9, family="monospace", weight="bold",
                         parse_math=False)
                fig.text(0.06, 0.945, "\n".join(page), fontsize=6.2,
                         family="monospace", va="top", parse_math=False)
                pdf.savefig(fig)
                plt.close(fig)
    size_mb = out.stat().st_size / 1e6
    print(f"Wrote {out} ({len(files)} files, {size_mb:.1f} MB; FASER limit 50 MB)")


if __name__ == "__main__":
    main()
```

```
#!/usr/bin/env python
"""Bridge: pipeline run records -> the EDA notebook's live-telemetry format.

The EDA notebook (What_Should_an_Agent_Remember_EDA.ipynb) ingests live data
via `DATA_SOURCE = "live"` + `LIVE_EXPORT_PATH` and gates it with
validate_schema() against RESEARCH_SCHEMA. This script converts
results/runs/*.jsonl into that exact shape, so Stage-2 migration really is the
advertised one-switch change (notebook Section 9.4 checklist).

Field mapping (pipeline -> notebook):
  condition          -> architecture   b=Baseline m1=Episodic m2=Hierarchical
                                       m3=Semantic m4=Decay
  task               -> task_type      longmemeval=LongMemEval,
                                       custom_ds=CustomLongHorizon
  success            -> task_success   (0/1)
  evidence_recall    -> recall_accuracy
  latency_ms         -> latency        (seconds)
  prompt_tokens      -> input_tokens
  completion_tokens  -> output_tokens
  n_retrieved        -> retrieval_depth
  memory_items       -> turn_count
  memory_written_tokens -> horizon_length  (history length in tokens — the
                                            honest live analogue of context_len)
  question_type / evidence_distance -> difficulty (LME: type; custom: terciles)
  failure_tag        -> status ("completed" | failure tag) + error_count

NOTE for the write-up: the notebook's synthetic ARCH_ORDER
(Baseline/Episodic/Semantic/Hierarchical/Retrieval) predates the final design.
For the live run, update ARCH_ORDER in the notebook to the labels emitted here
(Baseline/Episodic/Hierarchical/Semantic/Decay) — extending the canonical
label set for live arms is step 4 of the migration checklist done correctly,
not a relabelling of planted data.

Usage:
  python scripts/export_for_eda.py                      # all runs -> traces.jsonl
  python scripts/export_for_eda.py --out ../traces.jsonl # write next to notebook
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

from agentmem.logging.trajectory import load_records  # noqa: E402

ARCH_LABELS = {"b": "Baseline", "m1": "Episodic", "m2": "Hierarchical",
               "m3": "Semantic", "m4": "Decay"}
TASK_LABELS = {"longmemeval": "LongMemEval", "custom_ds": "CustomLongHorizon"}


def _difficulty(rec: dict) -> str:
    if rec.get("question_type"):          # Mode A: ability type IS the difficulty axis
        return str(rec["question_type"])
    d = rec.get("evidence_distance")      # Mode B: how far back the evidence lies
    if d is None:
        return "medium"
    return "easy" if d < 30 else ("medium" if d < 70 else "hard")


def convert(rec: dict) -> dict:
    return {
        # RESEARCH_SCHEMA columns
        "architecture": ARCH_LABELS.get(rec["condition"], rec["condition"]),
        "task_type": TASK_LABELS.get(rec["task"], rec["task"]),
        "difficulty": _difficulty(rec),
        "horizon_length": rec.get("memory_written_tokens"),
        "turn_count": rec.get("memory_items"),
        "task_success": int(bool(rec["success"])),
        "recall_accuracy": rec.get("evidence_recall"),
        "latency": (rec.get("latency_ms") or 0.0) / 1000.0,
        "input_tokens": rec.get("prompt_tokens"),
        "output_tokens": rec.get("completion_tokens"),
        "retrieval_depth": rec.get("n_retrieved"),
```

```
        "error_count": 0 if rec["success"] else 1,
        "status": "completed",
        # provenance extras (harmless to the notebook, vital for the appendix)
        "run_id": rec.get("run_id"), "mode": rec.get("mode"),
        "condition": rec.get("condition"), "item_id": rec.get("item_id"),
        "seed": rec.get("seed"), "failure_tag": rec.get("failure_tag"),
        "overhead_calls": rec.get("overhead_calls"),
        "memory_read_tokens": rec.get("memory_read_tokens"),
        "gbp_spent": rec.get("gbp_spent"),
    }


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--runs-dir", default=str(ROOT / "results" / "runs"))
    ap.add_argument("--out", default=str(ROOT / "results" / "traces.jsonl"))
    args = ap.parse_args()

    records: list[dict] = []
    for f in sorted(Path(args.runs_dir).glob("*.jsonl")):
        records.extend(load_records(f))
    if not records:
        sys.exit(f"No run records found in {args.runs_dir}")

    out = Path(args.out)
    out.parent.mkdir(parents=True, exist_ok=True)
    with out.open("w", encoding="utf-8") as fh:
        for rec in records:
            fh.write(json.dumps(convert(rec), ensure_ascii=False) + "\n")
    print(f"Wrote {len(records)} rows -> {out}")
    print('Notebook: set DATA_SOURCE = "live", LIVE_EXPORT_PATH = '
          f'"{out.name}" (place file beside the notebook or use a full path).')


if __name__ == "__main__":
    main()
```

```
#!/usr/bin/env python3
"""Build the four Chapter-4 figures from the merged run records.

    python scripts/make_ch4_figures.py --outdir writing/latex/figures

Produces fig4_1_cost_performance, fig4_2_per_ability,
fig4_3_success_by_condition, fig4_4_failure_modes — the two-panel layouts the
Ch4 captions describe. Supersedes the ad-hoc figures used before the ordering
control arms existed.
"""
from __future__ import annotations
import argparse, collections, json
from pathlib import Path
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

B_ORDER = ["b", "b_reversed", "m1", "m1_chrono", "m2", "m3", "m3_chrono", "m4", "m4_chrono"]
A_ORDER = ["b", "m1", "m2", "m3", "m4"]
LABEL = {"b": "B", "b_reversed": "B-rev", "m1": "M1", "m1_chrono": "M1-ch",
         "m2": "M2", "m3": "M3", "m3_chrono": "M3-ch", "m4": "M4", "m4_chrono": "M4-ch"}
CTRL = {"b_reversed", "m1_chrono", "m3_chrono", "m4_chrono"}
RNG = np.random.default_rng(7)


def load(path):
    d = collections.defaultdict(dict)
    for line in open(path):
        line = line.strip()
        if line:
            r = json.loads(line)
            d[r["condition"]][(r["item_id"], r["seed"])] = r
    return d


def boot(recs, n=3000):
    eps = collections.defaultdict(list)
    for r in recs:
        eps[str(r["item_id"]).split(":")[0]].append(r["success"])
    keys = list(eps)
    out = [np.mean([x for i in RNG.choice(len(keys), len(keys)) for x in eps[keys[i]]])
           for _ in range(n)]
    return np.percentile(out, 2.5), np.percentile(out, 97.5)


def fig_success(B, A, out):
    fig, axes = plt.subplots(1, 2, figsize=(13, 4.6),
                             gridspec_kw={"width_ratios": [9, 5]})
    for ax, d, order, title in ((axes[0], B, B_ORDER, "Mode B — Project-Tracker"),
                                (axes[1], A, A_ORDER, "Mode A — LongMemEval$_s$")):
        vals, los, his, cols = [], [], [], []
        for c in order:
            recs = list(d[c].values())
            m = np.mean([r["success"] for r in recs])
            lo, hi = boot(recs)
            vals.append(m); los.append(m - lo); his.append(hi - m)
            cols.append("#7f8c8d" if c == "b" else ("#2980b9" if c in CTRL else "#95a5a6"))
        x = np.arange(len(order))
        ax.bar(x, vals, yerr=[los, his], capsize=3, color=cols,
               edgecolor="black", linewidth=0.6)
        ax.axhline(vals[0], ls="--", lw=1, color="#c0392b", zorder=0)
        ax.set_xticks(x); ax.set_xticklabels([LABEL[c] for c in order])
        ax.set_ylim(0, 1.05); ax.set_title(title, fontsize=11)
        ax.set_ylabel("Task success")
        ax.grid(axis="y", alpha=0.3, lw=0.5)
    axes[0].text(0.015, 0.975, "blue = ordering-control arms\ndashed = baseline",
                 transform=axes[0].transAxes, ha="left", va="top", fontsize=8,
                 color="#444", bbox=dict(facecolor="white", edgecolor="#ccc",
                                         boxstyle="round,pad=0.35", alpha=0.9))
    fig.tight_layout(); fig.savefig(out, dpi=200); plt.close(fig)


def fig_cost(B, A, out):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))
    for ax, d, order, title, logx in ((axes[0], B, B_ORDER, "Mode B", False),
                                      (axes[1], A, A_ORDER, "Mode A (log scale)", True)):
        pts = []
```

```
        for c in order:
            recs = [r for r in d[c].values() if r.get("total_prompt_tokens")]
            if not recs:
                continue
            tot = np.mean([(r.get("total_prompt_tokens") or 0)
                           + (r.get("total_completion_tokens") or 0) for r in recs])
            s = np.mean([r["success"] for r in recs])
            col = "#7f8c8d" if c == "b" else ("#2980b9" if c in CTRL else "#c0392b")
            ax.scatter(tot, s, s=140, color=col, edgecolor="black", zorder=3)
            pts.append((tot, s, c))
        # stagger labels that would collide (several arms share a cost/accuracy point)
        placed = []
        for tot, s, c in sorted(pts, key=lambda p: (-p[1], p[0])):
            dy = 6
            while any(abs(s - ps) < 0.012 and abs(dy - pd) < 11 for ps, pd in placed):
                dy -= 12
            placed.append((s, dy))
            ax.annotate(LABEL[c], (tot, s), textcoords="offset points",
                        xytext=(9, dy), fontsize=9)
        if logx:
            ax.set_xscale("log")
        ax.set_xlabel("Total tokens per item (measured, incl. write path)")
        ax.set_ylabel("Task success"); ax.set_title(title, fontsize=11)
        ax.grid(alpha=0.3, lw=0.5)
    fig.tight_layout(); fig.savefig(out, dpi=200); plt.close(fig)


def fig_ability(A, out):
    per = collections.defaultdict(lambda: collections.defaultdict(list))
    for c, items in A.items():
        for r in items.values():
            t = (r.get("attr") or {}).get("question_type") or r.get("question_type") or "unknown
"
            per[t][c].append(r["success"])
    abilities = sorted(per, key=lambda t: -np.mean(per[t]["m1"] or [0]))
    x = np.arange(len(abilities)); w = 0.16
    fig, ax = plt.subplots(figsize=(11, 4.4))
    for i, c in enumerate(A_ORDER):
        vals = [np.mean(per[t][c]) if per[t][c] else 0 for t in abilities]
        ax.bar(x + (i - 2) * w, vals, w, label=LABEL[c], edgecolor="black", linewidth=0.4)
    ax.set_xticks(x)
    ax.set_xticklabels([t.replace("single-session-", "ss-") for t in abilities],
                       rotation=20, ha="right")
    ax.set_ylabel("Task success"); ax.set_ylim(0, 1.0)
    ax.set_title("Mode A success by ability type", fontsize=11)
    ax.legend(ncol=5, fontsize=9); ax.grid(axis="y", alpha=0.3, lw=0.5)
    fig.tight_layout(); fig.savefig(out, dpi=200); plt.close(fig)


def fig_failures(B, A, out):
    tags = ["retrieval_miss", "partial_retrieval", "stale_fact", "reasoning_failure", "no_answer
"]
    colours = ["#c0392b", "#e67e22", "#f1c40f", "#8e44ad", "#7f8c8d"]
    fig, axes = plt.subplots(1, 2, figsize=(13, 4.6),
                             gridspec_kw={"width_ratios": [9, 5]})
    for ax, d, order, title in ((axes[0], B, B_ORDER, "Mode B"),
                                (axes[1], A, A_ORDER, "Mode A")):
        x = np.arange(len(order)); bottom = np.zeros(len(order))
        for tag, col in zip(tags, colours):
            vals = []
            for c in order:
                items = d[c]; n = len(items)
                vals.append(sum(1 for r in items.values()
                                if (r.get("failure_tag") or "") == tag) / n)
            ax.bar(x, vals, bottom=bottom, color=col, label=tag.replace("_", " "),
                   edgecolor="black", linewidth=0.4)
            bottom += np.array(vals)
        ax.set_xticks(x); ax.set_xticklabels([LABEL[c] for c in order])
        ax.set_ylabel("Fraction of items"); ax.set_ylim(0, 1.0)
        ax.set_title(title, fontsize=11); ax.grid(axis="y", alpha=0.3, lw=0.5)
    axes[1].legend(fontsize=8, loc="upper right")
    fig.tight_layout(); fig.savefig(out, dpi=200); plt.close(fig)


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--modeb", default="results/rerun_2026_07_29/custom_ds_merged.jsonl")
    ap.add_argument("--modea", default="results/rerun_2026_07_29/longmemeval_s_merged.jsonl")
```

```
155      ap.add_argument("--outdir", default="results/figures")
156      a = ap.parse_args()
157      out = Path(a.outdir); out.mkdir(parents=True, exist_ok=True)
158      B, A = load(a.modeb), load(a.modea)
159      fig_success(B, A, out / "fig4_3_success_by_condition.png")
160      fig_cost(B, A, out / "fig4_1_cost_performance.png")
161      fig_ability(A, out / "fig4_2_per_ability.png")
162      fig_failures(B, A, out / "fig4_4_failure_modes.png")
163      print(f"wrote 4 figures -> {out}")
```

```python
#!/usr/bin/env python
"""Rebuild every dissertation figure from results/runs/*.jsonl in one command.

  python scripts/make_figures.py           # -> results/figures/*.png

Figures (one per RQ angle):
  fig1_success_by_condition   success rate + bootstrap 95% CI, per task
  fig2_success_vs_distance    Mode B: success vs evidence distance (memory decay in practice)
  fig3_cost_performance       tokens-per-question vs success (the efficiency frontier)
  fig4_failure_modes          stacked failure-tag composition per condition
"""

from __future__ import annotations

import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
sys.path.insert(0, str(ROOT))

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402
import pandas as pd  # noqa: E402

from agentmem.logging.trajectory import load_records  # noqa: E402
from analysis.stats import records_to_df  # noqa: E402

FIG_DIR = ROOT / "results" / "figures"
ORDER = ["b", "m1", "m2", "m3", "m4"]
LABELS = {"b": "B (none)", "m1": "M1 episodic", "m2": "M2 summary",
          "m3": "M3 semantic", "m4": "M4 decay"}


def _boot_ci(x: np.ndarray, n_boot: int = 5000, seed: int = 1) -> tuple[float, float]:
    rng = np.random.default_rng(seed)
    boots = [x[rng.integers(0, len(x), len(x))].mean() for _ in range(n_boot)]
    return float(np.percentile(boots, 2.5)), float(np.percentile(boots, 97.5))


def fig_success_by_condition(df: pd.DataFrame, task: str) -> None:
    fig, ax = plt.subplots(figsize=(7, 4.2))
    xs, means, err_lo, err_hi = [], [], [], []
    for c in ORDER:
        x = df.loc[df.condition == c, "success"].to_numpy()
        if not len(x):
            continue
        lo, hi = _boot_ci(x)
        xs.append(LABELS[c]); means.append(x.mean())
        err_lo.append(x.mean() - lo); err_hi.append(hi - x.mean())
    ax.bar(xs, means, yerr=[err_lo, err_hi], capsize=4, color="#4878a8")
    ax.set_ylabel("Task success rate")
    ax.set_ylim(0, 1)
    ax.set_title(f"Success by memory architecture - {task} (95% bootstrap CI)")
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"fig1_success_by_condition_{task}.png", dpi=200)
    plt.close(fig)


def fig_success_vs_distance(df: pd.DataFrame) -> None:
    d = df.dropna(subset=["evidence_distance"]) if "evidence_distance" in df else pd.DataFrame()
    if d.empty:
        return
    bins = [0, 20, 40, 60, 80, 200]
    fig, ax = plt.subplots(figsize=(7, 4.2))
    for c in ORDER:
        sub = d[d.condition == c]
        if sub.empty:
            continue
        grp = sub.groupby(pd.cut(sub["evidence_distance"], bins, right=False),
                          observed=True)["success"].mean()
        ax.plot([iv.mid for iv in grp.index], grp.values, marker="o", label=LABELS[c])
    ax.set_xlabel("Steps between evidence and question (evidence distance)")
    ax.set_ylabel("Success rate")
    ax.set_ylim(-0.02, 1.02)
```

```
    ax.set_title("Does memory survive distance? (Mode B custom task)")
    ax.legend(fontsize=8)
    fig.tight_layout()
    fig.savefig(FIG_DIR / "fig2_success_vs_distance.png", dpi=200)
    plt.close(fig)


def fig_cost_performance(df: pd.DataFrame, task: str) -> None:
    fig, ax = plt.subplots(figsize=(7, 4.2))
    for c in ORDER:
        sub = df[df.condition == c]
        if sub.empty:
            continue
        cost = (sub["prompt_tokens"] + sub["completion_tokens"]).mean()
        ax.scatter(cost, sub["success"].mean(), s=90)
        ax.annotate(LABELS[c], (cost, sub["success"].mean()),
                    textcoords="offset points", xytext=(6, 4), fontsize=8)
    ax.set_xlabel("Mean tokens per question (prompt + completion)")
    ax.set_ylabel("Task success rate")
    ax.set_title(f"Cost-performance frontier - {task}")
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"fig3_cost_performance_{task}.png", dpi=200)
    plt.close(fig)


def fig_failure_modes(df: pd.DataFrame, task: str) -> None:
    ct = pd.crosstab(df["condition"], df["failure_tag"], normalize="index")
    ct = ct.reindex([c for c in ORDER if c in ct.index])
    fig, ax = plt.subplots(figsize=(7, 4.2))
    bottom = np.zeros(len(ct))
    for tag in ct.columns:
        ax.bar([LABELS[c] for c in ct.index], ct[tag].values, bottom=bottom, label=tag)
        bottom += ct[tag].values
    ax.set_ylabel("Share of questions")
    ax.set_title(f"Failure-mode composition - {task}")
    ax.legend(fontsize=8, ncol=2)
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"fig4_failure_modes_{task}.png", dpi=200)
    plt.close(fig)


def main() -> None:
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    made = 0
    for f in sorted((ROOT / "results" / "runs").glob("*.jsonl")):
        df = records_to_df(load_records(f))
        if df.empty:
            continue
        task = f.stem
        fig_success_by_condition(df, task)
        fig_cost_performance(df, task)
        fig_failure_modes(df, task)
        fig_success_vs_distance(df)
        made += 1
    print(f"Figures rebuilt for {made} task file(s) -> {FIG_DIR}")


if __name__ == "__main__":
    main()
```

```
#!/usr/bin/env python
"""Full statistical battery over results/runs/*.jsonl -> results/report.md.

Per task: headline summary with Wilson CIs, primary success contrasts by EXACT
McNEMAR vs baseline (Holm-corrected), bootstrap CIs, per-seed variance, cost
contrasts on tokens and latency, logistic regression as odds ratios, and the
failure-tag crosstab. Numbers feed Ch4 directly; the report is regenerated,
never hand-edited.

SAMPLE DEFINITION. Mode B is reported at FULL n (every generated item) as the
primary analysis, with the evidence_distance >= GAP_FILTER subset alongside as a
labelled sensitivity analysis. The filter is applied in analysis, not enforced
by the generator, so it cannot be described as a design parameter; presenting
the filtered subset as primary is what made Ch4's "every value below corresponds
to the registered analysis" false.
"""

from __future__ import annotations

import sys
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
sys.path.insert(0, str(ROOT))

from agentmem.logging.trajectory import load_records  # noqa: E402
from agentmem.utils.manifest import git_commit  # noqa: E402
from analysis.stats import (  # noqa: E402
    cost_contrasts,
    failure_crosstab,
    logistic_regression,
    odds_ratio_table,
    per_seed_table,
    records_to_df,
    success_contrasts,
    summary_table,
)

CONDITIONS = ["b", "m1", "m2", "m3", "m4", "b_reversed"]

# Post-hoc gap filter, reported as SENSITIVITY only. Ch3 must not describe this
# as generator-enforced: the true minimum evidence_distance in the full run is 1.
GAP_FILTER = 20


def block(text: str) -> str:
    return f"```\n{text}\n```\n"


def report_task(df, task: str) -> str:
    out = [f"## Task: {task}\n"]
    out.append(f"Records: {len(df)} | seeds: {sorted(df.seed.unique().tolist())} "
               f"| conditions: {sorted(df.condition.unique().tolist())}\n")
    out.append("### Headline summary\n" + block(summary_table(df).to_string()))

    present = [c for c in CONDITIONS if c in set(df.condition)]
    others = [c for c in present if c != "b"]
    cluster = "episode_id" if "episode_id" in df.columns else None

    if others:
        out.append("### PRIMARY — success vs baseline: exact McNemar "
                   "(Holm-corrected) + risk difference with bootstrap CI\n")
        out.append("_McNemar, not Wilcoxon: on 0/1 outcomes every non-zero "
                   "difference has |d|=1 and the signed-rank statistic "
                   "degenerates._\n")
        sc = success_contrasts(df, "b", others, cluster=cluster)
        cols = ["contrast", "n_items", "n_discordant", "risk_diff", "ci_low",
                "ci_high", "odds_ratio", "p", "p_holm", "significant_05"]
        out.append(block(sc[cols].round(4).to_string(index=False)))
        out.append("_n_items is the MATCHED n. A paired test may only ever be "
                   "quoted against this number, never against the raw per-"
                   "condition record counts._\n")

        out.append("### Per-seed variance\n")
        ps = per_seed_table(df)
        out.append(block(ps.to_string()))
```

```
        risky = [f"{r['contrast']} (|effect|={abs(r['risk_diff']):.3f})"
                 for _, r in sc.iterrows()
                 if r["contrast"].split(" vs ")[0] in ps.index
                 and abs(r["risk_diff"]) < ps.loc[r["contrast"].split(" vs ")[0], "spread"]]
        if risky:
            out.append("**Effects smaller than their own seed-to-seed spread — "
                       "do not report these as findings without more seeds:** "
                       + "; ".join(risky) + "\n")

        out.append("### RQ3 — COST contrasts (tokens, latency), Holm-corrected "
                   "within metric\n")
        cc = cost_contrasts(df, "b", others)
        if len(cc):
            ccols = ["contrast", "metric", "mean_diff", "pct_change", "ci_low",
                     "ci_high", "p_holm", "significant_05"]
            out.append(block(cc[ccols].round(4).to_string(index=False)))
        if "total_prompt_tokens" not in df.columns:
            out.append("_WARNING: `prompt_tokens` counts the ANSWERING call only "
                       "and excludes extraction/write calls, so it understates "
                       "any condition with a write path. Re-run with the metered "
                       "runner, or use `scripts/reconstruct_cost.py`, before "
                       "quoting any token saving._\n")

    try:
        extra = ["question_type"] if "question_type" in df and df["question_type"].notna().any()
 \
            else (["evidence_distance"] if "evidence_distance" in df else [])
        model = logistic_regression(df.dropna(subset=["success"]),
                                    extra_terms=[f"C({extra[0]})"] if extra and extra[0] == "que
stion_type"
                                    else extra,
                                    cluster=cluster)
        out.append("### Logistic regression (success ~ condition"
                   + (f" + {extra[0]}" if extra else "")
                   + (", cluster-robust SEs" if cluster else "") + ")\n"
                   + block(str(model.summary2())))
        out.append("#### Odds ratios with 95% CIs\n"
                   + block(odds_ratio_table(model).to_string()))
    except Exception as e:  # small pilots can be separable — report, don't crash
        out.append(f"### Logistic regression\n_Not estimable on this data: {e}_\n")

    out.append("### Failure-mode composition (row-normalised)\n"
               + block(failure_crosstab(df).to_string()))
    return "\n".join(out)


def main() -> None:
    parts = [
        "# Experiment report (auto-generated — do not hand-edit)\n",
        f"Generated: {datetime.now(timezone.utc).isoformat()} | "
        f"commit: {git_commit()}\n",
    ]
    n_tasks = 0
    for f in sorted((ROOT / "results" / "runs").glob("*.jsonl")):
        df = records_to_df(load_records(f))
        if df.empty:
            continue
        # Mode B item_ids are "<episode>:q<step>"; queries within an episode
        # share a history and are NOT independent, so tests and CIs cluster here.
        if "item_id" in df.columns:
            df["episode_id"] = df["item_id"].astype(str).str.split(":").str[0]

        parts.append(report_task(df, f.stem))
        n_tasks += 1

        # Sensitivity: the post-hoc gap filter, clearly labelled as such.
        if "evidence_distance" in df.columns and df["evidence_distance"].notna().any():
            sub = df[df["evidence_distance"] >= GAP_FILTER]
            n_items_full = df["item_id"].nunique()
            n_items_sub = sub["item_id"].nunique()
            if 0 < n_items_sub < n_items_full:
                parts.append(
                    f"\n---\n\n# SENSITIVITY ANALYSIS — {f.stem}, "
                    f"evidence_distance >= {GAP_FILTER}\n\n"
                    f"Post-hoc filter applied in ANALYSIS, not enforced by the "
                    f"generator (true minimum distance in the full run is "
                    f"{int(df['evidence_distance'].min())}). Primary results are "
                    f"the full-sample section above ({n_items_full} items); this "
```

```
                    f"subset ({n_items_sub} items, median distance "
                    f"{sub['evidence_distance'].median():.0f}) is reported to show "
                    f"the conclusions' robustness to the gap threshold. Effect "
                    f"sizes move materially between the two, so neither may be "
                    f"quoted without stating which sample it came from.\n")
            parts.append(report_task(sub, f"{f.stem} (distance >= {GAP_FILTER})"))
    if not n_tasks:
        sys.exit("No run records found — run scripts/run_full_grid.py first.")
    out = ROOT / "results" / "report.md"
    out.write_text("\n".join(parts), encoding="utf-8")
    print(f"Wrote {out} ({n_tasks} task section(s))")


if __name__ == "__main__":
    main()
```

```
"""Merge parallel per-cell JSONL files into one runs file, deduping on record_key.

The parallel jobs (scripts/hpc/run_one_gpu.sh) write to their own files so they
never collide. Some cells also exist in the main grid file, so a naive `cat`
double-counts them into every rate you compute. This merges on the same key
TrajectoryWriter uses -- (condition, item_id, seed, mode) -- and reports what
came from where.

Precedence: later --input files win on a key collision. Put the file you trust
most LAST (normally the patched/most recent run).

Usage:
    python scripts/merge_runs.py \
        --input results/runs/longmemeval_s.jsonl \
        --input "results/runs/parallel/longmemeval_s_*.jsonl" \
        --out   results/runs/longmemeval_s_merged.jsonl

    # dry run first -- prints the coverage table, writes nothing
    python scripts/merge_runs.py --input ... --out ... --dry-run
"""

from __future__ import annotations

import argparse
import glob
import json
from collections import Counter, defaultdict
from pathlib import Path

EXPECTED_PER_CELL = 120  # Mode A items per (condition, seed)


def record_key(rec: dict) -> tuple:
    return (rec["condition"], rec["item_id"], rec["seed"], rec["mode"])


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--input", action="append", required=True,
                    help="file or glob; repeatable; later inputs win collisions")
    ap.add_argument("--out", required=True)
    ap.add_argument("--expected", type=int, default=EXPECTED_PER_CELL)
    ap.add_argument("--dry-run", action="store_true")
    args = ap.parse_args()

    paths: list[Path] = []
    for pattern in args.input:
        hits = sorted(glob.glob(pattern))
        if not hits:
            print(f"WARNING: no files match {pattern!r}")
        paths += [Path(h) for h in hits]

    merged: dict[tuple, dict] = {}
    source: dict[tuple, str] = {}
    overwritten = Counter()
    per_file = Counter()

    for p in paths:
        n = 0
        for line in p.read_text(encoding="utf-8").splitlines():
            if not line.strip():
                continue
            try:
                rec = json.loads(line)
                k = record_key(rec)
            except (json.JSONDecodeError, KeyError):
                continue
            if k in merged and source[k] != p.name:
                overwritten[f"{source[k]} -> {p.name}"] += 1
            merged[k] = rec
            source[k] = p.name
            n += 1
        per_file[p.name] = n
        print(f"  read {n:5d} records from {p.name}")

    print(f"\n{sum(per_file.values())} records read, {len(merged)} unique "
          f"({sum(per_file.values()) - len(merged)} duplicates collapsed)")
    if overwritten:
```

```
        print("\nCollisions resolved (later input won):")
        for pair, c in overwritten.most_common():
            print(f"  {c:5d}  {pair}")

    # Coverage table
    cells: dict[str, dict[int, int]] = defaultdict(lambda: defaultdict(int))
    for rec in merged.values():
        cells[rec["condition"]][rec["seed"]] += 1

    print(f"\nCoverage (expected {args.expected} per cell):")
    seeds = sorted({s for c in cells.values() for s in c})
    print("  cond  " + "".join(f"{'s'+str(s):>8}" for s in seeds) + f"{'total':>9}  status")
    for cond in sorted(cells):
        row = cells[cond]
        total = sum(row.values())
        full = args.expected * len(seeds)
        status = "COMPLETE" if total >= full else f"{100*total/full:.0f}%"
        print("  " + f"{cond:<6}" + "".join(f"{row.get(s, 0):>8}" for s in seeds)
              + f"{total:>9}  {status}")

    if args.dry_run:
        print("\ndry run -- nothing written")
        return

    out = Path(args.out)
    out.parent.mkdir(parents=True, exist_ok=True)
    with out.open("w", encoding="utf-8") as f:
        for k in sorted(merged, key=lambda k: (k[0], k[2], str(k[1]))):
            f.write(json.dumps(merged[k], ensure_ascii=False, default=str) + "\n")
    print(f"\nwrote {len(merged)} records -> {out}")


if __name__ == "__main__":
    main()
```

```
#!/usr/bin/env python
"""Decompose every B-vs-Mx effect into ORDERING and ARCHITECTURE components.

    python scripts/ordering_decomposition.py --file results/rerun_2026_07_29/custom_ds_merged.js
onl

WHY THIS EXISTS
---------------------------------------------------------------------------
`audit_prompt_order.py` established that every memory condition is contrasted
against a chronologically-ordered baseline, so every B-vs-Mx effect is
architecture PLUS presentation. That audit reports one mean inversion rate per
condition and stops there. It cannot say how much of any given deficit is
ordering, because with one number per condition, ordering and architecture are
perfectly collinear.

The identification insight this script exploits: the inversion rate is NOT
constant within a condition. M1, M3 and M4 select turns by relevance score, and
how badly that scrambles chronology depends on where the evidence happens to sit
in the episode. So within a single architecture, holding the memory mechanism
exactly fixed, some items are presented near-chronologically and some are
presented near-reversed. That within-condition variation identifies the ordering
slope without any new compute, and the slope can then be used to predict each
condition's success rate at the counterfactual iota = 0 (fully chronological)
that the baseline already enjoys.

This is the observational analogue of the "render-matched control" that Jiang et
al. (2026, arXiv:2607.16019) propose as a reusable protocol for separating layout
from mechanism, and of the ordering-only b_reversed arm already registered in
this repo. The experimental arm remains the stronger evidence; this recovers most
of the answer from data already on disk, and pre-registers the prediction that
the arm will test.

WHAT IS AND IS NOT IDENTIFIED
---------------------------------------------------------------------------
iota is observational, not randomised. It is a function of where the evidence
sits in the episode, so it correlates with evidence_distance, which is itself
predictive of success. Every model below therefore conditions on
evidence_distance, and Model 3 adds item fixed effects so the slope is estimated
only from items where two seeds of the SAME condition happened to draw different
layouts. Residual confounding by unobserved item difficulty cannot be excluded
and MUST be stated wherever these numbers are quoted. Read the adjusted effects
as "what the contrast looks like once measured presentation differences are
netted out", not as a causal architecture effect.

OUTPUTS
---------------------------------------------------------------------------
results/ordering_decomposition.md    tables + interpretation
results/figures/fig_ordering.png     iota distribution + fitted slope
results/figures/fig_pareto.png       cost-adjusted accuracy frontier
"""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import numpy as np
import pandas as pd

# ---------------------------------------------------------------------------
# ordering metric
# ---------------------------------------------------------------------------


def inversion_rate(steps: list[int]) -> float:
    """Kendall-style fraction of turn pairs shown out of chronological order.

    0.0 = perfectly chronological (the baseline's layout discipline)
    1.0 = perfectly reversed (the pre-patch M2 pattern)
    Equivalent to (1 - tau_b) / 2 for a permutation with no ties.
    """
    n = len(steps)
    if n < 2:
        return 0.0
    arr = np.asarray(steps)
    inv = sum(int(np.sum(arr[i + 1:] < arr[i])) for i in range(n - 1))
    return inv / (n * (n - 1) / 2)
```

```
 78
 79
 80  def load(path: Path) -> pd.DataFrame:
 81      rows = []
 82      for line in path.read_text(encoding="utf-8").splitlines():
 83          if not line.strip():
 84              continue
 85          rec = json.loads(line)
 86          prov = rec.get("provenance") or []
 87          steps = [p["step"] for p in prov
 88                   if isinstance(p, dict) and p.get("step") is not None
 89                   and p["step"] != -1]
 90          rec["iota"] = inversion_rate(steps)
 91          rec["n_shown"] = len(steps)
 92          rec["episode"] = str(rec.get("item_id", "")).split(":")[0]
 93          rec.pop("provenance", None)
 94          rows.append(rec)
 95      df = pd.DataFrame(rows)
 96      df["success"] = df["success"].astype(int)
 97      return df
 98
 99
100  # ---------------------------------------------------------------------------
101  # models
102  # ---------------------------------------------------------------------------
103
104
105  def fit_models(df: pd.DataFrame) -> dict:
106      import statsmodels.formula.api as smf
107
108      out = {}
109
110      # Model 1 — the dissertation's current specification, for reference.
111      m1 = smf.logit(
112          "success ~ C(condition, Treatment('b')) + evidence_distance", data=df
113      ).fit(disp=0, cov_type="cluster", cov_kwds={"groups": df["episode"]})
114      out["m1"] = m1
115
116      # Model 2 — same, plus the ordering covariate.
117      m2 = smf.logit(
118          "success ~ C(condition, Treatment('b')) + iota + evidence_distance", data=df
119      ).fit(disp=0, cov_type="cluster", cov_kwds={"groups": df["episode"]})
120      out["m2"] = m2
121
122      # Model 3 — item fixed effects. Slope identified ONLY from within-item,
123      # within-condition layout variation across seeds. The strictest available
124      # control for item difficulty.
125      sub = df[df["condition"].isin(["m1", "m3", "m4"])].copy()
126      keep = sub.groupby(["item_id", "condition"])["iota"].transform("nunique") > 1
127      sub_fe = sub[keep]
128      out["fe_n"] = len(sub_fe)
129      out["fe_items"] = sub_fe["item_id"].nunique()
130      if len(sub_fe) > 40 and sub_fe["success"].nunique() > 1:
131          try:
132              m3 = smf.logit(
133                  "success ~ iota + C(condition) + C(item_id)", data=sub_fe
134              ).fit(disp=0, method="bfgs", maxiter=400)
135              out["m3"] = m3
136          except Exception as exc:  # pragma: no cover
137              out["m3_error"] = str(exc)
138
139      # Model 4 — within each architecture separately. Architecture is literally
140      # constant here, so nothing about the mechanism can drive the slope.
141      per_cond = {}
142      for cond in ["m1", "m3", "m4"]:
143          d = df[df["condition"] == cond]
144          if d["iota"].nunique() < 3 or d["success"].nunique() < 2:
145              continue
146          try:
147              fit = smf.logit("success ~ iota + evidence_distance", data=d).fit(
148                  disp=0, cov_type="cluster", cov_kwds={"groups": d["episode"]}
149              )
150              per_cond[cond] = {
151                  "coef": fit.params["iota"],
152                  "se": fit.bse["iota"],
153                  "p": fit.pvalues["iota"],
154                  "ci": fit.conf_int().loc["iota"].tolist(),
155                  "n": len(d),
```

```
                "iota_sd": d["iota"].std(),
            }
        except Exception:
            continue
    out["per_cond"] = per_cond
    return out


def counterfactual(df: pd.DataFrame, m2) -> pd.DataFrame:
    """Predicted success for each condition at its OWN iota vs at iota = 0."""
    rows = []
    for cond in sorted(df["condition"].unique()):
        d = df[df["condition"] == cond].copy()
        observed = d["success"].mean()
        p_actual = m2.predict(d).mean()
        d0 = d.copy()
        d0["iota"] = 0.0
        p_chrono = m2.predict(d0).mean()
        rows.append(
            {
                "condition": cond,
                "n": len(d),
                "mean_iota": d["iota"].mean(),
                "sd_iota": d["iota"].std(),
                "observed": observed,
                "fitted": p_actual,
                "fitted_at_iota0": p_chrono,
                "ordering_penalty": p_actual - p_chrono,
            }
        )
    out = pd.DataFrame(rows).set_index("condition")
    base = out.loc["b"]
    out["raw_vs_b"] = out["observed"] - base["observed"]
    out["adj_vs_b"] = out["fitted_at_iota0"] - base["fitted_at_iota0"]
    out["attributable_to_ordering"] = out["raw_vs_b"] - out["adj_vs_b"]
    return out


def bootstrap_adjusted(df: pd.DataFrame, n_boot: int = 400, seed: int = 7) -> pd.DataFrame:
    """Episode-cluster bootstrap CIs on the ordering-adjusted contrasts."""
    import statsmodels.formula.api as smf

    rng = np.random.default_rng(seed)
    eps = df["episode"].unique()
    conds = [c for c in sorted(df["condition"].unique()) if c != "b"]
    draws = {c: [] for c in conds}
    for _ in range(n_boot):
        pick = rng.choice(eps, size=len(eps), replace=True)
        boot = pd.concat([df[df["episode"] == e] for e in pick], ignore_index=True)
        if boot["condition"].nunique() < df["condition"].nunique():
            continue
        try:
            fit = smf.logit(
                "success ~ C(condition, Treatment('b')) + iota + evidence_distance",
                data=boot,
            ).fit(disp=0)
        except Exception:
            continue
        b0 = boot.copy()
        b0["iota"] = 0.0
        b0["_p"] = fit.predict(b0)
        g = b0.groupby("condition")["_p"].mean()
        if "b" not in g:
            continue
        for c in conds:
            if c in g:
                draws[c].append(g[c] - g["b"])
    rows = []
    for c in conds:
        v = np.array(draws[c])
        if len(v) < 30:
            continue
        rows.append(
            {
                "contrast": f"{c} vs b",
                "adj_effect": v.mean(),
                "ci_low": np.percentile(v, 2.5),
                "ci_high": np.percentile(v, 97.5),
```

```
                "n_boot": len(v),
            }
        )
    return pd.DataFrame(rows)


# ---------------------------------------------------------------------------
# cost frontier
# ---------------------------------------------------------------------------


def confounding_check(df: pd.DataFrame) -> pd.DataFrame:
    """How badly is iota entangled with item difficulty within each condition?"""
    rows = []
    for cond in sorted(df["condition"].unique()):
        d = df[df["condition"] == cond]
        if d["iota"].nunique() < 3:
            rows.append({"condition": cond, "iota_sd": d["iota"].std(),
                         "r_iota_distance": np.nan, "r_iota_nshown": np.nan,
                         "note": "iota constant — no within-condition variation"})
            continue
        rows.append({
            "condition": cond,
            "iota_sd": d["iota"].std(),
            "r_iota_distance": d["iota"].corr(d["evidence_distance"]),
            "r_iota_nshown": d["iota"].corr(d["n_shown"]),
            "note": "",
        })
    return pd.DataFrame(rows).set_index("condition")


def power_for_arm(df: pd.DataFrame, effect_pp: float = 0.119,
                  alpha: float = 0.05, power: float = 0.80) -> dict:
    """How many items does a randomised ordering arm need?

    Uses the discordance rate actually observed in the M2-vs-B McNemar
    contrast to size a paired (McNemar) test, which is what a b_reversed vs b
    comparison would use.
    """
    from scipy import stats

    p_base = df.loc[df["condition"] == "b", "success"].mean()
    p_alt = p_base - effect_pp
    # Paired design: assume the same discordance structure as m2 vs b.
    b_succ = df[df["condition"] == "b"].set_index(["item_id", "seed"])["success"]
    m2_succ = df[df["condition"] == "m2"].set_index(["item_id", "seed"])["success"]
    joint = pd.concat([b_succ.rename("b"), m2_succ.rename("m2")], axis=1).dropna()
    disc = float(((joint["b"] != joint["m2"]).mean())) if len(joint) else 0.25

    z_a = stats.norm.ppf(1 - alpha / 2)
    z_b = stats.norm.ppf(power)
    # McNemar sample size (Connor 1987): n = (z_a*sqrt(disc) + z_b*sqrt(disc - d^2))^2 / d^2
    d = effect_pp
    n = ((z_a * np.sqrt(disc) + z_b * np.sqrt(max(disc - d ** 2, 1e-9))) ** 2) / d ** 2

    # Observational power: what iota SD would be needed for the pooled slope?
    # WITHIN-condition SD (pooled), not the cross-condition SD: only the
    # within-condition spread is usable for identification.
    iota_sd = float(
        df[df["condition"].isin(["m1", "m3", "m4"])]
        .groupby("condition")["iota"]
        .std()
        .pow(2)
        .mean()
        ** 0.5
    )
    return {
        "p_base": p_base,
        "p_alt": p_alt,
        "discordance": disc,
        "n_items_needed": int(np.ceil(n)),
        "n_items_available": int(df["item_id"].nunique()),
        "observational_iota_sd": iota_sd,
    }


def cost_frontier(df: pd.DataFrame, extract_every: int = 5,
                  completion_per_call: int = 60) -> pd.DataFrame:
```

```
    """Accuracy against AMORTISED total tokens, not answering-call tokens.

    overhead_calls is a cumulative per-episode counter, so it is differenced
    within episode before being charged to an item (the same correction
    reconstruct_cost.py applies).
    """
    d = df.copy()
    d = d.sort_values(["condition", "seed", "episode", "item_id"])
    d["calls_item"] = (
        d.groupby(["condition", "seed", "episode"])["overhead_calls"].diff().fillna(
            d["overhead_calls"]
        )
    ).clip(lower=0)

    mean_turn_tokens = (d["memory_read_tokens"] / d["memory_items"].clip(lower=1)).mean()
    extract_prompt = 45 + extract_every * (mean_turn_tokens + 5)
    d["write_tokens"] = d["calls_item"] * (extract_prompt + completion_per_call)
    d["total_tokens"] = d["prompt_tokens"] + d["completion_tokens"] + d["write_tokens"]

    g = d.groupby("condition").agg(
        success=("success", "mean"),
        answer_tokens=("prompt_tokens", "mean"),
        write_tokens=("write_tokens", "mean"),
        total_tokens=("total_tokens", "mean"),
        calls_per_item=("calls_item", "mean"),
        latency_ms=("latency_ms", "mean"),
        n=("success", "size"),
    )
    base = g.loc["b"]
    g["answer_pct_vs_b"] = 100 * (g["answer_tokens"] - base["answer_tokens"]) / base["answer_tok
ens"]
    g["total_pct_vs_b"] = 100 * (g["total_tokens"] - base["total_tokens"]) / base["total_tokens"
]

    # Pareto frontier on (total_tokens down, success up).
    dominated = []
    for c in g.index:
        dom = any(
            (g.loc[o, "success"] >= g.loc[c, "success"])
            and (g.loc[o, "total_tokens"] <= g.loc[c, "total_tokens"])
            and (o != c)
            and (
                g.loc[o, "success"] > g.loc[c, "success"]
                or g.loc[o, "total_tokens"] < g.loc[c, "total_tokens"]
            )
            for o in g.index
        )
        dominated.append(dom)
    g["dominated"] = dominated
    return g


# ---------------------------------------------------------------------------
# figures
# ---------------------------------------------------------------------------


def figures(df: pd.DataFrame, cf: pd.DataFrame, frontier: pd.DataFrame,
            outdir: Path) -> None:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    outdir.mkdir(parents=True, exist_ok=True)
    order = ["b", "m1", "m2", "m3", "m4"]
    order = [c for c in order if c in df["condition"].unique()]

    # --- Figure: ordering ----------------------------------------------------
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))

    ax = axes[0]
    data = [df.loc[df["condition"] == c, "iota"].values for c in order]
    bp = ax.boxplot(data, labels=[c.upper() for c in order], showmeans=True,
                    patch_artist=True)
    for patch in bp["boxes"]:
        patch.set_facecolor("#dfe6ef")
    ax.set_ylabel(r"inversion rate $\iota$")
    ax.set_xlabel("condition")
```

```
    ax.set_title("(a) Each condition occupies its own narrow band of $\\iota$;\n"
                 "only B ($\\iota$=0) and M2 ($\\iota$=1) anchor the axis")
    ax.axhline(0, ls=":", lw=1, color="grey")
    ax.set_ylim(-0.05, 1.05)

    # (b) WITHIN condition only. Pooling across conditions here would smuggle
    # the architecture difference back in as if it were an ordering slope.
    ax = axes[1]
    colours = {"m1": "#26456e", "m3": "#1e8449", "m4": "#c0392b"}
    for cond in ["m1", "m3", "m4"]:
        sub = df[df["condition"] == cond].copy()
        if len(sub) < 30:
            continue
        sub["bin"] = pd.qcut(sub["iota"], 4, duplicates="drop")
        g = sub.groupby("bin", observed=True).agg(
            iota=("iota", "mean"), succ=("success", "mean"), n=("success", "size")
        )
        se = np.sqrt(g["succ"] * (1 - g["succ"]) / g["n"])
        ax.errorbar(g["iota"], g["succ"], yerr=1.96 * se, fmt="o-", capsize=3,
                    color=colours[cond], label=cond.upper(), alpha=0.85)
    ax.set_xlabel(r"inversion rate $\iota$ (quartile bins, within condition)")
    ax.set_ylabel("success rate")
    ax.set_title("(b) Within each architecture the slope is flat and noisy:\n"
                 "the logged spread cannot identify the ordering effect")
    ax.set_xlim(0, 1)
    ax.legend(fontsize=8, loc="lower left")
    ax.grid(alpha=0.3)

    fig.tight_layout()
    fig.savefig(outdir / "fig_ordering.png", dpi=200)
    plt.close(fig)

    # --- Figure: Pareto ------------------------------------------------------
    fig, ax = plt.subplots(figsize=(7, 5))
    for c in frontier.index:
        r = frontier.loc[c]
        ax.scatter(r["total_tokens"], r["success"], s=150,
                   color="#c0392b" if r["dominated"] else "#1e8449",
                   edgecolor="black", zorder=3)
        ax.annotate(c.upper(), (r["total_tokens"], r["success"]),
                    xytext=(8, 6), textcoords="offset points", fontweight="bold")
        ax.scatter(r["answer_tokens"], r["success"], s=50, marker="x",
                   color="grey", zorder=2)
        ax.plot([r["answer_tokens"], r["total_tokens"]], [r["success"]] * 2,
                ls=":", color="grey", lw=1, zorder=1)
    ax.set_xlabel("mean tokens per item\n"
                  "(x = answering call only, o = amortised total incl. write path)")
    ax.set_ylabel("success rate")
    ax.set_title("Cost-adjusted frontier: M3's saving is on the answering call,\n"
                 "not on total spend")
    ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(outdir / "fig_pareto.png", dpi=200)
    plt.close(fig)


# ---------------------------------------------------------------------------


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--file", default="results/rerun_2026_07_29/custom_ds_merged.jsonl")
    ap.add_argument("--out", default="results/ordering_decomposition.md")
    ap.add_argument("--figdir", default="results/figures")
    ap.add_argument("--boot", type=int, default=400)
    args = ap.parse_args()

    df = load(Path(args.file))
    models = fit_models(df)
    cf = counterfactual(df, models["m2"])
    boot = bootstrap_adjusted(df, n_boot=args.boot)
    frontier = cost_frontier(df)
    figures(df, cf, frontier, Path(args.figdir))

    pd.set_option("display.width", 200)
    pd.set_option("display.max_columns", 40)

    m2 = models["m2"]
```

```
    iota_coef = m2.params["iota"]
    iota_ci = m2.conf_int().loc["iota"].tolist()
    iota_p = m2.pvalues["iota"]

    L = []
    A = L.append
    A("# Ordering decomposition — separating presentation from architecture\n")
    A("_Generated by `scripts/ordering_decomposition.py`. Observational, not "
      "randomised — read the identification caveat in §4 before quoting._\n")

    A("## 1. The inversion rate varies WITHIN each condition\n")
    A("This is what makes the decomposition possible. If every item in a "
      "condition shared one layout, ordering and architecture would be "
      "perfectly collinear and nothing below could be estimated.\n")
    A("```")
    A(df.groupby("condition")["iota"].describe()[
        ["count", "mean", "std", "min", "25%", "50%", "75%", "max"]
    ].round(3).to_string())
    A("```\n")

    A("## 2. The ordering slope\n")
    A(f"Pooled logit, cluster-robust by episode, controlling for condition and "
      f"evidence_distance:\n")
    A(f"- coefficient on iota: **{iota_coef:.3f}** "
      f"(95% CI [{iota_ci[0]:.3f}, {iota_ci[1]:.3f}], p = {iota_p:.4g})")
    A(f"- odds ratio for a fully reversed vs fully chronological layout: "
      f"**{np.exp(iota_coef):.3f}** "
      f"[{np.exp(iota_ci[0]):.3f}, {np.exp(iota_ci[1]):.3f}]\n")

    if models.get("per_cond"):
        A("Estimated separately within each architecture — the mechanism is "
          "literally held constant, so it cannot be driving the slope:\n")
        A("```")
        pc = pd.DataFrame(models["per_cond"]).T
        pc = pc[["n", "iota_sd", "coef", "se", "p"]].astype(float).round(4)
        A(pc.to_string())
        A("```\n")

    if "m3" in models:
        fe = models["m3"]
        A(f"With item fixed effects (n = {models['fe_n']} records over "
          f"{models['fe_items']} items where two seeds of the same condition "
          f"drew different layouts), the iota coefficient is "
          f"**{fe.params['iota']:.3f}** (p = {fe.pvalues['iota']:.4g}). "
          f"This is the strictest available control for item difficulty.\n")
    elif "m3_error" in models:
        A(f"_Item-fixed-effects model did not converge: {models['m3_error']}_\n")
    else:
        A(f"_Item-fixed-effects model not estimable: only {models.get('fe_n', 0)} "
          f"records show within-item layout variation._\n")

    A("## 3. Decomposition of every contrast\n")
    A("`raw_vs_b` is the effect as currently reported. `adj_vs_b` is the same "
      "contrast with every condition set to the baseline's chronological "
      "layout (iota = 0). The difference is what presentation was worth.\n")
    A("```")
    A(cf[["n", "mean_iota", "sd_iota", "observed", "fitted_at_iota0",
          "raw_vs_b", "adj_vs_b", "attributable_to_ordering"]].round(4).to_string())
    A("```\n")
    if len(boot):
        A("Episode-cluster bootstrap CIs on the ordering-adjusted contrasts:\n")
        A("```")
        A(boot.round(4).to_string(index=False))
        A("```\n")

    A("## 3b. VERDICT — the observational route is underpowered, and that is "
      "the finding\n")
    conf = confounding_check(df)
    pw = power_for_arm(df)
    A("Two facts kill the observational identification, and both are worth "
      "stating explicitly in Ch3 rather than discovered by an examiner:\n")
    A(f"1. **Within-condition range is far too narrow.** The ordering effect "
      f"that matters spans iota = 0 to 1. The variation actually available "
      f"inside M1/M3/M4 has SD = {pw['observational_iota_sd']:.3f}, i.e. the "
      f"data only ever wobbles about +/- 9 points along a 100-point axis. The "
      f"pooled slope's CI "
      f"([{iota_ci[0]:.2f}, {iota_ci[1]:.2f}] in log-odds) is consistent with "
      f"anything from a large harm to a large benefit. It is not evidence of no "
```

```
      f"effect; it is an absence of evidence.\n")
    A(f"2. **Seeds do not re-randomise the layout.** Retrieval order is "
      f"deterministic given the item, so re-running the same item under a "
      f"different seed reproduces the same iota. Only "
      f"{models.get('fe_n', 0)} records show within-item layout variation, so "
      f"the item-fixed-effects estimator has nothing to work with. This is a "
      f"property of the harness, not of the data volume — more seeds would not "
      f"help.\n")
    A("Residual entanglement between iota and item difficulty:\n")
    A("```")
    A(conf.round(4).to_string())
    A("```\n")
    A("The positive within-M1 slope in §2 is very likely this confound rather "
      "than a real benefit of scrambling: items whose evidence sits further "
      "back get scrambled more, and `evidence_distance` independently predicts "
      "success with a positive coefficient. Do not report it as a finding.\n")
    A("**Power required for the experimental arm.** A paired `b_reversed` vs "
      "`b` McNemar test, sized on the discordance rate actually observed in "
      f"the M2-vs-B contrast ({pw['discordance']:.3f}), needs "
      f"**n = {pw['n_items_needed']} items** to detect the "
      f"{100 * 0.119:.1f}pp ordering effect at 80% power, alpha = 0.05. The "
      f"Mode B generator already produces {pw['n_items_available']} distinct "
      f"items, so **the arm is adequately powered on a single seed** and costs "
      f"one run of one condition. That is the cheapest defensible answer "
      f"available to this project.\n")

    A("## 4. Identification caveat — quote this alongside the numbers\n")
    A("iota is observational. It is determined by where the evidence happens to "
      "sit in the episode, so it correlates with `evidence_distance`, which "
      "independently predicts success. Every model conditions on "
      "evidence_distance and the fixed-effects model additionally holds the item "
      "constant, but residual confounding by unobserved item difficulty cannot "
      "be excluded. These are ordering-ADJUSTED contrasts, not causal "
      "architecture effects. The registered `b_reversed` arm and a "
      "`chronological_order: true` run of M1/M3/M4 remain the experimental test; "
      "this analysis pre-registers what that test should find.\n")

    A("## 5. Cost-adjusted frontier\n")
    A("`answer_tokens` is what Ch4 currently reports. `total_tokens` amortises "
      "the write path using per-item extraction calls differenced within "
      "episode (overhead_calls is cumulative).\n")
    A("```")
    A(frontier.round(3).to_string())
    A("```\n")

    A("## 6. Figures\n")
    A("- `results/figures/fig_ordering.png` — within-condition iota spread and "
      "the fitted success-vs-iota relationship\n")
    A("- `results/figures/fig_pareto.png` — accuracy against answering-call vs "
      "amortised total cost\n")

    out = Path(args.out)
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text("\n".join(L), encoding="utf-8")
    print("\n".join(L))
    print(f"\n[written] {out}")


if __name__ == "__main__":
    main()
```

```python
#!/usr/bin/env python
"""Reconstruct TRUE per-item LLM cost, including the write path (B3).

    python scripts/reconstruct_cost.py --file results/runs/custom_ds.jsonl

WHY THIS EXISTS
Ch3 defines token cost as "total prompt and completion tokens per item", but
`prompt_tokens` in the run records is the ANSWERING call only. M3 additionally
makes ~21.6 extraction calls per item, none of which are counted. The reported
"45% token saving" is therefore a saving on the answering call, not on cost as
Ch3 defines it, and the Ch5 "strictly better default" / efficient-frontier
argument rests on it.

A NOTE ON memory_written_tokens
The expert review treated M3's mean `memory_written_tokens` (~1,171) as its
write-path cost. It is not: every condition logs the same figure — the baseline
logs ~1,171 too — because it counts tokens written into the LOCAL store, which
for B/M1/M4 is a Python list and costs nothing. Adding it to M3 alone would
overstate the correction. The real uncounted spend is the extraction CALLS.

WHAT THIS SCRIPT DOES
Reconstructs extraction spend analytically from what IS logged:
    extraction prompt  ~= template + (extract_every turns x mean tokens/turn)
    extraction output  ~= facts emitted, bounded by the per-call cap
    per item           = overhead_calls x (prompt + output)
This is an ESTIMATE with stated assumptions and it is the acceptable route only
if you say so in Ch3. The clean route is to re-run M3 with the P3-metered runner
(`scripts/run_remediation.py --with-m3`), which logs total_prompt_tokens
directly. If those columns are present this script reports the measured figures
and flags the estimate as unnecessary.
"""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import pandas as pd

# EXTRACT_PROMPT in semantic.py, minus the {chunk} placeholder.
EXTRACT_TEMPLATE_TOKENS = 45
# Each excerpt line is prefixed "[role step N] ".
LINE_PREFIX_TOKENS = 5


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--file", default="results/runs/custom_ds.jsonl")
    ap.add_argument("--extract-every", type=int, default=5,
                    help="must match configs/conditions/m3.yaml")
    ap.add_argument("--completion-per-call", type=int, default=60,
                    help="assumed mean extractor output tokens per call")
    args = ap.parse_args()

    path = Path(args.file)
    rows = [json.loads(ln) for ln in path.read_text(encoding="utf-8").splitlines()
            if ln.strip()]
    df = pd.DataFrame(rows)

    if "total_prompt_tokens" in df.columns and df["total_prompt_tokens"].notna().any():
        print("total_prompt_tokens is present — reporting MEASURED cost.\n")
        g = df.groupby("condition").agg(
            measured_prompt=("total_prompt_tokens", "mean"),
            measured_completion=("total_completion_tokens", "mean"),
            llm_calls=("total_llm_calls", "mean")).round(1)
        g["total"] = (g.measured_prompt + g.measured_completion).round(1)
        base = g.loc["b", "total"]
        g["vs_baseline_%"] = (100 * (g.total - base) / base).round(1)
        print(g.to_string())
        return

    print("total_prompt_tokens NOT logged — reconstructing analytically.")
    print(f"assumptions: extract_every={args.extract_every}, "
          f"template={EXTRACT_TEMPLATE_TOKENS} tok/call, "
          f"output={args.completion_per_call} tok/call\n")

    df["tokens_per_turn"] = df["memory_written_tokens"] / df["memory_items"].clip(lower=1)
```

```
 79
 80      # `overhead_calls` is memory.stats().overhead_calls — a CUMULATIVE per-episode
 81      # counter, so within an episode it rises 16, 17, 18, ... across queries. Using
 82      # it as a per-item figure charges every query for the whole episode's
 83      # extraction and overstates M3's spend roughly fourfold. Difference it within
 84      # (condition, seed, episode) ordered by query step to get the calls actually
 85      # incurred since the previous query — the amortisation the runner's own
 86      # comment describes.
 87      df["episode_id"] = df["item_id"].str.split(":").str[0]
 88      df["q_step"] = (df["item_id"].str.split(":q").str[-1]
 89                      .astype(float, errors="ignore"))
 90      df = df.sort_values(["condition", "seed", "episode_id", "q_step"])
 91      grp = df.groupby(["condition", "seed", "episode_id"])["overhead_calls"]
 92      df["overhead_calls_incremental"] = (
 93          grp.diff().fillna(df["overhead_calls"]).clip(lower=0))
 94
 95      g = df.groupby("condition").agg(
 96          answer_prompt=("prompt_tokens", "mean"),
 97          answer_completion=("completion_tokens", "mean"),
 98          overhead_cumulative=("overhead_calls", "mean"),
 99          overhead_calls=("overhead_calls_incremental", "mean"),
100          tokens_per_turn=("tokens_per_turn", "mean"),
101      )
102
103      chunk = args.extract_every * (g.tokens_per_turn + LINE_PREFIX_TOKENS)
104      per_call = EXTRACT_TEMPLATE_TOKENS + chunk + args.completion_per_call
105      g["extraction_est"] = (g.overhead_calls * per_call).round(1)
106
107      g["reported_total"] = (g.answer_prompt + g.answer_completion).round(1)
108      g["true_total_est"] = (g.reported_total + g.extraction_est).round(1)
109
110      base_rep = g.loc["b", "reported_total"]
111      base_true = g.loc["b", "true_total_est"]
112      g["reported_vs_b_%"] = (100 * (g.reported_total - base_rep) / base_rep).round(1)
113      g["TRUE_vs_b_%"] = (100 * (g.true_total_est - base_true) / base_true).round(1)
114
115      cols = ["answer_prompt", "answer_completion", "overhead_cumulative",
116              "overhead_calls", "extraction_est", "reported_total", "true_total_est",
117              "reported_vs_b_%", "TRUE_vs_b_%"]
118      print(g[cols].round(1).to_string())
119      print("\noverhead_cumulative = the raw logged counter (whole episode to date)")
120      print("overhead_calls      = incremental calls charged to THIS query (amortised)")
121
122      if "m3" in g.index:
123          rep, true = g.loc["m3", "reported_vs_b_%"], g.loc["m3", "TRUE_vs_b_%"]
124          print(f"\nM3: reported {rep:+.1f}% vs baseline on the answering call alone;")
125          print(f"    {true:+.1f}% once extraction calls are included.")
126          if true > -10:
127              print("\n    The 45% saving does not survive. Withdraw the abstract claim,")
128              print("    the Ch4 'dominating the baseline outright' line, and the Ch5")
129              print("    efficient-frontier / £0.0099 -> £0.0055 costing.")
130          else:
131              print("\n    A saving survives, but it is smaller than reported. State it")
132              print("    as amortised per episode and show the write path explicitly.")
133
134      print("\nSensitivity — TRUE_vs_b_% for M3 under other output assumptions:")
135      for out_tok in (20, 40, 60, 80, 120):
136          pc = EXTRACT_TEMPLATE_TOKENS + chunk + out_tok
137          tot = g.reported_total + g.overhead_calls * pc
138          v = 100 * (tot.loc["m3"] - tot.loc["b"]) / tot.loc["b"]
139          print(f"  {out_tok:>4} tok/call -> {v:+6.1f}%")
140
141
142  if __name__ == "__main__":
143      main()
```

```
#!/usr/bin/env python
"""Entry point. One command per (task, condition, seed) cell; resumable.

Examples:
  python scripts/run_experiment.py --task custom_ds --condition m1 --seed 1
  python scripts/run_experiment.py --task longmemeval_s --condition b --seed 1 --limit 20
  python scripts/run_experiment.py --task custom_ds --all-conditions --seed 1
"""

from __future__ import annotations

import argparse
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

import yaml  # noqa: E402
from dotenv import load_dotenv  # noqa: E402

from agentmem.harness.runner import run_mode_a, run_mode_b  # noqa: E402
from agentmem.llm.client import make_llm  # noqa: E402
from agentmem.logging.trajectory import TrajectoryWriter  # noqa: E402
from agentmem.memory import ALL_ARMS, CONDITIONS  # noqa: E402
from agentmem.tasks.custom_ds import generate_episodes  # noqa: E402
from agentmem.tasks.longmemeval import load_longmemeval  # noqa: E402
from agentmem.utils.manifest import write_manifest  # noqa: E402


def load_yaml(path: Path) -> dict:
    return yaml.safe_load(path.read_text(encoding="utf-8")) or {}


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--task", required=True, choices=["longmemeval_s", "custom_ds"])
    # --condition accepts control arms (e.g. b_reversed); --all-conditions does
    # NOT, so the registered main grid stays exactly the five pre-specified arms.
    ap.add_argument("--condition", choices=list(ALL_ARMS))
    ap.add_argument("--all-conditions", action="store_true")
    ap.add_argument("--seed", type=int, default=1)
    ap.add_argument("--limit", type=int, default=None, help="cap items/episodes (pilot)")
    ap.add_argument("--config", default="configs/base.yaml")
    ap.add_argument("--out", default=None, help="results JSONL (default per task)")
    args = ap.parse_args()
    if not args.condition and not args.all_conditions:
        ap.error("need --condition or --all-conditions")

    load_dotenv(ROOT / ".env")
    config = load_yaml(ROOT / args.config)
    task_cfg = load_yaml(ROOT / "configs" / "tasks" / f"{args.task}.yaml")
    out = Path(args.out) if args.out else ROOT / "results" / "runs" / f"{args.task}.jsonl"
    writer = TrajectoryWriter(out)
    llm = make_llm(config)
    conditions = list(CONDITIONS) if args.all_conditions else [args.condition]

    if args.task == "custom_ds":
        episodes = generate_episodes(
            n_episodes=task_cfg.get("n_episodes", 10), seed=args.seed,
            n_steps=task_cfg.get("n_steps", 120),
            n_projects=task_cfg.get("n_projects", 6),
            n_queries=task_cfg.get("n_queries", 10),
            update_prob=task_cfg.get("update_prob", 0.25),
            distractor_prob=task_cfg.get("distractor_prob", 0.35),
        )[: args.limit]
        run = lambda cond, params: run_mode_b(  # noqa: E731
            episodes, cond, llm, config, writer, seed=args.seed, condition_params=params)
    else:
        data_file = ROOT / task_cfg["data_file"]
        if not data_file.exists():
            sys.exit(f"Missing {data_file}. See data/README.md for download steps.")
        items = load_longmemeval(
            data_file, cap=task_cfg.get("cap"), seed=args.seed,
            exclude_types=tuple(task_cfg.get("exclude_types", [])),
        )[: args.limit]
        run = lambda cond, params: run_mode_a(  # noqa: E731
            items, cond, llm, config, writer, seed=args.seed, condition_params=params)
```

```
79
80      write_manifest(out.parent, {**config, "task": task_cfg, "cli": vars(args)})
81      for cond in conditions:
82          cond_cfg = load_yaml(ROOT / "configs" / "conditions" / f"{cond}.yaml")
83          n = run(cond, cond_cfg.get("params", {}))
84          print(f"[{args.task}] condition={cond} seed={args.seed}: "
85                f"{n} new records -> {out}  (spent £{llm.cost.gbp_spent:.4f})")
86
87
88  if __name__ == "__main__":
89      main()
```

```python
#!/usr/bin/env python
"""ONE COMMAND reruns the entire experiment (FASER reproducibility requirement):

  python scripts/run_full_grid.py

For every seed in configs/base.yaml x every task with data x every condition:
run (resumable — completed cells are skipped), then rebuild the stats report,
the figures, and the EDA export. Safe to interrupt and rerun.
"""

from __future__ import annotations

import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

import yaml  # noqa: E402


def sh(*args: str) -> None:
    print("+", " ".join(args))
    subprocess.run([sys.executable, *args], check=True, cwd=ROOT)


def main() -> None:
    config = yaml.safe_load((ROOT / "configs" / "base.yaml").read_text(encoding="utf-8"))
    seeds = config.get("seeds", [1])
    tasks = ["custom_ds"]
    if (ROOT / "data" / "longmemeval_s.json").exists():
        tasks.append("longmemeval_s")
    else:
        print("NOTE: data/longmemeval_s.json missing -> skipping Mode A "
              "(see data/README.md for the download command)")

    for task in tasks:
        for seed in seeds:
            sh("scripts/run_experiment.py", "--task", task,
               "--all-conditions", "--seed", str(seed))

    sh("scripts/make_report.py")
    sh("scripts/make_figures.py")
    sh("scripts/export_for_eda.py")
    print("\nFull grid complete. See results/report.md, results/figures/, "
          "results/traces.jsonl")


if __name__ == "__main__":
    main()
```

```python
#!/usr/bin/env python
"""Pilot: small all-conditions grid on the custom task (and LongMemEval if the
data file is present), then print the headline summary table.

Mock mode by default (configs/base.yaml provider: mock) — validates the whole
pipeline on a CPU laptop with zero cost. Use it to fix budgets before real runs.

  python scripts/run_pilot.py
"""

from __future__ import annotations

import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

from agentmem.logging.trajectory import load_records  # noqa: E402


def sh(*args: str) -> None:
    subprocess.run([sys.executable, *args], check=True, cwd=ROOT)


def main() -> None:
    sh("scripts/run_experiment.py", "--task", "custom_ds",
       "--all-conditions", "--seed", "1", "--limit", "3")
    if (ROOT / "data" / "longmemeval_s.json").exists():
        sh("scripts/run_experiment.py", "--task", "longmemeval_s",
           "--all-conditions", "--seed", "1", "--limit", "10")

    sys.path.insert(0, str(ROOT))
    from analysis.stats import failure_crosstab, records_to_df, summary_table

    for task in ["custom_ds", "longmemeval_s"]:
        recs = load_records(ROOT / "results" / "runs" / f"{task}.jsonl")
        if not recs:
            continue
        df = records_to_df(recs)
        print(f"\n=== {task}: headline summary ===")
        print(summary_table(df).to_string())
        print(f"\n=== {task}: failure tags (row-normalised) ===")
        print(failure_crosstab(df).to_string())


if __name__ == "__main__":
    main()
```

```
#!/usr/bin/env python
"""ONE COMMAND for the post-review Mode B re-runs (R1 + R2).

    python scripts/run_remediation.py --dry-run     # show the plan, touch nothing
    python scripts/run_remediation.py               # do it

WHAT IT DOES, AND WHY

R1 — re-run M2 on Mode B, all seeds.
    Two patches changed M2's behaviour, so every existing M2 Mode B record is
    void and must be regenerated:
      P1  summary.py now restores chronological prompt order (it previously
          returned turns newest-first and never re-sorted, so on histories
          below the fold threshold M2 was the baseline with reversed turns).
      P2  the fold threshold moved 1500 -> 800, so the summariser actually
          fires. At 1500 it fired on 0/258 items; at 800 it fires on 258/258.
    Because TrajectoryWriter resumes by (condition, item_id, seed, mode), the
    old M2 rows are dropped first — otherwise the re-run is a silent no-op.

R2 — run the b_reversed ordering-only control, all seeds.
    Identical turn selection to b, reverse-chronological prompt layout. Isolates
    the ordering effect that was previously confounded with M2, and pre-empts
    "how do you know M2's deficit isn't ordering?" in the viva.

NOT DONE HERE
    M3's Mode B re-run (R3) is optional — P3 changed only what is logged, not
    what the model does, so M3's success rates are unaffected. Add --with-m3 if
    you want true total_prompt_tokens for M3 rather than reconstructing cost
    analytically. Mode A (R4) is a separate, much longer job: finish M1 seeds 1
    and 3 before adding M2-M4, and keep the design balanced.

Every step is resumable and safe to interrupt. Drops are backed up to
results/runs/*.jsonl.<timestamp>.bak before anything is rewritten.
"""

from __future__ import annotations

import argparse
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

import yaml  # noqa: E402

RUNS = ROOT / "results" / "runs" / "custom_ds.jsonl"


def sh(args: list[str], dry: bool) -> None:
    print("+", "python", " ".join(args))
    if not dry:
        subprocess.run([sys.executable, *args], check=True, cwd=ROOT)


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--dry-run", action="store_true",
                    help="print the plan without running or dropping anything")
    ap.add_argument("--with-m3", action="store_true",
                    help="also re-run M3 so total_prompt_tokens is measured, not "
                         "reconstructed (R3 — optional, see module docstring)")
    ap.add_argument("--seeds", type=int, nargs="*", default=None,
                    help="override seeds (default: seeds from configs/base.yaml)")
    ap.add_argument("--skip-report", action="store_true",
                    help="don't rebuild report/figures at the end")
    args = ap.parse_args()
    dry = args.dry_run

    config = yaml.safe_load((ROOT / "configs" / "base.yaml").read_text(encoding="utf-8"))
    seeds = args.seeds if args.seeds is not None else config.get("seeds", [1])
    provider = config.get("provider", "?")

    rerun = ["m2"] + (["m3"] if args.with_m3 else [])

    print(f"\nprovider={provider}  seeds={seeds}  target={RUNS.relative_to(ROOT)}")
    if provider == "mock":
```

```
        print("\n!! provider is 'mock' — these runs would NOT be citable.\n"
              "   Set the real provider in configs/base.yaml before running.")
        if not dry:
            raise SystemExit(1)
    print(f"\nplan:\n  R1  drop + re-run {', '.join(c.upper() for c in rerun)} "
          f"on custom_ds, seeds {seeds}\n"
          f"  R2  run b_reversed (ordering control) on custom_ds, seeds {seeds}\n")

    # --- R1: clear the void rows, then regenerate them ----------------------
    for cond in rerun:
        sh(["scripts/drop_condition.py", "--file", str(RUNS), "--condition", cond], dry)
    for cond in rerun:
        for seed in seeds:
            sh(["scripts/run_experiment.py", "--task", "custom_ds",
                "--condition", cond, "--seed", str(seed)], dry)

    # --- R2: the ordering-only control arm ----------------------------------
    for seed in seeds:
        sh(["scripts/run_experiment.py", "--task", "custom_ds",
            "--condition", "b_reversed", "--seed", str(seed)], dry)

    if not args.skip_report:
        sh(["scripts/make_report.py"], dry)
        sh(["scripts/make_figures.py"], dry)

    print("\ndone." if not dry else "\ndry run — nothing was run or dropped.")
    print("Next: rewrite Ch4 §4.4 (real M2), §4.6 (ordering control), §4.3 "
          "(corrected token cost) and the abstract around the new numbers.")


if __name__ == "__main__":
    main()
```

```python
#!/usr/bin/env python
"""Judge validation protocol (reported in Ch3): is the automated grader
trustworthy enough to carry the headline metric?

Two-phase workflow:
  1. SAMPLE: draw a stratified sample of graded records and write
      results/judge_validation_sample.csv with an empty `human_label` column.
        python scripts/validate_judge.py sample --n 50
      -> fill human_label with 1 (correct) / 0 (incorrect), blind to the
         judge's verdict (the judge column is withheld in a separate key file).
  2. SCORE: compute agreement + Cohen's kappa between human and judge.
        python scripts/validate_judge.py score
"""

from __future__ import annotations

import argparse
import random
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

import pandas as pd  # noqa: E402

from agentmem.logging.trajectory import load_records  # noqa: E402

SAMPLE_CSV = ROOT / "results" / "judge_validation_sample.csv"
KEY_CSV = ROOT / "results" / "judge_validation_key.csv"  # judge verdicts, kept apart


def sample(n: int, seed: int) -> None:
    records = []
    for f in sorted((ROOT / "results" / "runs").glob("*.jsonl")):
        records.extend(load_records(f))
    if not records:
        sys.exit("No run records to sample from.")
    rng = random.Random(seed)
    succ = [r for r in records if r["success"]]
    fail = [r for r in records if not r["success"]]
    rng.shuffle(succ), rng.shuffle(fail)
    chosen = succ[: n // 2] + fail[: n - n // 2]
    rng.shuffle(chosen)

    blind = pd.DataFrame([{
        "run_id": r["run_id"], "question": r["question"],
        "gold": r["gold"], "prediction": r["prediction"], "human_label": "",
    } for r in chosen])
    key = pd.DataFrame([{"run_id": r["run_id"], "judge_label": int(r["success"])}
                        for r in chosen])
    SAMPLE_CSV.parent.mkdir(parents=True, exist_ok=True)
    blind.to_csv(SAMPLE_CSV, index=False)
    key.to_csv(KEY_CSV, index=False)
    print(f"Wrote {len(blind)} rows -> {SAMPLE_CSV}")
    print(f"Fill `human_label` (1/0) WITHOUT looking at {KEY_CSV.name}, then run:")
    print("  python scripts/validate_judge.py score")


def score() -> None:
    blind = pd.read_csv(SAMPLE_CSV)
    key = pd.read_csv(KEY_CSV)
    df = blind.merge(key, on="run_id")
    df = df.dropna(subset=["human_label"])
    if df.empty:
        sys.exit("human_label column is still empty.")
    h = df["human_label"].astype(int)
    j = df["judge_label"].astype(int)
    agree = float((h == j).mean())
    po = agree
    pe = float((h.mean() * j.mean()) + ((1 - h.mean()) * (1 - j.mean())))
    kappa = (po - pe) / (1 - pe) if pe < 1 else float("nan")
    print(f"n={len(df)}  raw agreement={agree:.3f}  Cohen's kappa={kappa:.3f}")
    print("Disagreements (inspect + discuss in Ch3):")
    for _, row in df[h != j].iterrows():
        print(f"  {row.run_id}: judge={row.judge_label} human={int(row.human_label)} "
              f"| gold={row.gold!r} pred={str(row.prediction)[:60]!r}")

```

```
 79
 80  def main() -> None:
 81      ap = argparse.ArgumentParser()
 82      sub = ap.add_subparsers(dest="cmd", required=True)
 83      s = sub.add_parser("sample")
 84      s.add_argument("--n", type=int, default=50)
 85      s.add_argument("--seed", type=int, default=1)
 86      sub.add_parser("score")
 87      args = ap.parse_args()
 88      if args.cmd == "sample":
 89          sample(args.n, args.seed)
 90      else:
 91          score()
 92
 93
 94  if __name__ == "__main__":
 95      main()
```

```python
#!/usr/bin/env python
"""Checks for the b/seed-1 missing-record patch. Avoids fighting PowerShell quoting.

    python scripts/verify_patch.py before    # inspect the downloaded patch record
    python scripts/verify_patch.py after     # verify the merged dataset is 1800/1800
"""

from __future__ import annotations

import collections
import json
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
MERGED = ROOT / "results" / "rerun_2026_07_29" / "longmemeval_s_merged.jsonl"
PATCH = ROOT / "results" / "patch_b_s1" / "patch_only.jsonl"
ITEM = "gpt4_78cf46a3"


def load(p: Path) -> list[dict]:
    if not p.exists():
        sys.exit(f"FAIL: {p} not found")
    return [json.loads(ln) for ln in p.read_text(encoding="utf-8").splitlines() if ln.strip()]


def before() -> None:
    recs = load(PATCH)
    if len(recs) != 1:
        sys.exit(f"FAIL: expected exactly 1 record in {PATCH.name}, got {len(recs)}")
    r = recs[0]
    got = (r["condition"], r["seed"], r["mode"], r["item_id"])
    want = ("b", 1, "A", ITEM)
    if got != want:
        sys.exit(f"FAIL: record identity {got} != {want}")
    if r.get("prompt_version") != "v1.0":
        sys.exit(f"FAIL: prompt_version is {r.get('prompt_version')!r}, expected 'v1.0' — "
                 "the prompt changed, this record is not comparable to the other 1,799")
    if r.get("gbp_spent") is None or r.get("prompt_tokens", 0) <= 0:
        sys.exit("FAIL: cost fields look unpopulated")
    # A mock-provider record is detectable: mock latency is sub-millisecond.
    if float(r.get("latency_ms", 0)) < 50:
        sys.exit(f"FAIL: latency_ms={r['latency_ms']} is implausibly low for vLLM — "
                 "this looks like a mock record, do NOT merge it")
    print(f"OK  identity      {got}")
    print(f"OK  prompt_version {r['prompt_version']}")
    print(f"    success={r['success']}  failure_tag={r['failure_tag']}")
    print(f"    prompt_tokens={r['prompt_tokens']}  latency_ms={r['latency_ms']}")
    print(f"    memory_read_tokens={r['memory_read_tokens']} (DP1 budget 1200)")
    print("\nSafe to merge.")


def after() -> None:
    recs = load(MERGED)
    problems = []
    if len(recs) != 1800:
        problems.append(f"record count is {len(recs)}, expected 1800")
    cells = collections.Counter((r["condition"], r["seed"]) for r in recs if r["mode"] == "A")
    bad = {k: v for k, v in cells.items() if v != 120}
    if bad:
        problems.append(f"cells not at 120: {bad}")
    keys = {(r["condition"], r["item_id"], r["seed"], r["mode"]) for r in recs}
    if len(keys) != len(recs):
        problems.append(f"{len(recs) - len(keys)} duplicate keys")
    if not any(r["item_id"] == ITEM and r["condition"] == "b" and r["seed"] == 1 for r in recs):
        problems.append(f"{ITEM} still missing from b/seed 1")
    if problems:
        for p in problems:
            print(f"FAIL: {p}")
        sys.exit(1)

    b = [r for r in recs if r["condition"] == "b" and r["mode"] == "A"]
    hits = sum(1 for r in b if r["success"])
    print(f"OK  1800/1800 records, all 15 cells at 120, no duplicate keys")
    print(f"OK  {ITEM} present in b/seed 1")
    print(f"\nBaseline Mode A success rate is now {hits}/{len(b)} = {hits / len(b):.4f}")
    print("(was 14/359 = 0.0390 — update Table 4.2 and the audit report to match)")

```

```

if __name__ == "__main__":
    mode = sys.argv[1] if len(sys.argv) > 1 else ""
    if mode == "before":
        before()
    elif mode == "after":
        after()
    else:
        sys.exit(__doc__)
```

```
import sys
from pathlib import Path

import pytest

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
sys.path.insert(0, str(ROOT))

from agentmem.llm.client import MockLLM  # noqa: E402
from agentmem.memory import CONDITIONS, make_memory  # noqa: E402
from agentmem.memory.base import Turn  # noqa: E402


@pytest.fixture
def llm():
    return MockLLM()


@pytest.fixture
def turns():
    """Long fact-laden stream: 60 turns, facts + distractors."""
    out = []
    for i in range(60):
        if i % 3 == 0:
            content = f"Update: project P{i} budget is set to £{100 + i}k."
        else:
            content = "The all-hands meeting moved to the main auditorium."
        out.append(Turn(role="system", content=content, step=i))
    return out


def all_memories(llm):
    return {c: make_memory(c, {}, llm=llm) for c in CONDITIONS}
```

```python
"""DP1: read() NEVER exceeds budget_tokens, in any condition, at any budget."""

import pytest
from conftest import all_memories

from agentmem.utils.tokens import count_tokens


@pytest.mark.parametrize("budget", [10, 50, 200, 1200])
def test_read_respects_budget(llm, turns, budget):
    for cond, mem in all_memories(llm).items():
        mem.reset()
        for t in turns:
            mem.write(t)
        ctx = mem.read("What is the budget of project P0?", budget_tokens=budget)
        assert ctx.tokens <= budget, f"{cond}: {ctx.tokens} > {budget}"
        # Reported tokens must be consistent with the returned text.
        assert count_tokens(ctx.text) <= budget + 5, cond


def test_zero_budget_returns_empty(llm, turns):
    for cond, mem in all_memories(llm).items():
        mem.reset()
        for t in turns[:10]:
            mem.write(t)
        ctx = mem.read("anything?", budget_tokens=0)
        assert ctx.text == "", cond
        assert ctx.tokens == 0, cond
```

```python
"""Loader + Mode-A regression tests against a fixture in the OFFICIAL
LongMemEval schema (tests/fixtures/longmemeval_fixture.json, 12 items,
2 per question type). Guards the real-data path before the download lands.
"""

from pathlib import Path

from conftest import ROOT

from agentmem.harness.runner import run_mode_a
from agentmem.logging.trajectory import TrajectoryWriter, load_records
from agentmem.tasks.longmemeval import load_longmemeval

FIXTURE = Path(ROOT) / "tests" / "fixtures" / "longmemeval_fixture.json"


def test_loader_parses_official_schema():
    items = load_longmemeval(FIXTURE, seed=1)
    assert len(items) == 12
    for it in items:
        assert it.question and it.answer and it.turns
        assert it.evidence_steps, f"{it.item_id}: no evidence steps"
        assert all(0 <= s < len(it.turns) for s in it.evidence_steps)
        # Evidence turns really contain has_answer content from answer sessions.
        assert all("[" in it.turns[s].content for s in it.evidence_steps)


def test_stratified_cap_preserves_type_mix():
    items = load_longmemeval(FIXTURE, cap=6, seed=1)
    types = [i.question_type for i in items]
    assert len(items) == 6
    assert len(set(types)) == 6, "cap must keep one of each type (stratified)"
    # Deterministic for a fixed seed.
    again = load_longmemeval(FIXTURE, cap=6, seed=1)
    assert [i.item_id for i in items] == [i.item_id for i in again]


def test_mode_a_end_to_end_on_fixture(llm, tmp_path):
    items = load_longmemeval(FIXTURE, cap=4, seed=1)
    writer = TrajectoryWriter(tmp_path / "out.jsonl")
    cfg = {"budgets": {"memory_read_tokens": 300}}
    n = run_mode_a(items, "m1", llm, cfg, writer, seed=1, condition_params={})
    assert n == 4
    recs = load_records(tmp_path / "out.jsonl")
    assert all(r["mode"] == "A" and r["condition"] == "m1" for r in recs)
    assert all(r["memory_read_tokens"] <= 300 for r in recs)
    # Resumability: rerun writes nothing new.
    writer2 = TrajectoryWriter(tmp_path / "out.jsonl")
    assert run_mode_a(items, "m1", llm, cfg, writer2, seed=1, condition_params={}) == 0
```

```python
"""Interface conformance + architecture-specific behaviour for every condition."""

from conftest import all_memories

from agentmem.memory.base import MemoryContext, MemoryStats, Turn


def test_interface_conformance(llm, turns):
    for cond, mem in all_memories(llm).items():
        mem.reset()
        for t in turns[:12]:
            mem.write(t)
        ctx = mem.read("What is the budget of project P0?", budget_tokens=500)
        assert isinstance(ctx, MemoryContext), cond
        assert isinstance(ctx.provenance, list), cond
        st = mem.stats()
        assert isinstance(st, MemoryStats), cond
        assert st.items == 12, cond
        assert st.tokens_written > 0, cond
        mem.reset()
        assert mem.stats().items == 0, cond
        assert mem.read("q?", 100).text == "", f"{cond}: reset must clear state"


def test_baseline_keeps_most_recent(llm):
    from agentmem.memory.none import NoMemory

    mem = NoMemory()
    for i in range(50):
        mem.write(Turn(role="user", content=f"message number {i} with padding words",
                       step=i))
    ctx = mem.read("irrelevant?", budget_tokens=60)
    steps = [p["step"] for p in ctx.provenance]
    assert steps, "baseline returned nothing"
    assert min(steps) > 30, f"recency window should hold recent turns, got {steps}"


def test_episodic_retrieves_relevant_over_recent(llm):
    from agentmem.memory.episodic import EpisodicMemory

    mem = EpisodicMemory()
    mem.write(Turn(role="system",
                   content="Update: project Falcon budget is set to £250k.", step=0))
    for i in range(1, 40):
        mem.write(Turn(role="system",
                       content="The all-hands meeting moved to the auditorium.", step=i))
    ctx = mem.read("What is the budget of project Falcon?", budget_tokens=60)
    assert any(p["step"] == 0 for p in ctx.provenance), \
        "episodic memory should retrieve the old relevant turn"


def test_summary_folds_and_charges_overhead(llm):
    from agentmem.memory.summary import SummaryMemory

    mem = SummaryMemory(llm=llm, buffer_token_threshold=100)
    for i in range(30):
        mem.write(Turn(role="system",
                       content=f"Update: project Nimbus budget is set to £{i}k.", step=i))
    assert mem.stats().overhead_calls > 0, "summariser never ran"
    ctx = mem.read("What is the budget of project Nimbus?", budget_tokens=400)
    assert any(p["kind"] == "summary" for p in ctx.provenance)


def test_semantic_update_on_conflict(llm):
    from agentmem.memory.semantic import SemanticMemory

    mem = SemanticMemory(llm=llm, extract_every=1)
    mem.write(Turn(role="system",
                   content="Update: project Zephyr budget is set to £100k.", step=0))
    mem.write(Turn(role="system",
                   content="Update: project Zephyr budget is set to £999k.", step=1))
    ctx = mem.read("What is the budget of project Zephyr?", budget_tokens=300)
    assert "999" in ctx.text, "newest value must win"
    assert "100k" not in ctx.text, "stale value should have been replaced"


def test_decay_prefers_recent_among_equally_relevant(llm):
    from agentmem.memory.decay import DecayMemory
```

```
 79
 80      mem = DecayMemory(decay_lambda=0.1)
 81      mem.write(Turn(role="system",
 82                     content="Update: project Vortex status is blocked.", step=0))
 83      mem.write(Turn(role="system",
 84                     content="Update: project Vortex status is on track.", step=50))
 85      ctx = mem.read("What is the status of project Vortex?", budget_tokens=20)
 86      assert ctx.provenance and ctx.provenance[0]["step"] == 50, \
 87          "decay should rank the recent duplicate first"
 88
 89
 90  def test_declared_prompt_ordering_policy(llm):
 91      """Pin each condition's prompt ORDER, because ordering is a live confound.
 92
 93      B and M2 lay turns out chronologically. M1 and M4 lay them out in score
 94      order by default — which is why any B-vs-M1 or B-vs-M4 contrast currently
 95      mixes architecture with presentation, the same way the pre-patch M2
 96      contrast did. This test exists so that if anyone changes an ordering policy,
 97      they are forced to notice that the affected results need regenerating.
 98      """
 99      from agentmem.memory import make_memory
100
101      chronological = {"b": True, "m2": True, "m1": False, "m3": False, "m4": False}
102      for cond, expect_chrono in chronological.items():
103          mem = make_memory(cond, {}, llm=llm)
104          for i in range(30):
105              mem.write(Turn(role="system",
106                             content=f"Update: project Nimbus milestone {i} logged.",
107                             step=i))
108          ctx = mem.read("What milestone was logged?", budget_tokens=300)
109          steps = [p["step"] for p in ctx.provenance if p.get("kind") != "summary"]
110          if expect_chrono:
111              assert steps == sorted(steps), \
112                  f"{cond} must emit chronologically, got {steps}"
113
114
115  def test_retrieval_memories_can_opt_into_chronological_order(llm):
116      """M1/M4 must be able to produce chronological prompts on request.
117
118      This is what makes the ordering confound testable: run M1/M4 twice, once
119      each way, and the difference is attributable to layout alone.
120      """
121      from agentmem.memory import make_memory
122
123      for cond in ("m1", "m4"):
124          mem = make_memory(cond, {"chronological_order": True}, llm=llm)
125          for i in range(30):
126              mem.write(Turn(role="system",
127                             content=f"Update: project Nimbus milestone {i} logged.",
128                             step=i))
129          ctx = mem.read("What milestone was logged?", budget_tokens=300)
130          steps = [p["step"] for p in ctx.provenance]
131          assert steps == sorted(steps), \
132              f"{cond} with chronological_order=True must emit oldest-first, got {steps}"
133          assert ctx.text.split("\n")[0].endswith(f"milestone {steps[0]} logged."), \
134              f"{cond}: text and provenance must be reordered together"
135
136
137  def test_m3_chrono_is_a_layout_only_control(llm):
138      """m3_chrono must differ from m3 in prompt ORDER and nothing else.
139
140      The arm was originally run via an HPC-side patch (job 2848922); this test
141      pins the restored in-repo implementation: same fact store, same selection,
142      same token count — only the emitted layout changes (oldest-first by
143      earliest source step).
144      """
145      from agentmem.memory import make_memory
146
147      def run(cond):
148          mem = make_memory(cond, {"chronological_order": cond.endswith("chrono")},
149                            llm=llm)
150          for i in range(30):
151              mem.write(Turn(role="system",
152                             content=f"Update: project Nimbus milestone {i} logged.",
153                             step=i))
154          return mem.read("What milestone was logged?", budget_tokens=300)
155
156      score_ordered = run("m3")
```

```
    chrono = run("m3_chrono")
    assert score_ordered.provenance, "m3 must extract facts under MockLLM"
    assert [p["step"] for p in chrono.provenance] == \
        sorted(p["step"] for p in chrono.provenance), \
        "m3_chrono must emit oldest-first by earliest source step"
    assert {p["steps"][0] for p in chrono.provenance} == \
        {p["steps"][0] for p in score_ordered.provenance}, \
        "m3_chrono must select the SAME facts as m3"
    assert chrono.tokens == score_ordered.tokens, \
        "layout control must not change the token count"


def test_b_reversed_is_ordering_only_control():
    """b_reversed must differ from b in prompt ORDER and nothing else.

    If selection or token count diverged, the arm would no longer isolate the
    ordering effect and could not be subtracted from the M2 contrast.
    """
    from agentmem.memory import make_memory

    turns = [Turn(role="user", content=f"turn {i} " + ("padding " * i), step=i)
             for i in range(40)]
    for budget in (120, 400):
        out = {}
        for cond, params in [("b", {}), ("b_reversed", {"chronological_order": False})]:
            mem = make_memory(cond, params)
            for t in turns:
                mem.write(t)
            out[cond] = mem.read("q?", budget_tokens=budget)

        b, rev = out["b"], out["b_reversed"]
        assert sorted(p["step"] for p in b.provenance) == \
               sorted(p["step"] for p in rev.provenance), \
            f"budget={budget}: control arm must select the same turns as b"
        assert b.tokens == rev.tokens, \
            f"budget={budget}: control arm must spend the same read budget as b"

        b_steps = [p["step"] for p in b.provenance]
        rev_steps = [p["step"] for p in rev.provenance]
        assert len(b_steps) > 1, "need >1 turn for ordering to be observable"
        assert b_steps == sorted(b_steps), "b must be chronological"
        assert rev_steps == sorted(rev_steps, reverse=True), \
            "b_reversed must be reverse-chronological"
        assert b.text != rev.text, "the two arms must differ in prompt layout"


def test_provenance_is_not_truncated_by_the_runner():
    """Regression: runner.py logged provenance[:20] AFTER sorting it.

    Because B sorts ascending and the reverse-chronological arms sort
    descending, truncating to the first 20 entries made the two conditions
    appear to select disjoint turn sets (Jaccard 0.0 on every item) when in
    fact they select identical sets. Any ordering/inversion audit run against
    truncated provenance is invalid. Pin the full-provenance contract.
    """
    import inspect
    from agentmem.harness import runner

    src = inspect.getsource(runner)
    assert 'ctx_info["provenance"][:20]' not in src, (
        "runner must log FULL provenance; truncation silently corrupts the "
        "prompt-order audit (see scripts/audit_prompt_order.py)"
    )


def test_b_reversed_is_an_ordering_only_control():
    """b_reversed must differ from b in LAYOUT ONLY, never in selection."""
    from agentmem.memory.base import Turn
    from agentmem.memory.none import NoMemory

    turns = [Turn(step=i, role="system", content=f"Turn {i}: note number {i} here.")
             for i in range(60)]

    def read(chrono):
        m = NoMemory(chronological_order=chrono)
        for t in turns:
            m.write(t)
        return m.read("what is the budget?", 400)
```

```

    a, b = read(True), read(False)
    sa = [p["step"] for p in a.provenance]
    sb = [p["step"] for p in b.provenance]
    assert set(sa) == set(sb), "selection must be identical"
    assert a.tokens == b.tokens, "token count must be identical"
    assert sb == list(reversed(sa)), "layout must be exactly reversed"


def test_all_modules_import_under_python38():
    """Regression: CERES serves vLLM under Python 3.8.

    PEP 585 generics (list[dict]) and PEP 604 unions (dict | None) are
    evaluated at import time on 3.8 and raise TypeError unless the module
    opts into lazy annotations. A code upload once clobbered a hand-applied
    cluster-side fix and every grid cell died on import. Pin the contract.
    """
    import os
    import re

    root = os.path.join(os.path.dirname(__file__), "..")
    offenders = []
    for top in ("src", "scripts", "analysis"):
        for dirpath, dirnames, filenames in os.walk(os.path.join(root, top)):
            dirnames[:] = [d for d in dirnames
                           if d not in ("__pycache__", ".pytest_cache")]
            for fn in filenames:
                if not fn.endswith(".py"):
                    continue
                path = os.path.join(dirpath, fn)
                with open(path, encoding="utf-8") as fh:
                    src = fh.read()
                if "from __future__ import annotations" in src:
                    continue
                if re.search(r"->\s*(list|dict|set|tuple|type)\[|"
                             r":\s*(list|dict|set|tuple)\[|\|\s*None", src):
                    offenders.append(os.path.relpath(path, root))
    assert not offenders, (
        "these modules use 3.9+ annotation syntax without "
        "`from __future__ import annotations` and will fail on CERES: "
        + ", ".join(sorted(offenders))
    )
```

```python
"""Metrics/eval are pure functions over logged data — test them as such."""

import math

from agentmem.eval.failures import tag_failure
from agentmem.eval.judge import exact_match
from agentmem.eval.recall import evidence_metrics
from agentmem.tasks.custom_ds import generate_episode, grade_answer
from agentmem.utils.costs import BudgetExceededError, CostTracker
from agentmem.utils.tokens import count_tokens, truncate_to_tokens


def test_evidence_metrics():
    prov = [{"step": 1}, {"step": 5}, {"kind": "fact", "steps": [7, 9]}]
    m = evidence_metrics(prov, evidence_steps=[5, 7, 100])
    assert math.isclose(m["evidence_recall"], 2 / 3)
    assert math.isclose(m["evidence_precision"], 2 / 4)
    assert evidence_metrics(prov, [])["evidence_recall"] is None


def test_exact_match_normalisation():
    assert exact_match("The deadline is 14 July 2026.", "14 July 2026")
    assert exact_match("£250k", "250k")
    assert not exact_match("I don't know.", "14 July 2026")


def test_failure_tags():
    assert tag_failure(True, "x", 1.0) == "success"
    assert tag_failure(False, "", 1.0) == "no_answer"
    assert tag_failure(False, "wrong", 0.0) == "retrieval_miss"
    assert tag_failure(False, "wrong", 0.5) == "partial_retrieval"
    assert tag_failure(False, "wrong", 1.0) == "reasoning_failure"
    assert tag_failure(False, "old value", 1.0, stale_hit=True) == "stale_fact"


def test_custom_task_deterministic_and_answerable():
    ep1 = generate_episode("e", seed=42)
    ep2 = generate_episode("e", seed=42)
    assert [t.content for t in ep1.turns] == [t.content for t in ep2.turns]
    assert ep1.queries, "episode must contain queries"
    for q in ep1.queries:
        assert q.evidence_steps and max(q.evidence_steps) < q.step
        # Gold value really is stated at the evidence step.
        assert any(q.answer in ep1.turns[s].content for s in q.evidence_steps)
    assert grade_answer(f"The answer is {ep1.queries[0].answer}.",
                        ep1.queries[0].answer)


def test_tokens_and_truncate():
    text = "word " * 400
    assert count_tokens(text) > 100
    cut = truncate_to_tokens(text, 50)
    assert count_tokens(cut) <= 50


def test_cost_kill_switch():
    t = CostTracker(price_key="groq/llama-3.3-70b-versatile", gbp_cap=0.0001)
    try:
        t.add(1_000_000, 1_000_000)
        raise AssertionError("kill-switch did not fire")
    except BudgetExceededError:
        pass
```

```python
"""Statistical analysis over JSONL run records.

Protocol:
- Primary (binary outcomes): pairwise condition contrasts on task_success via
  EXACT McNEMAR over per-item paired outcomes, with Holm correction across the
  declared contrast family. Wilcoxon signed-rank is retained only for
  continuous metrics — on 0/1 data every non-zero difference has |d| = 1, the
  ranks are fully tied, and the signed-rank statistic degenerates. McNemar is
  the correct paired test for dichotomous outcomes and conditions only on the
  discordant pairs.
- Effect size + uncertainty: risk difference with a Wilson-style bootstrap CI,
  plus the paired odds ratio (b/c on the discordant cells) for binary outcomes.
- Cost (RQ3): the SAME machinery applied to token and latency columns, so cost
  is reported with the statistical care Ch2 §2.5 promises rather than as bare
  means. Continuous metrics use Wilcoxon signed-rank, which is appropriate
  there, plus a paired bootstrap CI on the mean difference.
- Secondary: logistic regression success ~ condition (+ question_type in
  Mode A, + evidence_distance in Mode B), reported as odds ratios with CIs.
- RQ4: failure-tag crosstab per condition.

Every p-value that enters the dissertation must pass through holm_correct()
with its family declared — see DECLARED_FAMILIES below.
"""

from __future__ import annotations

import numpy as np
import pandas as pd
from scipy import stats as sps

# Cost metrics. total_prompt_tokens/total_completion_tokens are the P3-metered
# columns that include the write path (extraction calls, memory writes);
# prompt_tokens counts only the answering call and UNDERSTATES M2/M3, which is
# what produced the withdrawn "45% saving" claim. Prefer the total_* columns and
# fall back only when re-runs have not yet produced them.
COST_METRICS = ("total_prompt_tokens", "total_completion_tokens", "latency_ms")
COST_METRICS_LEGACY = ("prompt_tokens", "completion_tokens", "latency_ms")


def records_to_df(records: list[dict]) -> pd.DataFrame:
    df = pd.DataFrame(records)
    if not df.empty:
        df["success"] = df["success"].astype(int)
    return df


def paired_item_means(df: pd.DataFrame, cond_a: str, cond_b: str,
                      metric: str = "success") -> pd.DataFrame:
    """Per-item means (across seeds) of `metric` for two conditions, inner-joined.

    The inner join is what makes the test PAIRED. Reporting an unpaired contrast
    as paired is exactly the B4 error (Table 4.4 paired a 341-record baseline
    against a 120-record M1); `n_items` returned here is the true matched n and
    is the only n that may be quoted alongside a paired test.
    """
    piv = (df[df.condition.isin([cond_a, cond_b])]
           .groupby(["item_id", "condition"])[metric].mean().unstack())
    return piv.dropna(subset=[cond_a, cond_b])


def mcnemar_contrast(df: pd.DataFrame, cond_a: str, cond_b: str,
                     metric: str = "success") -> dict:
    """EXACT McNemar on paired binary outcomes — the primary success test.

    Items are dichotomised at 0.5 when averaged over seeds (a tie contributes to
    neither discordant cell). The exact binomial form is used rather than the
    chi-square approximation because the discordant counts here are small.

    Returns b/c (the discordant cells), the paired odds ratio b/c, and the risk
    difference. Note the p-value tests the discordant pairs only; the risk
    difference is the effect size and should always be reported with it.
    """
    piv = paired_item_means(df, cond_a, cond_b, metric)
    a = (piv[cond_a].to_numpy() > 0.5).astype(int)
    b_ = (piv[cond_b].to_numpy() > 0.5).astype(int)
    n01 = int(((a == 0) & (b_ == 1)).sum())   # only cond_b succeeds
    n10 = int(((a == 1) & (b_ == 0)).sum())   # only cond_a succeeds
    n_disc = n01 + n10
```

```
    p = 1.0 if n_disc == 0 else float(
        sps.binomtest(n10, n_disc, 0.5, alternative="two-sided").pvalue)
    odds = float("inf") if n01 == 0 and n10 > 0 else (
        0.0 if n10 == 0 and n01 > 0 else (float(n10 / n01) if n01 else float("nan")))
    return {"contrast": f"{cond_a} vs {cond_b}", "test": "exact McNemar",
            "n_items": len(piv), "n_discordant": n_disc,
            "n_only_a": n10, "n_only_b": n01,
            "risk_diff": float(piv[cond_a].mean() - piv[cond_b].mean()),
            "odds_ratio": odds, "p": p}


def wilcoxon_contrast(df: pd.DataFrame, cond_a: str, cond_b: str,
                      metric: str = "success") -> dict:
    """Paired Wilcoxon signed-rank on any metric column.

    CORRECT for continuous metrics (tokens, latency). For binary success use
    mcnemar_contrast(): on 0/1 data all non-zero |d| equal 1, ranks tie
    completely, and the signed-rank statistic degenerates.
    """
    piv = paired_item_means(df, cond_a, cond_b, metric)
    a, b = piv[cond_a].to_numpy(), piv[cond_b].to_numpy()
    diff = a - b
    base = {"contrast": f"{cond_a} vs {cond_b}", "metric": metric,
            "test": "Wilcoxon signed-rank", "n_items": len(piv)}
    if len(piv) == 0 or np.allclose(diff, 0):
        return {**base, "mean_diff": 0.0, "median_diff": 0.0, "W": None, "p": 1.0}
    W, p = sps.wilcoxon(a, b, zero_method="wilcox", alternative="two-sided")
    return {**base, "mean_diff": float(diff.mean()),
            "median_diff": float(np.median(diff)), "W": float(W), "p": float(p)}


def bootstrap_diff_ci(
    df: pd.DataFrame, cond_a: str, cond_b: str, metric: str = "success",
    n_boot: int = 10_000, seed: int = 1, alpha: float = 0.05,
    cluster: str | None = None,
) -> dict:
    """Paired bootstrap CI on the mean difference in `metric`.

    `cluster` resamples whole clusters (e.g. "episode_id") instead of items.
    Item-level resampling treats queries from the same episode as independent
    when they are not, which narrows the interval — the same pseudo-replication
    that inflated the Mode A CI in B4. Pass cluster="episode_id" for Mode B.
    """
    piv = paired_item_means(df, cond_a, cond_b, metric)
    diff = (piv[cond_a] - piv[cond_b])
    rng = np.random.default_rng(seed)

    if cluster and cluster in df.columns:
        key = (df.drop_duplicates("item_id").set_index("item_id")[cluster]
                .reindex(diff.index))
        groups = [g.to_numpy() for _, g in diff.groupby(key)]
        k = len(groups)
        boots = np.array([
            np.concatenate([groups[i] for i in rng.integers(0, k, k)]).mean()
            for _ in range(n_boot)])
    else:
        d = diff.to_numpy()
        n = len(d)
        boots = np.array([d[rng.integers(0, n, n)].mean() for _ in range(n_boot)])

    lo, hi = np.percentile(boots, [100 * alpha / 2, 100 * (1 - alpha / 2)])
    return {"contrast": f"{cond_a} vs {cond_b}", "metric": metric,
            "mean_diff": float(diff.mean()), "ci_low": float(lo),
            "ci_high": float(hi), "n_items": len(diff),
            "resampled": cluster or "item"}


def cost_contrasts(df: pd.DataFrame, baseline: str = "b",
                   conditions: list[str] | None = None,
                   metrics: tuple[str, ...] | None = None) -> pd.DataFrame:
    """RQ3: paired tests + CIs on cost metrics, Holm-corrected WITHIN metric.

    This is what Ch2 §2.5 promises ("cost reported with the same statistical
    care as task success") and what Table 4.1 currently does not deliver.

    Holm families are per-metric: correcting tokens and latency jointly would
    be over-conservative, since they are separate pre-declared questions.
    """
```

```
    if metrics is None:
        # Pick the metered set only if the P3 total_* columns are actually
        # present. Testing membership metric-by-metric would silently select
        # just latency_ms (it appears in both tuples) and drop the token tests.
        metered = [m for m in COST_METRICS if m in df.columns]
        use_metered = "total_prompt_tokens" in df.columns
        metrics = tuple(metered) if use_metered else \
            tuple(m for m in COST_METRICS_LEGACY if m in df.columns)
    conds = conditions or [c for c in sorted(df.condition.unique()) if c != baseline]

    rows: list[dict] = []
    for metric in metrics:
        if metric not in df.columns or df[metric].isna().all():
            continue
        fam = [wilcoxon_contrast(df, c, baseline, metric) for c in conds]
        fam = holm_correct(fam)
        for r, c in zip(fam, conds):
            ci = bootstrap_diff_ci(df, c, baseline, metric)
            base_mean = df[df.condition == baseline][metric].mean()
            rows.append({**r, "ci_low": ci["ci_low"], "ci_high": ci["ci_high"],
                         "pct_change": (100.0 * r["mean_diff"] / base_mean
                                        if base_mean else float("nan"))})
    return pd.DataFrame(rows)


def success_contrasts(df: pd.DataFrame, baseline: str = "b",
                      conditions: list[str] | None = None,
                      cluster: str | None = None) -> pd.DataFrame:
    """RQ1/RQ2: the primary success family — exact McNemar + risk difference + CI.

    One declared Holm family covering every condition-vs-baseline contrast in
    this mode. Use this instead of assembling contrasts by hand, so nothing
    escapes correction (the previous family covered only the four Mode B
    contrasts and left Mode A and the logit coefficients uncorrected).
    """
    conds = conditions or [c for c in sorted(df.condition.unique()) if c != baseline]
    fam = holm_correct([mcnemar_contrast(df, c, baseline) for c in conds])
    rows = []
    for r, c in zip(fam, conds):
        ci = bootstrap_diff_ci(df, c, baseline, "success", cluster=cluster)
        rows.append({**r, "ci_low": ci["ci_low"], "ci_high": ci["ci_high"],
                     "resampled": ci["resampled"]})
    return pd.DataFrame(rows)


def per_seed_table(df: pd.DataFrame, metric: str = "success") -> pd.DataFrame:
    """Per-seed means per condition + the spread across seeds (A5).

    Seed spread on Mode A reached 16.7pp — larger than several of the reported
    effects — so a single-seed row cannot be presented as a condition estimate.
    Report `spread` next to any effect it exceeds.
    """
    piv = df.groupby(["condition", "seed"])[metric].mean().unstack()
    piv.columns = [f"seed_{c}" for c in piv.columns]
    piv["mean"] = piv.mean(axis=1)
    piv["spread"] = piv.filter(like="seed_").max(axis=1) - \
        piv.filter(like="seed_").min(axis=1)
    piv["n_seeds"] = df.groupby("condition")["seed"].nunique()
    return piv.round(4)


def holm_correct(results: list[dict]) -> list[dict]:
    """Holm-Bonferroni over a DECLARED family of contrast results.

    The family is whatever list you pass. Declare it explicitly in Ch3 and keep
    Mode A, Mode B and the logit coefficients in separate declared families —
    do not silently omit a contrast from the family it belongs to.
    """
    m = len(results)
    order = sorted(range(m), key=lambda i: results[i]["p"])
    out = [dict(r) for r in results]
    running_max = 0.0
    for rank, i in enumerate(order):
        adj = min(1.0, (m - rank) * results[i]["p"])
        running_max = max(running_max, adj)
        out[i]["p_holm"] = running_max
        out[i]["significant_05"] = running_max < 0.05
    return out
```

```
235
236
237 def logistic_regression(df: pd.DataFrame, extra_terms: list[str] | None = None,
238                          cluster: str | None = None):
239     """success ~ C(condition, Treatment('b')) [+ extras]. Returns fitted model.
240
241     `cluster` (e.g. "episode_id") switches to cluster-robust standard errors.
242     Mode B queries from the same episode share a history and are not
243     independent; plain SEs are too narrow.
244     """
245     import statsmodels.formula.api as smf
246
247     terms = ["C(condition, Treatment('b'))"] + (extra_terms or [])
248     formula = "success ~ " + " + ".join(terms)
249     model = smf.logit(formula, data=df)
250     if cluster and cluster in df.columns:
251         return model.fit(disp=False, cov_type="cluster",
252                          cov_kwds={"groups": df[cluster]})
253     return model.fit(disp=False)
254
255
256 def odds_ratio_table(model, alpha: float = 0.05) -> pd.DataFrame:
257     """Coefficients as odds ratios with CIs — the effect size Ch4 is missing.
258
259     Ch4 currently reports bare log-odds/mean differences. An examiner will ask
260     for the effect size and its interval, not just the p-value.
261     """
262     ci = model.conf_int(alpha=alpha)
263     out = pd.DataFrame({
264         "coef": model.params,
265         "odds_ratio": np.exp(model.params),
266         "or_ci_low": np.exp(ci[0]),
267         "or_ci_high": np.exp(ci[1]),
268         "p": model.pvalues,
269     })
270     return out.round(4)
271
272
273 def wilson_ci(successes: int, n: int, alpha: float = 0.05) -> tuple[float, float]:
274     """Wilson score interval for a single proportion.
275
276     Preferred over the normal approximation at the rates in this study: Mode A
277     baseline success is ~0.03, where a Wald interval can cross zero.
278     """
279     if n == 0:
280         return (float("nan"), float("nan"))
281     z = sps.norm.ppf(1 - alpha / 2)
282     p = successes / n
283     denom = 1 + z**2 / n
284     centre = (p + z**2 / (2 * n)) / denom
285     half = z * np.sqrt(p * (1 - p) / n + z**2 / (4 * n**2)) / denom
286     return (float(max(0.0, centre - half)), float(min(1.0, centre + half)))
287
288
289 def failure_crosstab(df: pd.DataFrame) -> pd.DataFrame:
290     return pd.crosstab(df["condition"], df["failure_tag"], normalize="index").round(3)
291
292
293 def summary_table(df: pd.DataFrame) -> pd.DataFrame:
294     """Headline per-condition table: success (with Wilson CI), recall, cost.
295
296     Cost columns: `prompt_tokens` is the ANSWERING call only and understates any
297     condition with a write path — quoting it as total cost is what produced the
298     withdrawn 45% saving. `total_prompt_tokens` (P3-metered) includes extraction
299     and write calls and is the column Ch3's definition actually refers to. Both
300     are shown so the gap is visible rather than hidden.
301     """
302     spec = {
303         "n": ("success", "size"),
304         "success_rate": ("success", "mean"),
305         "evidence_recall": ("evidence_recall", "mean"),
306         "prompt_tokens_answer_only": ("prompt_tokens", "mean"),
307         "completion_tokens": ("completion_tokens", "mean"),
308         "memory_written_tokens": ("memory_written_tokens", "mean"),
309         "overhead_calls": ("overhead_calls", "mean"),
310         "latency_ms": ("latency_ms", "mean"),
311     }
312     if "total_prompt_tokens" in df.columns:
```

```
        spec["total_prompt_tokens"] = ("total_prompt_tokens", "mean")
        spec["total_llm_calls"] = ("total_llm_calls", "mean")
    spec = {k: v for k, v in spec.items() if v[0] in df.columns}
    agg = df.groupby("condition").agg(**spec)

    ci = df.groupby("condition")["success"].agg(["sum", "size"]).apply(
        lambda r: wilson_ci(int(r["sum"]), int(r["size"])), axis=1)
    agg["success_ci_low"] = [c[0] for c in ci]
    agg["success_ci_high"] = [c[1] for c in ci]
    return agg.round(3).sort_index()
```

```yaml
# Base config — model, temperature, seeds, budgets, paths.
# No magic constants in code; everything experiment-relevant lives here.
# provider: mock (dev, CPU laptop) | university_hpc (vLLM server) | groq | openrouter
# NOTE: budget/model values are PILOT DEFAULTS — freeze after the W3 pilot and
# record the frozen values in the manifest + dissertation.

provider: mock
model: ""                    # e.g. meta-llama/Llama-3.1-8B-Instruct on HPC; FIXED across conditions
temperature: 0.0
seeds: [1, 2, 3]

budgets:
  memory_read_tokens: 1200   # DP1 Equal-Budget — identical for every condition
  max_steps_per_item: 200
  gbp_cap: 30                # kill-switch (CostTracker raises BudgetExceededError)

paths:
  data: data/
  results: results/runs/
```

```
1  # B — NoMemory baseline: recency window fitted to the shared budget.
2  condition: b
3  params: {}
```

```yaml
# b_reversed — ORDERING-ONLY CONTROL ARM (not part of the registered main grid).
#
# WHY THIS ARM EXISTS
# Before the P1 patch, SummaryMemory.read() returned turns newest-first and never
# restored chronology, while NoMemory.read() did. On Mode B histories (mean ~1171
# tokens) the old 1500-token fold threshold never fired, so M2 never summarised
# anything: the ONLY difference between M2 and B was reverse-chronological prompt
# ordering. The 0.744 -> 0.626 deficit originally attributed to hierarchical
# summarisation was therefore an ordering artefact.
#
# Rather than discard that observation, this arm measures it deliberately.
# b_reversed is byte-identical to b in turn SELECTION (recency-first against the
# same capped read budget) and differs only in the ORDER those turns are laid out
# in the prompt. Any success-rate difference between b and b_reversed is
# attributable to prompt ordering alone.
#
# This also pre-empts the obvious examiner question on the corrected M2 result:
# "how do you know M2's deficit isn't just ordering?" — because the ordering
# effect is measured separately here and can be subtracted.
#
# Report as an incidental finding in Ch4 §4.6, not as a memory architecture.
condition: b_reversed
params:
  chronological_order: false   # skip the chronological re-sort in NoMemory.read()
```

```yaml
# M1 — Episodic retrieval. retriever: tfidf (CPU) | embedding (HPC).
condition: m1
params:
  retriever: tfidf
```

```
# m1_chrono — MITIGATION ARM. Identical to m1 in every respect except that the
# retrieved turns are re-sorted into chronological order before filling the
# prompt slot (retrieval.reorder_chronologically).
#
# Selection, token count and read budget are unchanged, so m1 vs m1_chrono is a
# single-factor manipulation of layout — the same contrast as b vs b_reversed,
# applied to a score-ordered architecture. Tests whether the ordering tax
# measured on the baseline can be removed from a retrieval memory for free.
condition: m1_chrono
params:
  retriever: tfidf
  chronological_order: true
```

```
# M2 — Hierarchical summary (MemGPT-style). Overhead LLM calls charged to condition.
#
# THRESHOLD RATIONALE (record this in Ch3 alongside DP1):
# The fold threshold must sit BELOW the shared read budget, otherwise the
# summariser can only fire on histories the baseline could not have read
# anyway, and the condition silently degenerates to the baseline.
# Sweep REGENERATED 3 Aug 2026 against the canonical post-fix n=219 panel
# (results/rerun_2026_07_29/custom_ds_merged.jsonl, condition 'b',
# memory_written_tokens). The figures below previously cited the superseded
# 258-item pre-provenance-fix panel AND misreported the 1200 row as 0/258.
# Mode B histories on the canonical panel: min 872, median 1168, mean 1170,
# max 1453 tokens.
#   1500 -> fires on   0/219 items (0%)    <- the broken pilot setting: above
#                                             every history in the panel
#   1200 -> fires on  92/219 items (42%)   selective => M2 becomes a mixture
#   1000 -> fires on 185/219 items (84%)   selective => M2 becomes a mixture
#    800 -> fires on 219/219 items (100%)  <- chosen: fires on every item,
#                                             still well below the 1200 budget
condition: m2
params:
  buffer_token_threshold: 800    # fold oldest raw turns into summary above this
  fold_fraction: 0.5             # fraction of buffer folded per summarisation
```

```
1  # M3 — Structured semantic fact store (custom, Mem0-style update-on-conflict).
2  condition: m3
3  params:
4    retriever: tfidf
5    extract_every: 5              # turns per extraction batch (1 overhead call each)
```

```
# m3_chrono — MITIGATION ARM for the semantic fact store. See m1_chrono.yaml.
# Identical to m3 except that the selected facts are re-sorted oldest-first
# (by earliest source step) before filling the prompt slot
# (retrieval.reorder_chronologically).
#
# Selection, token count and read budget are unchanged, so m3 vs m3_chrono is a
# single-factor manipulation of layout for the fact-store architecture. Result
# (HPC job 2848922): success did NOT recover (0.753 -> 0.740, n.s.) — M3's
# ordering damage is incurred at extraction (write) time, not read time. See
# supervision/ADDENDUM_m3_chrono_2026-07-30.md.
condition: m3_chrono
params:
  retriever: tfidf
  extract_every: 5              # turns per extraction batch (1 overhead call each)
  chronological_order: true
```

```yaml
# M4 — Retrieval + exponential recency decay (Generative-Agents formula).
condition: m4
params:
  retriever: tfidf
  decay_lambda: 0.01            # per-step decay rate; sweep in pilot if time allows
```

```yaml
# m4_chrono — MITIGATION ARM for the decay condition. See m1_chrono.yaml.
# Decisive for Ch5: M4's deficit (-0.054) is currently not separable from its
# inversion rate (0.430). Holding decay fixed while restoring chronology
# attributes the deficit to the mechanism or to the layout, and nothing else.
condition: m4_chrono
params:
  retriever: tfidf
  decay_lambda: 0.01
  chronological_order: true
```

```yaml
# Custom long-horizon Project-Tracker task (Mode B) — deterministic generator.
task: custom_ds
n_episodes: 10
n_steps: 120
n_projects: 6
n_queries: 10
update_prob: 0.25
distractor_prob: 0.35
```

```
1  # LongMemEval _s subset (Mode A) — stratified capping by question_type (14 Jun decision).
2  task: longmemeval_s
3  data_file: data/longmemeval_s.json    # download per data/README.md (gitignored)
4  cap: 120                              # capped subset size; stratified by question_type
5  exclude_types: []                     # e.g. [abstention] if descoped later
```

```
# Core (everything in the pipeline runs with these)
python-dotenv>=1.0
pyyaml>=6.0
requests>=2.31
numpy>=1.26
pandas>=2.2
scipy>=1.13
statsmodels>=0.14
matplotlib>=3.8
pytest>=8.0
jupyter>=1.0            # submitted .ipynb
nbformat>=5.9

# Optional — better token counts (heuristic fallback built in)
# tiktoken>=0.7

# Optional — embedding retriever for M1/M4 on HPC (configs: retriever: embedding)
# sentence-transformers>=2.7

# Optional — dataset download helper (see data/README.md)
# huggingface_hub>=0.23
```

```
  1  # agent-memory
  2
  3  Code for the MA981 MSc dissertation **"What Should an Agent Remember? A
  4  Comparative Study of Memory Architectures for Long-Horizon LLM Agents"**
  5  (University of Essex, 2026).
  6
  7  One fixed open-weight LLM is compared under five memory conditions, all
  8  behind the same locked interface (`src/agentmem/memory/base.py`), under the
  9  Equal-Budget Principle (DP1): every condition reads memory with the SAME
 10  token budget — architectures differ only in what they spend it on.
 11
 12  | Cond | Class            | Idea                                            |
 13  |------|------------------|-------------------------------------------------|
 14  | B    | `NoMemory`       | recency window (baseline)                        |
 15  | M1   | `EpisodicMemory` | verbatim turns, relevance retrieval              |
 16  | M2   | `SummaryMemory`  | hierarchical running summary (MemGPT-style)      |
 17  | M3   | `SemanticMemory` | LLM-extracted fact store, update-on-conflict     |
 18  | M4   | `DecayMemory`    | retrieval x exponential recency decay             |
 19
 20  Two evaluation modes: **Mode A** memory probing (LongMemEval subset) and
 21  **Mode B** memory use during execution (custom Project-Tracker task).
 22
 23  ## Quickstart (CPU laptop, zero cost)
 24
 25  ```bash
 26  pip install -r requirements.txt
 27  python -m pytest tests/ -q          # 20 tests, all invariants
 28  python scripts/run_pilot.py         # small all-conditions grid, mock LLM
 29  ```
 30
 31  ## The real run
 32
 33  1. Get data: see `data/README.md` (LongMemEval download).
 34  2. Point at a model: `docs/HPC_RUN_GUIDE.md` (vLLM on HPC; Groq fallback).
 35  3. `python scripts/run_full_grid.py` — one command, resumable; produces
 36     `results/report.md`, `results/figures/`, `results/traces.jsonl`.
 37
 38  Or execute `Run_Experiments.ipynb` top-to-bottom (the FASER submission entry
 39  point does the same steps with visible outputs).
 40
 41  ## Scripts
 42
 43  - `run_experiment.py`  one (task, condition, seed) cell; resumable
 44  - `run_full_grid.py`   whole grid + report + figures + EDA export
 45  - `run_pilot.py`       quick mock-mode sanity grid
 46  - `make_report.py`     stats battery -> results/report.md
 47  - `make_figures.py`    4 dissertation figures -> results/figures/
 48  - `export_for_eda.py`  run records -> EDA notebook live-telemetry format
 49  - `validate_judge.py`  blind human-vs-judge agreement (Cohen's kappa)
 50  - `export_code_pdf.py` machine-readable code PDF (FASER)
 51
 52  ## Layout
 53
 54  ```
 55  src/agentmem/   memory/ (5 conditions + locked ABC)  llm/  harness/  tasks/
 56                  eval/ (recall, judge, failures)  logging/  utils/
 57  configs/        base.yaml (single source of truth) + conditions/ + tasks/
 58  analysis/       stats.py (Wilcoxon+Holm, bootstrap CI, logit, crosstabs)
 59  tests/          pytest suite incl. official-schema LongMemEval fixture
 60  docs/           HPC_RUN_GUIDE.md
 61  results/        runs/*.jsonl (gitignored), report.md, figures/
 62  ```
 63
 64  Reproducibility: fixed seeds throughout, config-driven constants, resumable
 65  JSONL with per-record run ids, manifest (git commit + platform) per run.
```

````
# Data (gitignored — only this README is committed)

## LongMemEval (Mode A)

1. Get `longmemeval_s.json` from the official release
   (https://github.com/xiaowu0162/LongMemEval). On your machine:

   ```bash
   pip install huggingface_hub
   hf download xiaowu0162/longmemeval longmemeval_s.json --repo-type dataset --local-dir data/
   ```

   (If the file is shipped compressed, e.g. `.tar.gz`, extract so the JSON
   sits at the path below. The GitHub README's Google Drive link is the
   fallback if HF layout changes.)
2. Place it at `data/longmemeval_s.json`.
3. The capped, stratified subset
````

```
# Copy to .env and fill in. NEVER commit .env (it's gitignored).

# University HPC (primary): OpenAI-compatible server, e.g. vLLM:
#   vllm serve meta-llama/Llama-3.1-8B-Instruct --port 8000
HPC_BASE_URL=http://localhost:8000/v1
HPC_API_KEY=

# Hosted fallbacks
GROQ_API_KEY=
OPENROUTER_API_KEY=

# hard spend cap in GBP — client kill-switch halts all runs at this value
BUDGET_GBP_CAP=30
```